

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ
MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



KHÓA LUẬN TỐT NGHIỆP

**NGHIÊN CỨU VÀ TRIỂN KHAI MÔ HÌNH
G2MONO-DEPTH BẰNG LIDAR 2D**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên thực hiện: **NGÔ TRẦN QUỐC BẢO**

MSSV - 22139004

NGÔ ĐÌNH THÁI LONG

MSSV - 22139038

LÊ THÀNH NGÂN

MSSV - 22139044

TP. Hồ Chí Minh, Tháng 06, 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ
MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

KHÓA LUẬN TỐT NGHIỆP

**NGHIÊN CỨU VÀ TRIỂN KHAI MÔ HÌNH
G2MONO-DEPTH BẰNG LIDAR 2D**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên thực hiện: **NGÔ TRẦN QUỐC BẢO**

MSSV - 22139004

NGÔ ĐÌNH THÁI LONG

MSSV - 22139038

LÊ THÀNH NGÂN

MSSV - 22139044

Người hướng dẫn khoa học: PGS. TS. Nguyễn Mạnh Hùng

TP. Hồ Chí Minh, Tháng 06, 2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Nghiên Cứu và Triển Khai mô hình G2-MonoDepth bằng Lidar 2D.....

Sinh viên: + Ngô Trần Quốc Bảo
+ Ngô Đình Thái Long
+ Lê Thành Ngân

MSSV: 22139002
MSSV: 22139038
MSSV: 22139044

Hướng dẫn: PGS. TS. Nguyễn Mạnh Hùng

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:
Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.
Đề tài tích hợp nhiều công nghệ, giúp giảm chi phí cho mobile robot.
2. Phương pháp thực hiện/ phân tích/ thiết kế:
Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.
Phương pháp PnP hợp lý, tối ưu đồ thị qua ONNX và biên dịch sang TensorRT Engine trên GPU giúp cải thiện tốc độ tính toán trong khâu vận hành.
3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:
Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.
Kết quả tin cậy, giảm 91,4% sai số AbsRel và đạt tốc độ xử lý thời gian thực ~10,3 FPS ở định dạng TensorRT .
4. Kết luận và đề xuất:
Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.
Kết luận bám sát mục tiêu.
5. Hình thức trình bày và bố cục báo cáo:
Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.
Bố cục 7 chương rõ ràng, văn phong nhất quán đúng quy chuẩn mẫu báo cáo.
6. Kỹ năng chuyên nghiệp và tính sáng tạo:
Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.
Kỹ năng làm việc nhóm tốt trong việc phân tách luồng xử lý trạm biên Raspberry Pi 5 và máy chủ GPU Tuy nhiên chưa đánh giá khả năng xử lý tại biên trên các máy tính Jetson.
7. Tài liệu trích dẫn
Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.
Trích dẫn trung thực và tuân thủ chính xác theo đúng quy định.
8. Đánh giá về sự trùng lặp của đề tài
Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.
Khẳng định đề tài không trùng lặp nhờ đóng góp mới trong việc tự xây dựng Private Dataset (600 mẫu train) từ cảm biến thật.
9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*
Tổng độ trễ End-to-End của hệ thống xử lý phân tán vẫn chịu sự phụ thuộc cục bộ vào băng thông mạng kết nối không dây (TCP/HTTP). Hoặc tính toán ngay tại biên.
Cần hoàn thiện thêm sơ đồ khối tổng thể kết nối phần cứng trực quan ở Chương 4 để tăng tính thuyết phục cao hơn cho quyền báo cáo.
10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Sinh viên có tinh thần chủ động, thái độ nghiên cứu nghiêm túc, đam mê khoa học và hoàn thành khối lượng thực nghiệm lớn đúng tiến độ.

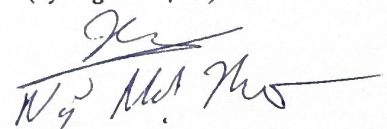
Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 24 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)



* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Ngô Trần Quốc Bảo MSSV: 22139004

Ngô Đình Thái Long MSSV: 22139038

Lê Thành Ngân MSSV: 22139044

1. Tên đề tài: Nghiên cứu và triển khai mô hình G2-MonoDepth bằng LiDAR 2D

2. Các số liệu, tài liệu ban đầu:

- Tài liệu lý thuyết về bài toán hoàn thiện bản đồ độ sâu (Depth Map Completion) và mô hình G2-MonoDepth.
- Phương pháp hiệu chuẩn nội tại camera và hiệu chuẩn ngoại tại Camera-LiDAR.
- Tài liệu tối ưu hóa mô hình đồ thị tĩnh (ONNX, ONNX Simplifier, NVIDIA TensorRT FP16).
- Trọng số tiền huấn luyện từ bộ dữ liệu KITTI của mô hình G2-MonoDepth và tập dữ liệu thực tế tự thu thập.

3. Nội dung thực hiện đề tài:

- Thiết kế, lắp đặt hệ thống phần cứng.
- Thực hiện hiệu chuẩn nội tại camera và hiệu chuẩn ngoại tại LiDAR-Camera.
- Xây dựng bộ dữ liệu thực tế gồm 621 mẫu trong nhà.
- Fine-tuning mô hình G2-MonoDepth.
- Tối ưu hóa mô hình: chuyển đổi PyTorch sang ONNX, tinh giản đồ thị và biên dịch sang TensorRT Engine hỗ trợ FP16 để suy diễn thời gian thực.

4. Sản phẩm:

- Mô hình phần cứng thu thập và đồng bộ không gian cảm biến hoàn chỉnh.
- Bộ dữ liệu thực tế (Private Dataset) gồm 621 mẫu hình học trong nhà.
- Trọng số mô hình G2-MonoDepth đã được tinh chỉnh và tối ưu hóa.
- Mã nguồn chương trình.
- Quyền báo cáo đồ án tốt nghiệp.

TRƯỜNG NGÀNH

GIẢNG VIÊN HƯỚNG DẪN

LỜI CAM ĐOAN

Chúng tôi, Ngô Trần Quốc Bảo, Ngô Đình Thái Long và Lê Thành Ngân, sinh viên ngành Hệ thống nhúng và IoT, khoa Điện - Điện tử, trường Đại học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh, xin cam đoan rằng khóa luận tốt nghiệp với đề tài này là sản phẩm nghiên cứu độc lập và sáng tạo của nhóm. Khóa luận được hoàn thành dựa trên nền tảng kiến thức chuyên môn, kỹ năng nghiên cứu và nỗ lực của các thành viên trong nhóm, không sao chép nội dung hay kết quả của bất kỳ đồ án, bài báo khoa học, hay tài liệu nào khác đã được công bố trước đây. Tất cả nội dung tham khảo từ các nguồn tài liệu khác, bao gồm sách, báo, bài báo khoa học, trang web, v.v., đều được trích dẫn đầy đủ và chính xác theo đúng quy định của trường và các tiêu chuẩn học thuật quốc tế. Chúng tôi cam đoan rằng thông tin trong khóa luận là trung thực và khách quan, không có hành vi gian lận hay đạo văn dưới bất kỳ hình thức nào. Chúng tôi nhận thức rõ hậu quả của việc vi phạm cam đoan này và sẵn sàng chịu trách nhiệm trước hội đồng và nhà trường nếu có bất kỳ sai phạm nào được phát hiện.

Người thực hiện tiểu luận
(Ký và ghi rõ họ tên)

Ngô Trần Quốc Bảo

Ngô Đình Thái Long

Lê Thành Ngân

LỜI CẢM ƠN

Trong quá trình hoàn thành khóa luận tốt nghiệp với đề tài này, chúng tôi đã nhận được rất nhiều sự tư vấn và lời khuyên quý báu từ quý Thầy Cô và bạn bè ngành Hệ thống nhúng và IoT, khoa Điện - Điện tử, Trường Đại học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh. Nhờ đó, nhóm chúng tôi mới có thể hoàn thành đề tài một cách tốt nhất. Chúng tôi xin gửi lời cảm ơn chân thành đến quý Thầy Cô và các bạn đã tận tình góp ý, giúp đỡ trong suốt quá trình thực hiện khóa luận. Đặc biệt, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến Thầy Nguyễn Mạnh Hùng, người đã luôn đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài. Những góp ý, định hướng, đánh giá và lộ trình mà Thầy đề ra từ những ngày đầu thực hiện khóa luận đã mang lại ý nghĩa to lớn, góp phần định hình tư duy và định hướng phát triển của nhóm trong suốt quá trình nghiên cứu. Mặc dù nhóm đã cố gắng hoàn thiện đề tài một cách tốt nhất, song khó tránh khỏi những thiếu sót và hạn chế. Chúng tôi rất mong nhận được những ý kiến đóng góp và nhận xét từ quý Thầy Cô để có thể tiếp tục học hỏi, hoàn thiện kiến thức, và phát triển đề tài tốt hơn trong tương lai. Chúng tôi xin chân thành cảm ơn!

TÓM TẮT

Đề tài này trình bày một phương pháp nghiên cứu và triển khai thực nghiệm dựa trên học sâu cho bài toán hoàn thiện bản đồ độ sâu (Depth Map Completion), tập trung đặc biệt vào việc đồng bộ hình học giữa camera màu và cảm biến LiDAR 2D đơn dòng quét thưa, nhằm đóng góp vào việc giảm giá thành chế tạo và nâng cao năng lực tránh vật cản của các hệ thống robot tự hành cỡ nhỏ.

Về mặt thực tiễn, trong quá trình đo đạc khoảng cách, các phương pháp ước tính độ sâu đơn kính thường gặp hạn chế lớn về mặt thiếu thang đo vật lý chính xác, trong khi các hệ thống LiDAR 3D đắt tiền lại vượt quá chi phí triển khai thực tế. Để khắc phục vấn đề này, chúng tôi các dữ liệu thưa thớt thu được từ cảm biến LiDAR 2D quét vòng (RPLidar A1) sẽ được ánh xạ trực tiếp lên mặt phẳng ảnh thông qua áp dụng phép chiếu PnP (Perspective-n-Point) và bia target chữ V biến dạng.

Để khôi phục bản đồ độ sâu dày từ dữ liệu thưa thớt cực hạn cùng tồn tại trên một khung hình, chúng tôi đề xuất tinh chỉnh mô hình G2-MonoDepth, một mạng mã hóa - giải mã sâu (Encoder-Decoder) xử lý dữ liệu dưới dạng $RGB + X$ (với X là kênh mật độ độ sâu thưa thớt - Sparse Depth), thông qua việc tích hợp bộ tải dữ liệu thực tế chuyên dụng thích ứng tốt hơn với dữ liệu đầu vào và một cấu hình tối ưu hóa tĩnh (Static Shape) giúp mô hình tập trung tính toán ổn định, qua đó bảo toàn không gian không bị lệch tọa độ và loại bỏ các toán tử hình học dư thừa. Thông qua việc tăng cường hiệu suất học các đặc trưng hình học trong bộ tải dữ liệu thực tế tự xây dựng (gồm 600 mẫu huấn luyện và 21 mẫu kiểm thử), sự tinh chỉnh này làm giàu khả năng thích ứng trước các sai số vật lý và nhiễu phản xạ của cảm biến thật, từ đó dẫn đến việc hoàn thiện bản đồ độ sâu dày chính xác hơn trên hình ảnh thực tế.

Kết quả thực nghiệm trên tập kiểm thử thực tế độc lập (21 ảnh) cho thấy mạng nơ-ron sau tinh chỉnh đạt hiệu năng cải thiện rõ rệt so với mô hình gốc (baseline - epoch 100) [1]. Trên môi trường PyTorch, mô hình đề xuất giúp giảm sai số tương đối trung bình AbsRel từ 5,131 xuống còn 0,440 (giảm 91,4%), đồng thời cải thiện sai số RMSE tuyệt đối từ 23,15 mm xuống 14,44 mm. Khi biên dịch sang TensorRT engine để tối ưu hóa phần cứng, mô hình đạt mức RMSE tối ưu là 14,13 mm và AbsRel là 0,384.

Đồng thời, đề tài đề xuất quy trình chuyển đổi đồ thị tĩnh sang định dạng ONNX kết hợp đơn giản hóa đồ thị bằng ONNX Simplifier, sau đó biên dịch tối ưu hóa thành công sang TensorRT Engine hỗ trợ tính toán bán chính xác FP16 trên GPU máy chủ. Giải pháp này không những loại bỏ hoàn toàn các toán tử siêu dữ liệu dư thừa mà còn mang lại bước nhảy vọt về hiệu năng thực

thi. Thời gian suy diễn trung bình của mô hình TensorRT chỉ mất 0,0940 giây/ảnh (tương đương $\sim 10,6$ FPS), nhanh gấp 5,5 lần so với mô hình PyTorch gốc và gấp 16,3 lần so với mô hình ONNX chạy trên CPU, đồng thời bảo toàn cấu trúc hình học sắc nét của vật thể mà không gặp phải sai số lượng hóa đáng kể.

MỤC LỤC

Danh mục hình	ix
Danh mục bảng	x
Các từ viết tắt	xi
1 GIỚI THIỆU ĐỀ TÀI	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu đề tài	2
1.3 Đối tượng và phạm vi nghiên cứu	2
1.4 Phương pháp nghiên cứu	3
1.5 Ý nghĩa khoa học và thực tiễn của đề tài	3
1.6 Cấu trúc của khóa luận	4
2 CƠ SỞ LÝ THUYẾT VÀ CÁC CÔNG NGHỆ LIÊN QUAN	5
2.1 Bài toán hoàn thiện bản đồ độ sâu (Depth Map Completion	5
2.1.1 Định nghĩa bài toán hoàn thiện độ sâu	5
2.1.2 Các thách thức chính trong Depth Map Completion thực tế	6
2.2 Mạng nơ-ron nhận tạo trong Depth Map Completion và G2-MonoDepth model . .	6
2.2.1 Kiến trúc Encoder - Decoder và mạng U-Net	6
2.2.2 Cơ chế ReZero (Residual with Zero initialization)	7
2.2.3 Phân tích G2-MonoDepth model	8
2.3 Công nghệ tối ưu hóa mô hình và xây dựng ứng dụng	9
2.3.1 Định dạng model ONNX và ONNX Runtime	9
2.3.2 Bộ tối ưu hóa hiệu năng cao NVIDIA TensorRT	10
2.4 Các chỉ số đánh giá độ chính xác (Evaluation Metrics)	11
3 CÂN CHỈNH CẢM BIẾN LIDAR 2D VÀ CAMERA BẰNG TARGET CHỮ	
V	13
3.1 Sơ đồ khối phần cứng	13
3.2 Hiệu chuẩn camera (Camera Intrinsic Calibration)	15
3.2.1 Mô hình camera lỗ kim và hiện tượng méo ảnh	15
3.2.2 Thực nghiệm hiệu chuẩn camera	17

3.3	Cân chỉnh thông số ngoại tại giữa LiDAR và Camera (Extrinsic Calibration) . . .	20
3.3.1	Thiết lập hệ tọa độ cảm biến	21
3.3.2	Xác định TVEC và RVEC	22
3.4	Thực nghiệm cân chỉnh cảm biến và đánh giá sai số	29
4	XÂY DỰNG BỘ DỮ LIỆU THỰC TẾ VÀ QUY TRÌNH TIỀN XỬ LÝ . . .	31
4.1	Sự khác biệt giữa dữ liệu giả lập và thực tế	31
4.2	Quy trình thu thập dữ liệu thực tế	32
4.2.1	Thiết bị và bố trí thực nghiệm	32
4.2.2	Script thu thập dữ liệu	33
4.2.3	Cấu trúc dữ liệu thu được	33
4.3	Thiết kế công cụ xử lý dữ liệu chuyên dụng (data_tools_real.py)	34
4.3.1	Đọc và phân tích tọa độ LiDAR từ tập tin JSON	34
4.3.2	Kỹ thuật tăng cường dữ liệu đồng bộ 5 kênh (Spatial Transform)	36
5	FINE-TUNING VÀ TỐI ƯU HÓA MÔ HÌNH	38
5.1	Quy trình huấn luyện Fine-tuning G2-MonoDepth model trên tập dữ liệu thực tế .	39
5.1.1	Thiết lập môi trường và cấu hình huấn luyện	39
5.1.2	Chiến lược thiết lập siêu tham số	42
5.1.3	Cấu hình hàm loss tùy chỉnh và sự thích ứng với dữ liệu thực tế	45
5.1.4	Phân tích kết quả hội tụ	47
5.2	Quy trình tối ưu hóa và biên dịch mô hình inference cho hệ thống	52
5.2.1	Chuyển đổi mô hình PyTorch sang định dạng ONNX	52
5.2.2	Tối ưu hóa bằng ONNX Simplifier	55
5.2.3	Biên dịch sang TensorRT Engine hỗ trợ tối ưu hóa FP16	55
6	THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	58
6.1	Thực nghiệm và đánh giá độ chính xác (Quantitative Accuracy Benchmarking) . .	58
6.1.1	Thiết lập Benchmark so sánh	58
6.1.2	Đánh giá sai số trên tập kiểm thử thực tế	59
6.1.3	Phân tích kết quả cải thiện sai số	62
6.2	Thực nghiệm đánh giá hiệu năng inference (Performance Benchmarking)	63
6.2.1	So sánh tốc độ xử lý (FPS) và thời gian inference của các định dạng mô hình	64
6.2.2	Phân tích độ trễ truyền thông và đo lường tài nguyên hệ thống	66
6.3	Đánh giá chất lượng trực quan (Qualitative Evaluation) qua ứng dụng demo . . .	67
6.3.1	So sánh ảnh độ sâu đầu ra trực quan	67
6.3.2	Đánh giá khả năng bảo toàn ranh giới vật thể và độ trơn tru	69
7	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	70
7.1	Kết luận	70

7.1.1	Các đóng góp chính của đề tài	70
7.1.2	Những hạn chế còn tồn tại	72
7.2	Hướng phát triển	73

DANH MỤC HÌNH

2.2.1 Mô hình đơn giản của kiến trúc luồng Encoder-Decoder	7
2.2.2 Khối ReZero	8
2.2.3 Mô hình G2-MonoDepth	9
2.3.4 Hệ sinh thái ONNX	10
2.3.5 Hệ thống TensorRT	10
3.1.1 Sơ đồ khối calibration cho phần cứng	14
3.1.2 Phần cứng thực hiện calibration	14
3.2.3 Mô hình camera lỗ kim	15
3.2.4 Mặt phẳng ảo của mô hình camera lỗ kim	16
3.2.5 Méo ảnh theo tâm	17
3.2.6 Hiệu chuẩn camera bằng ảnh checkerboard.	17
3.3.7 Target hình chữ V biến dạng	20
3.3.8 Flow hoạt động của label.py	24
3.3.9 Giao diện labeling tools. Cửa sổ bên trái là lidar view và bên phải là camera view	25
3.3.10Giao diện Lidar-Camera fusion, cửa sổ bên trái là Lidar View còn bên phải là cửa sổ ảnh màu RGB + với các điểm được chiếu lên từ Lidar	28
3.4.1Sử dụng realsense để đánh giá	30
4.3.1 Quy trình xử lý dữ liệu từ file JSON của Lidar	35
5.1.1 Biến thiên hàm loss (trái) và sai số chiều sâu tương đối rdepth (phải) trên lô dữ liệu đầu tiên (batch-1) qua 95 epoch huấn luyện tinh chỉnh. Đường nét đứt biểu thị trung bình trượt ($k = 5$), đường chấm biểu thị giá trị trung bình toàn cục. Vùng tô mờ đại diện cho khoảng hội tụ sâu ổn định (từ epoch 70 đến 95).	48
5.1.2 So sánh trị số adepth huấn luyện (trái) và chỉ số rgrad huấn luyện (phải) giữa các trạng thái mô hình tiêu biểu được lựa chọn để phân tích. Cột màu xanh lá đánh dấu mô hình đạt mức adepth và rgrad tối ưu nhất (Epoch 78).	50
6.2.1 Đồ thị giám sát tài nguyên máy chủ hiển thị bộ đệm VRAM trong quá trình suy diễn bằng TensorRT Engine.	66
6.3.2 Ảnh RGB gốc dùng làm bối cảnh đối chứng.	67

6.3.3 Trực quan hóa khả năng bảo toàn ranh giới vật thể sắc nét và duy trì độ trơn trượt bề mặt của mô hình đã được fine-tune (Epoch 57).	69
--	----

DANH MỤC BẢNG

4.1.1 Bảng so sánh dữ liệu ảnh giữa Dataset thực tế và Dataset mô phỏng	31
5.1.1 Thiết lập siêu tham số hệ thống trong quá trình huấn luyện tinh chỉnh sâu.	43
5.1.2 Thống kê tổng hợp hàm loss (batch-1) và sai số rdepth qua các giai đoạn huấn luyện tinh chỉnh.	49
5.1.3 So sánh hiệu năng thực thi và sai số kiểm thử liên nền tảng giữa các định dạng tối ưu hóa.	50
6.1.1 Kết quả đánh giá định lượng và so sánh sai số trên tập kiểm thử thực tế Private_Test (21 mẫu) bằng PyTorch inference trên GPU.	60
6.2.2 So sánh thời gian inference trung bình và tốc độ xử lý (FPS) giữa các định dạng mô hình trên tập kiểm thử độc lập (21 ảnh).	64
6.3.3 Bảng so sánh trực quan bản đồ giả màu độ sâu giữa nhãn chuẩn, mô hình Baseline và mô hình đề xuất tinh chỉnh tại Epoch 57.	68

CÁC TỪ VIẾT TẮT

Các từ viết tắt	Định nghĩa
AbsRel	Absolute Relative Error
CPU	Central Processing Unit
DDP	Distributed Data Parallel
DP	Data Parallel
FOV	Field of View
FP16	Floating Point 16-bit
FP32	Floating Point 32-bit
FPS	Frames Per Second
GPU	Graphics Processing Unit
H-FOV	Horizontal Field of View
HMI	Human-Machine Interface
IoT	Internet of Things
JSON	JavaScript Object Notation
MAE	Mean Absolute Error
MDE	Monocular Depth Estimation
NCCL	NVIDIA Collective Communications Library
OOM	Out of Memory
ONNX	Open Neural Network Exchange
PnP	Perspective-n-Point
RAM	Random Access Memory
RGB	Red, Green, Blue
RGB-D	Red, Green, Blue - Depth
RMSE	Root Mean Squared Error
ROS2	Robot Operating System 2
SLAM	Simultaneous Localization and Mapping
TCP/HTTP	Transmission Control Protocol / Hypertext Transfer Protocol
V-FOV	Vertical Field of View
VRAM	Video Random Access Memory

Chương 1

GIỚI THIỆU ĐỀ TÀI

1.1 LÝ DO CHỌN ĐỀ TÀI

Trong xu thế phát triển của robot dịch vụ và xe tự hành, việc thu nhận Dense Depth Map đóng vai trò quyết định giúp hệ thống nhận biết cấu trúc không gian ba chiều, định vị và lập quỹ đạo di chuyển tránh vật cản. Các giải pháp sử dụng cảm biến chủ động cao cấp như LiDAR 3D nhiều dòng quét mặc dù cung cấp khoảng cách vật lý chính xác cao nhưng lại có giá thành quá đắt đỏ, tiêu thụ nhiều năng lượng và công kênh, gây khó khăn lớn khi triển khai thương mại hóa trên các dòng robot dịch vụ cỡ nhỏ

Ngược lại, phương pháp ước tính độ sâu từ một camera màu (Monocular Depth Estimation - MDE) [2] giúp tối ưu hóa chi phí phần cứng nhưng lại vướng phải hạn chế cốt lõi về mặt toán học là hiện tượng mơ hồ về thang đo. Do thiếu các tham chiếu không gian tuyệt đối, camera đơn kính chỉ có thể dự đoán độ sâu tương đối giữa các vật thể mà không thể xác định được khoảng cách vật lý chính xác tính bằng mét, dẫn đến sự thiếu an toàn và thiếu tin cậy khi đưa vào vận hành hệ thống tự hành ngoài thực tế.

Để giải quyết bài toán cân bằng giữa chi phí phần cứng và độ chính xác hình học, giải pháp kết hợp một camera độ sâu RGB-D phổ thông với một cảm biến Single-line LiDAR giá thành rẻ đã được đề xuất. Trong thiết lập hệ thống lai này, LiDAR 2D chỉ quét một đường duy nhất để cung cấp các điểm đo khoảng cách tuyệt đối đóng vai trò làm neo thang đo, từ đó bài toán hoàn thiện bản đồ độ sâu được đặt ra để khôi phục bản đồ độ sâu dày từ dữ liệu thưa thớt đầu vào.

Tuy nhiên, hầu hết các mạng nơ-ron hoàn thiện độ sâu hiện nay đều được thiết kế cho dữ liệu LiDAR nhiều dòng và đòi hỏi năng lực tính toán rất lớn. Để giải quyết bài toán này mà không làm phình to cấu trúc phần cứng của robot, việc phân tách kiến trúc hệ thống là rất cần thiết: sử dụng một trạm thu nhận ngoại vi nhỏ gọn để đóng gói dữ liệu và truyền về một máy chủ Laptop cấu hình mạnh mẽ, nơi đồ thị tính toán được tối ưu hóa chuyên sâu nhằm chạy suy diễn thời gian thực mượt mà trên bộ vi xử lý đồ họa (GPU). Do đó, đề tài "Nghiên cứu và triển khai mô hình G2Mono-Depth bằng Lidar 2D " được lựa chọn triển khai.

1.2 MỤC TIÊU ĐỀ TÀI

Mục tiêu chính của đề tài là nghiên cứu và hiện thực hóa một hệ thống hoàn thiện độ sâu có chi phí tối ưu, hoạt động ổn định và đạt hiệu năng xử lý cao thông qua 4 mục tiêu cụ thể sau:

Đầu tiên, đề tài hướng tới thiết lập hệ thống hiệu chuẩn và cân chỉnh hình học tích hợp giữa Camera độ sâu và LiDAR 2D. Mục tiêu này bao gồm việc xác định các ma trận Intrinsic (thông số nội tại) của ống kính và ma trận Extrinsic (thông số ngoại tại) giữa hai cảm biến bằng phép chiếu phối cảnh hình học, tạo cơ sở toán học để chiếu chính xác tia quét của LiDAR 2D lên mặt phẳng ảnh pixel màu RGB.

Thứ hai, đề tài tập trung xây dựng một bộ dữ liệu độ sâu thực tế phục vụ huấn luyện và kiểm thử hệ thống. Bộ dữ liệu này được thu thập trong môi trường trong nhà bằng thiết bị thật bao gồm cảm biến RPLidar A1 và camera Intel RealSense D435 đóng vai trò ghi nhận đồng thời ảnh màu RGB và nhẵn độ sâu sau khi đã xử lý lọc nhiễu phần cứng, đạt quy mô tổng thể 600 mẫu huấn luyện và 21 mẫu kiểm thử.

Thứ ba, đề tài thực hiện huấn luyện Fine-tuning mô hình mạng nơ-ron tổng quát G2-MonoDepth. Quá trình này sử dụng bộ dữ liệu thực tế vừa đóng gói để huấn luyện lại mô hình, giúp mạng nơ-ron làm quen với đặc tính phân phối không gian, sai số vật lý và các dạng nhiễu phản xạ bề mặt của cảm biến thật thay vì các dữ liệu giả lập lý thuyết.

Cuối cùng, đề tài hướng đến tối ưu hóa đồ thị tính toán của mạng nơ-ron, chuyển đổi thành công từ framework PyTorch sang định dạng mô hình NVIDIA TensorRT chạy trực tiếp trên lõi đồ họa GPU. Quá trình này áp dụng các kỹ thuật hợp nhất lớp và tối ưu hóa nhân tính toán chuyên sâu để đạt tốc độ suy diễn tối đa, phục vụ kiểm thử đối sánh định lượng và trực quan hóa kết quả đầu ra.

1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Về đối tượng nghiên cứu, đề tài tập trung vào cấu trúc mạng nơ-ron sâu tổng quát G2-MonoDepth[1] xử lý dữ liệu RGB+X, các phương pháp toán học xác định ma trận biến đổi không gian phục vụ cân chỉnh ngoại tại Camera-LiDAR, cùng các kỹ thuật tối ưu hóa đồ thị AI hiệu năng cao trên lõi đồ họa bao gồm chuyển đổi định dạng ONNX trung gian, đơn giản hóa topo mạng và biên dịch sang TensorRT Engine Execution.

Về phạm vi nghiên cứu, thực nghiệm phần cứng được giới hạn trên máy chủ Laptop cấu hình bộ vi xử lý đồ họa GPU NVIDIA (đóng vai trò trung tâm xử lý thuật toán) phối hợp với cụm

thu nhận dữ liệu ngoại vi gồm bo mạch nhúng Raspberry Pi 5, cảm biến LiDAR 2D RPLidar A1 và camera độ sâu chuyên dụng Intel RealSense D435. Phần mềm và công cụ triển khai bao gồm thư viện thị giác máy tính OpenCV, framework ONNX Runtime, và bộ công cụ tăng tốc đồ họa NVIDIA TensorRT.

Để phục vụ cho việc kiểm chứng và tái lập thực nghiệm, toàn bộ mã nguồn của hệ thống (bao gồm các kịch bản thu nhận dữ liệu đồng bộ, công cụ gắn nhãn, chương trình huấn luyện song song phân tán DDP, mã nguồn xuất ONNX tĩnh và kịch bản biên dịch TensorRT Engine) được chúng tôi quản lý và cập nhật công khai tại kho lưu trữ GitHub: <https://github.com/baongongo147/HCMUTE-KLTN-G2-MonoDepth>.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Để đạt được các mục tiêu đề ra, đề tài áp dụng phương pháp nghiên cứu lý thuyết hệ thống kết hợp chặt chẽ với thực nghiệm thực tế:

Giai đoạn đầu tập trung nghiên cứu lý thuyết về hình học camera phối cảnh, nguyên lý đo khoảng cách laser và cấu trúc biểu diễn dữ liệu RGB+X của mạng nơ-ron. Tiếp theo là công tác thực nghiệm phần cứng, thiết kế khung và lập trình đồng bộ không gian giữa các thiết bị cảm biến ngoại vi để thực hiện trích xuất, đóng gói tập dữ liệu thô.

Giai đoạn sau tập trung vào xử lý dữ liệu và huấn luyện tối ưu thuật toán, cụ thể là xây dựng module nạp dữ liệu chuyên dụng và chạy Fine-tuning trên hạ tầng đám mây GPU. Cuối cùng, tiến hành biên dịch tối ưu hóa mô hình sang TensorRT Engine, triển khai thực nghiệm đo đạc trực tiếp các chỉ số sai số toán học, tốc độ khung hình (FPS) trên phần cứng và phân tích định tính các ảnh kết quả độ sâu đầu ra.

1.5 Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN CỦA ĐỀ TÀI

Đề tài chứng minh tính khả thi của phương pháp luận toán học: kết hợp một ảnh màu 2D giàu ngữ cảnh ngữ nghĩa với một dòng quét LiDAR 2D cực thưa để khôi phục chính xác bản đồ độ sâu tuyệt đối mang giá trị mét thực tế. Nghiên cứu cung cấp một giải pháp thực nghiệm giúp loại bỏ hiện tượng mơ hồ thang đo của camera đơn kính, làm sáng tỏ cơ chế lan truyền chiều sâu hình học của mạng nơ-ron dựa trên điểm neo vật lý thưa.

Kết quả của đề tài giúp giảm đáng kể chi phí cấu thành phần cứng nhận thức không gian 3D, mở ra cơ hội ứng dụng đại trà thuật toán Depth Completion trên các dòng robot di động và

robot dịch vụ giá thành thấp. Đồng thời, giải pháp tối ưu hóa mô hình bằng TensorRT chạy trực tiếp trên GPU giúp tận dụng tối đa năng lực xử lý phần cứng có sẵn, tăng tốc độ xử lý đạt chuẩn thời gian thực và đảm bảo tính sẵn sàng triển khai của các ứng dụng robot trong thực tế.

1.6 CẤU TRÚC CỦA KHÓA LUẬN

Chương 1: Giới thiệu đề tài

Chương 2: Cơ sở lý thuyết và các công nghệ liên quan

Chương 3: Cân chỉnh cảm biến Lidar và camera sử dụng target V

Chương 4: Xây dựng bộ dữ liệu thực tế và quy trình tiền xử lý

Chương 5: Fine-tuning và tối ưu hóa mô hình

Chương 6: Thực nghiệm và đánh giá kết quả

Chương 7: Kết luận và hướng phát triển

Chương 2

CƠ SỞ LÝ THUYẾT VÀ CÁC CÔNG NGHỆ LIÊN QUAN

2.1 BÀI TOÁN HOÀN THIỆN BẢN ĐỒ ĐỘ SÂU (DEPTH MAP COMPLETION)

2.1.1 Định nghĩa bài toán hoàn thiện độ sâu

Về mặt toán học, hoàn thiện bản đồ độ sâu là bài toán khôi phục một ma trận mật độ cao, chứa đầy đủ thông tin khoảng cách vật lý của toàn bộ các pixel trong không gian từ nguồn dữ liệu đầu vào thưa thớt. Dữ liệu đầu vào bao gồm:

- Một ảnh màu ma trận chuẩn $I \in \mathbb{R}^{H \times W \times 3}$ cung cấp thông tin ngữ cảnh màu sắc kèm cấu trúc biên cạnh.
- Một bản đồ độ sâu thưa ban đầu $D_{sparse} \in \mathbb{R}^{H \times W}$, trong đó chỉ có một tỷ lệ rất nhỏ các pixel mang giá trị khoảng cách thực từ cảm biến, các pixel còn lại nhận giá trị bằng 0.

Mục tiêu của thuật toán là học được một hàm ánh xạ phi tuyến f thông qua mạng nơ-ron sâu nhằm ước tính bản đồ độ sâu đầy đủ $\hat{D} = f(I, D_{sparse})$ sao cho sai số giữa bản đồ dự đoán này và bản đồ Ground Truth D_{gt} là nhỏ nhất. Khác với bài toán Monocular Depth Estimation thông thường vốn chỉ dựa vào ảnh màu và dễ bị mơ hồ về thang đo do thiếu tham chiếu hình học. Sự hiện diện của ma trận D_{sparse} đóng vai trò là tập hợp các điểm neo toán học. Cơ chế này buộc mạng nơ-ron phải quy đổi các đặc trưng hình học trừu tượng từ ảnh màu về đúng thang đo mét của thế giới thực của thế giới thực.

2.1.2 Các thách thức chính trong Depth Map Completion thực tế

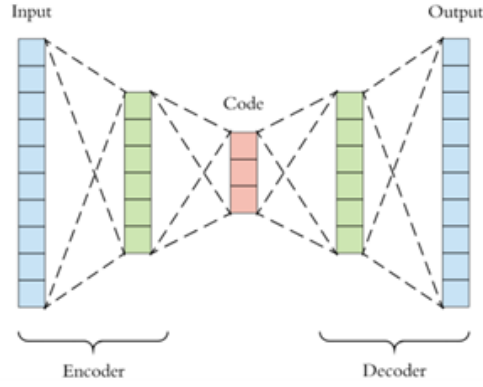
Thách thức lớn nhất khi triển khai Depth Completion trong kịch bản thực tế của đề tài là sự thừa thớt của dòng dữ liệu đầu vào. Trong khi các nghiên cứu tiêu chuẩn thế giới sử dụng dữ liệu LiDAR 3D nhiều dòng quét cung cấp mật độ điểm thô chiếm từ 4% đến 5% diện tích ảnh, hệ thống LiDAR 2D đơn dòng quét vòng trong đề tài này chỉ cung cấp một đường cắt ngang duy nhất với mật độ dữ liệu thấp hơn 0.1%. Điều này khiến phần lớn không gian ma trận độ sâu đầu vào hoàn toàn là các lỗ thủng thông tin rời rạc.

Thách thức thứ hai đến từ các nhiễu hình học và nhiễu vật lý bản chất của cảm biến thật khi vận hành. Phép chiếu hình học từ hệ tọa độ của cảm biến LiDAR quét vòng lên mặt phẳng pixel của camera màu luôn tồn tại một sai số chiếu lại nhất định từ quá trình hiệu chuẩn Extrinsics, dẫn đến việc các điểm độ sâu bị lệch khỏi vị trí biên cạnh thực tế của vật thể. Ngoài ra, bản thân tia quét laser khi gặp các bề mặt hấp thụ ánh sáng, bề mặt phản xạ mạnh hoặc các cạnh góc sắc nét sẽ sinh ra các điểm đo có giá trị bị nhiễu động đột biến, gây hiện tượng "bóng ma độ sâu" nếu mạng nơ-ron không có cơ chế lọc nhiễu tốt.

Thách thức cuối cùng nằm ở bài toán bảo toàn các cạnh biên sắc nét của vật thể trong bản đồ độ sâu độ phân giải cao kết quả. Do bản chất của các phép toán tích chập trong mạng nơ-ron sâu có xu hướng làm mịn xuôi dòng các giá trị pixel lân cận, mô hình rất dễ làm mờ ranh giới phân tách giữa một vật thể ở gần và nền không gian ở xa, sinh ra hiện tượng Bleeding Artifacts. Điều này cực kỳ nguy hiểm cho hệ thống tránh vật cản của robot tự hành, đòi hỏi thuật toán phải tận dụng tối đa thông tin tần số cao từ ảnh màu RGB để hướng dẫn quá trình lan truyền độ sâu một cách chính xác tại các vùng biên đột biến.

2.2 MẠNG NƠ-RON NHẬN TẠO TRONG DEPTH MAP COMPLETION VÀ G2-MONODEPTH MODEL

2.2.1 Kiến trúc Encoder - Decoder và mạng U-Net



Hình 2.2.1: Mô hình đơn giản của kiến trúc luồng Encoder-Decoder

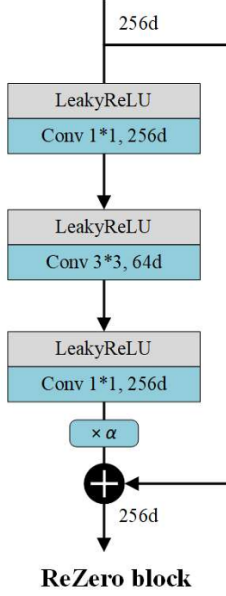
Kiến trúc luồng Encoder-Decoder tích hợp Skip Connections, hay cấu trúc mạng U-Net [3], là nền tảng cốt lõi của hầu hết các mô hình xử lý ảnh dạng image-to-image và hoàn thiện độ sâu hiện đại.

Nhánh Encoder thực hiện các phép toán tích chập kết hợp Down-sampling liên tiếp nhằm nén thông tin đầu vào thành các không gian Multi-scale Features. Quá trình này giúp mô hình nắm bắt được Global Context của bức ảnh và hiểu được mối quan hệ hình học xa giữa các cấu trúc vật thể lớn trong cảnh quan.

Nhánh Decoder đảm nhận nhiệm vụ ngược lại, sử dụng các phép toán nội suy hoặc Up-sampling để khôi phục từng bước độ phân giải không gian của bản đồ độ sâu về kích thước ảnh gốc ban đầu. Để bù đắp lượng thông tin hình học chi tiết và cấu trúc biên cạnh tần số cao bị thất thoát trong các lớp nén của Encoder, Skip Connections được thiết lập để truyền trực tiếp các ma trận đặc trưng từ nhánh mã hóa sang nhánh giải mã ở cùng một cấp độ phân giải. Sự kết hợp này cho phép mạng nơ-ron vừa giữ được hiểu biết ngữ cảnh vĩ mô, vừa tái tạo được các chi tiết độ sâu vi mô cực kỳ sắc nét tại các vùng biên của vật thể.

2.2.2 Cơ chế ReZero (Residual with Zero initialization)

Trong các mạng nơ-ron tích chập có kiến trúc rất sâu, việc lan truyền thông tin đạo hàm một cách ổn định qua hàng chục lớp toán tử luôn đối mặt với hiện tượng Vanishing Gradient hoặc Exploding Gradient. Kiến trúc ResNet truyền thống giải quyết điều này bằng cách sử dụng các khối kết nối tắt dạng cộng thẳng $x + F(x)$. Tuy nhiên, khi số lượng tầng tăng lên quá lớn, sự tích lũy tín hiệu từ các nhánh phụ không được kiểm soát có thể làm thay đổi phân phối kích hoạt của mạng, làm chậm đáng kể tốc độ hội tụ toán học trong quá trình huấn luyện ban đầu.



Hình 2.2.2: Khối ReZero

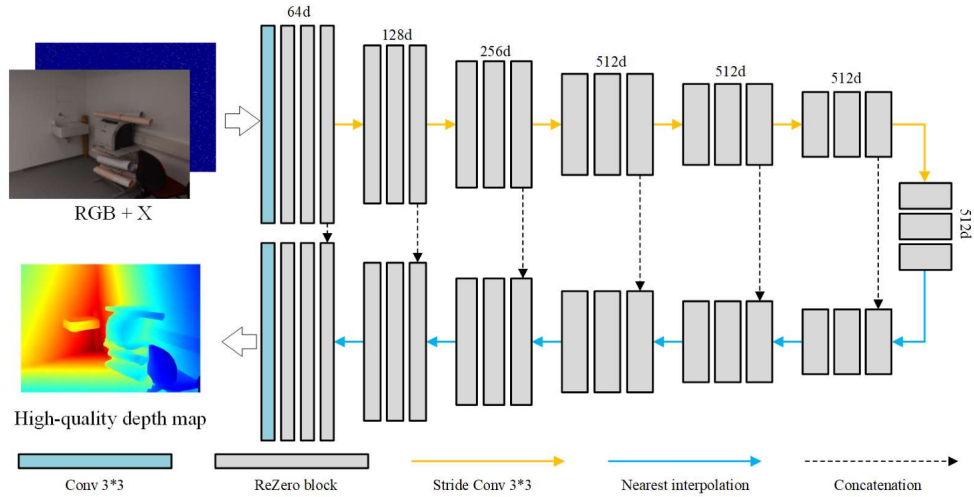
Cơ chế ReZero (Residual with Zero initialization) [4] giải quyết triệt để rào cản này bằng cách chèn một hệ số vô hướng có khả năng học được α vào trước khi thực hiện phép cộng khối residual, biểu diễn dưới dạng công thức toán học:

$$x + \alpha F(x)$$

Giá trị của α được khởi tạo chính xác bằng 0 tại thời điểm bắt đầu quá trình huấn luyện mạng. Điều này có nghĩa là ở các vòng lặp huấn luyện đầu tiên, toàn bộ nhánh chứa trọng số phi tuyến $F(x)$ bị triệt tiêu hoàn toàn, biến mạng nơ-ron sâu thành một hàm đồng nhất tuyến tính thuần túy giúp tín hiệu đạo hàm truyền thẳng 100% qua các tầng mạng mà không hề bị suy giảm, tăng tốc độ hội tụ ổn định cho các bài toán ước tính độ sâu phức tạp.

2.2.3 Phân tích G2-MonoDepth model

Mô hình G2-MonoDepth nổi bật như một khung kiến trúc tổng quát có khả năng xử lý bài toán ước tính độ sâu từ các nguồn dữ liệu hỗn hợp nhờ vào việc chuẩn hóa định dạng dữ liệu đầu vào dưới dạng cấu trúc thống nhất mang tên RGB+X. Ở đây, thành phần "RGB" đại diện cho ảnh màu 3 kênh thông thường, trong khi "X" đại diện cho thông tin độ sâu thô thu được từ bất kỳ loại cảm biến nào. Đối với đề tài này, thành phần X được cấu thành từ hai ma trận bổ trợ nhau: một ma trận độ sâu thưa D_{sparse} chứa dòng quét duy nhất của LiDAR 2D và một ma trận mặt nạ nhị phân lỗ thủng (Hole Mask) $M \in \{0, 1\}^{H \times W}$, định nghĩa các vị trí pixel có dữ liệu hợp lệ mang giá trị 1 và các vị trí trống mang giá trị 0.



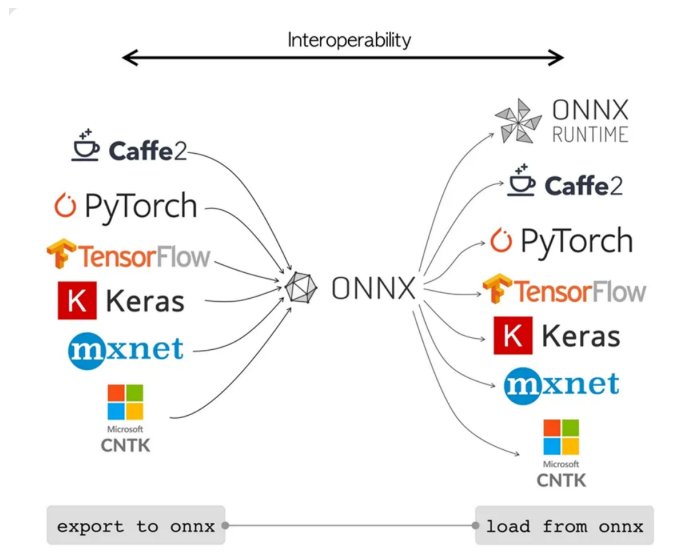
Hình 2.2.3: Mô hình G2-MonoDepth

Việc ép toàn bộ dữ liệu cảm biến đa dạng về không gian tensor 5 kênh đồng nhất (R, G, B, D_{sparse}, M) giúp mô hình G2-MonoDepth trở nên linh hoạt hơn. Mạng nơ-ron không cần phải thay đổi cấu trúc đồ thị tính toán hay cấu hình lại các lớp tích chập đầu vào khi mật độ điểm LiDAR thay đổi từ cực thưa (0.1%) cho đến dày đặc (100%). Sự kết hợp giữa ảnh ngữ cảnh và ma trận mặt nạ lỗ thủng giúp các bộ lọc tích chập của mạng nhận biết một cách tường minh đâu là thông tin đo đặc vật lý đáng tin cậy để làm neo thang đo, và đâu là các vùng không gian trống cần phải thực hiện nội suy và lan truyền độ sâu dựa trên ngữ nghĩa ảnh màu.

2.3 CÔNG NGHỆ TỐI ƯU HÓA MÔ HÌNH VÀ XÂY DỰNG ỨNG DỤNG

2.3.1 Định dạng model ONNX và ONNX Runtime

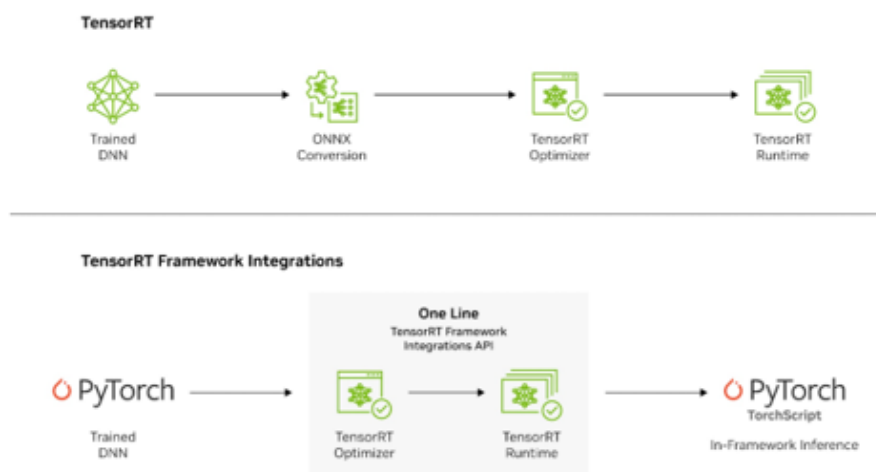
ONNX (Open Neural Network Exchange) định nghĩa một chuẩn mô hình định dạng đồ thị chung độc lập, cho phép các kỹ sư AI dễ dàng chuyển đổi các tham số và cấu trúc mạng nơ-ron giữa các framework huấn luyện khác nhau như PyTorch hay TensorFlow. Một tệp mô hình lưu dưới dạng .onnx bản chất là một đồ thị định hướng không tuần hoàn (DAG - Directed Acyclic Graph), trong đó mỗi Node đại diện cho một toán tử toán học cụ thể (như Conv, ReLU, MaxPool) với các liên kết đầu vào và đầu ra được định kiểu dữ liệu một cách nghiêm ngặt, loại bỏ hoàn toàn sự phụ thuộc vào mã nguồn Python của framework gốc.



Hình 2.3.4: Hệ sinh thái ONNX

Đi kèm với định dạng đồ thị này là ONNX Runtime, một bộ thư viện thực thi tính toán hiệu năng cao được tối ưu hóa sâu ở cấp độ phần cứng. Trong chuỗi xử lý của đề tài, định dạng ONNX Runtime đóng vai trò là một cấu trúc trung gian quan trọng. Sau khi đồ thị tính toán động của PyTorch được xuất sang định dạng ONNX tĩnh và tối giản hóa, tệp tin này sẽ làm tiền đề để bộ biên dịch chuyên dụng nạp vào không gian phần cứng đồ họa, phục vụ cho các bước tối ưu hóa cấp thấp tiếp theo.

2.3.2 Bộ tối ưu hóa hiệu năng cao NVIDIA TensorRT



Hình 2.3.5: Hệ thống TensorRT

NVIDIA TensorRT là một nền tảng phần mềm và bộ tối ưu hóa suy diễn học sâu hiệu năng cao, được thiết kế chuyên biệt cho các bộ vi xử lý đồ họa GPU của NVIDIA. Khi tiếp nhận một đồ thị tính toán tĩnh từ định dạng ONNX, TensorRT tiến hành phân tích và tái cấu trúc lại toàn bộ

các nút mạng thông qua cơ chế Layer Fusion — ví dụ như gom các toán tử tích chập, chuẩn hóa kênh và hàm kích hoạt thành một kernel duy nhất. Điều này giúp giảm thiểu đáng kể độ trễ liên quan đến quản lý bộ nhớ đệm, đồng thời tối ưu hóa luồng dữ liệu giữa các lõi CUDA và Tensor Cores trên GPU.

Bên cạnh đó, TensorRT còn tăng tốc độ thực thi nhờ cơ chế Kernel Tuning dựa trên đặc tính phần cứng cụ thể của hệ thống, cho phép lựa chọn cấu hình thực thi tối ưu nhất cho từng lớp mạng. Đồng thời, nền tảng này thiết lập một vùng Static Memory Buffer cố định ngay tại thời điểm nạp mô hình, từ đó loại bỏ hoàn toàn chi phí cấp phát và giải phóng bộ nhớ động trong suốt quá trình suy diễn thời gian thực. Sự kết hợp giữa tối ưu hóa đồ thị tính toán, tinh chỉnh kernel và quản lý bộ nhớ hiệu quả giúp TensorRT đạt được hiệu năng vượt trội về cả độ trễ lẫn thông lượng xử lý trong các ứng dụng suy diễn học sâu trên GPU.

2.4 CÁC CHỈ SỐ ĐÁNH GIÁ ĐỘ CHÍNH XÁC (EVALUATION METRICS)

Để đánh giá một cách định lượng và khoa học độ chính xác hình học của bản đồ độ sâu dự đoán $\hat{D}$ so với bản đồ độ sâu chuẩn Ground Truth D_{gt} , nghiên cứu sử dụng hệ thống hai chỉ số toán học tiêu chuẩn trong thị giác máy tính, được tính toán dựa trên tập hợp tất cả các pixel hợp lệ N có trong không gian giám sát:

Sai số căn trung bình bình phương (Root Mean Squared Error - RMSE): Chỉ số RMSE đo lường độ lệch tiêu chuẩn của các sai số dự đoán giữa mô hình và thực tế, được định nghĩa bằng công thức toán học:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\hat{D}_i - D_{gt,i} \right)^2}$$

Ý nghĩa thực tế của chỉ số RMSE là nó cực kỳ nhạy cảm với các lỗi có giá trị sai số lớn (Outliers) do các hiệu số đã được bình phương lên trước khi lấy trung bình. Chỉ số này đo trực tiếp bằng đơn vị chiều sâu vật lý (centimet hoặc mét), giá trị RMSE càng nhỏ thể hiện mô hình càng ít gặp các lỗi dự đoán sai lệch nghiêm trọng về khoảng cách tuyệt đối toàn cục, đóng vai trò quyết định trong tác vụ định vị cự ly và chạm của robot.

Sai số tuyệt đối tương đối (Absolute Relative Error - AbsRel): Chỉ số AbsRel chuẩn hóa sai số tuyệt đối tại từng pixel dựa trên chính giá trị khoảng cách Ground Truth tại pixel đó, biểu diễn qua công thức:

$$AbsRel = \frac{1}{N} \sum_{i=1}^N \frac{|\hat{D}_i - D_{gt,i}|}{D_{gt,i}}$$

Do là một chỉ số không có đơn vị (tính theo tỷ lệ phần trăm), AbsRel đóng vai trò cực kỳ

quan trọng để đánh giá tính nhất quán và độ đồng nhất về mặt tỷ lệ hình học của mô hình trên toàn bộ khung hình. Chỉ số này phản ánh thực tế không gian rằng một sai số nhỏ ở khoảng cách gần nguy hiểm và nghiêm trọng hơn rất nhiều so với cùng sai số đó nhưng ở hậu cảnh rất xa, giúp đánh giá độ trơn cấu trúc phối cảnh mà mạng nơ-ron khôi phục được.

Chương 3

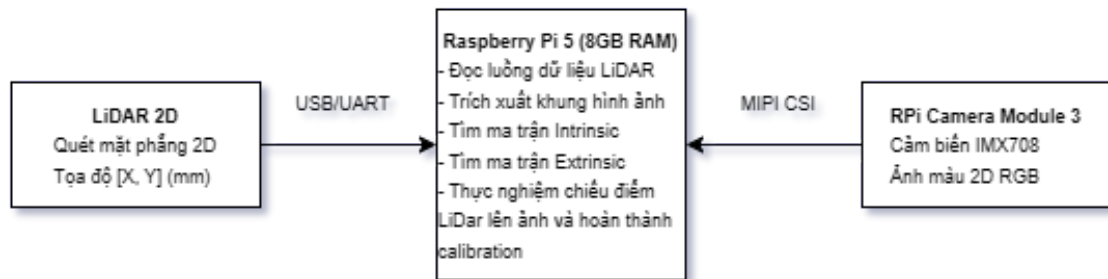
CÂN CHỈNH CẢM BIẾN LIDAR 2D VÀ CAMERA BẰNG TARGET CHỮ V

Trong các hệ thống nhận diện và cảm nhận môi trường hiện đại, việc kết hợp dữ liệu từ nhiều nguồn cảm biến (Multi-sensor Fusion) đóng vai trò quyết định nhằm tăng cường độ chính xác và tính tin cậy. Tuy nhiên, LiDAR và Camera là hai cảm biến hoàn toàn khác biệt cả về bản chất vật lý, định dạng dữ liệu lẫn hệ tọa độ không gian. Camera cung cấp thông tin dạng ma trận điểm ảnh 2D RGB với độ phân giải màu sắc cao, nhưng thiếu thông tin chiều sâu trực tiếp. Còn LiDAR cung cấp đám mây điểm 3D (Point cloud) chứa tọa độ không gian chính xác, nhưng thừa thớt và thiếu thông tin về cấu trúc màu sắc.

Mục tiêu chính của chương này là giải quyết bài toán đồng bộ hình học (Geometric Alignment). Quá trình này tìm ra mối quan hệ dịch chuyển không gian giữa hai cảm biến, cho phép chiếu chính xác một điểm 3D từ hệ tọa độ LiDAR lên mặt phẳng ảnh 2D của Camera nhằm đồng bộ hóa dữ liệu "màu sắc - chiều sâu" tại từng vị trí.

3.1 SƠ ĐỒ KHỐI PHẦN CỨNG

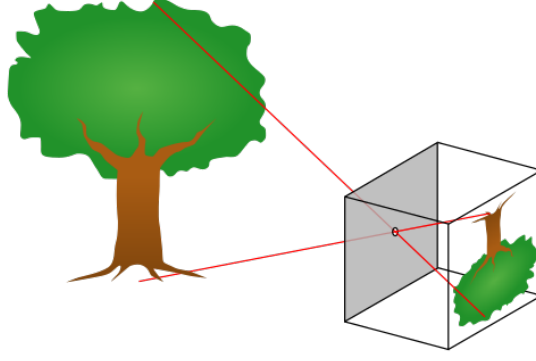
Hệ thống cân chỉnh cảm biến được xây dựng dựa trên ba thành phần cốt lõi với các vai trò chức năng riêng biệt nhằm đảm bảo tính toàn vẹn và tốc độ truyền tải tín hiệu. Đầu não của hệ thống là bộ vi xử lý trung tâm Raspberry Pi 5, đóng vai trò tiếp nhận, điều phối luồng dữ liệu đồng thời thực thi mã nguồn xử lý python scripts thông qua các thư viện tính toán ma trận hiệu năng cao như NumPy và OpenCV. Nhiệm vụ thu nhận thông tin hình ảnh màu sắc 2D được đảm nhiệm bởi Raspberry Pi Camera Module 3 tích hợp cảm biến Sony IMX708 độ phân giải cao và tính năng lấy nét tự động, thiết bị này được kết nối trực tiếp với bo mạch chủ thông qua giao tiếp MIPI CSI 4. Song song với đó, thành phần thứ ba là cảm biến LiDAR 2D (Triangulation Lidar A1M8-R6) chịu trách nhiệm quét cắt ngang môi trường xung quanh để liên tục trả về mảng dữ liệu chứa góc quét và khoảng cách vật lý của vật thể (đơn vị milimét), luồng thông tin này được truyền tải một về bộ xử lý trung tâm thông qua cổng kết nối USB/UART.



Hình 3.1.1: Sơ đồ khối calibration cho phần cứng



Hình 3.1.2: Phần cứng thực hiện calibration



Hình 3.2.3: Mô hình camera lỗ kim

3.2 HIỆU CHUẨN CAMERA (CAMERA INTRINSIC CALIBRATION)

Trước khi thực hiện đồng bộ hình học giữa LiDAR 2D và Camera, việc xác định chính xác các thông số hình học bên trong của thấu kính (Tham số nội) là bắt buộc. Hệ thống sử dụng phương pháp cân chỉnh kinh điển của Zhang[5] dựa trên tấm chuẩn hình bàn cờ mẫu (Checkerboard Pattern) do tính ổn định cao và khả năng trích xuất đặc trưng góc cạnh một cách chính xác.

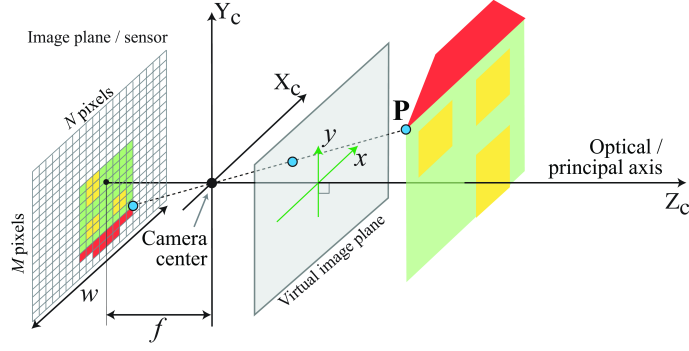
3.2.1 Mô hình camera lỗ kim và hiện tượng méo ảnh

Để máy tính có thể hiểu và xử lý thông tin hình ảnh từ camera, luồng ánh sáng đi qua thấu kính vật lý cần được mô hình hóa bằng các công thức toán học không gian. Hệ thống áp dụng mô hình camera lỗ kim (Pinhole Camera Model) làm nền tảng hình học, đồng thời kết hợp các hàm toán học để bù sai số do hiện tượng méo ảnh gây ra.

Trong mô hình camera lỗ kim lý tưởng như hình 3.2.3, toàn bộ các tia sáng từ vật thể trong không gian 3D đều đi qua một điểm duy nhất gọi là tâm chiếu, trước khi hội tụ trên mặt phẳng cảm biến tạo thành ảnh ngược. Để thuận tiện cho việc tính toán toán học, mặt phẳng ảnh thực tế được dịch chuyển đối xứng qua quang tâm về phía trước vật thể, tạo thành Mặt phẳng ảnh ảo (Virtual Image Plane) giúp hình ảnh không bị lộn ngược.

Giả sử một điểm P (hình 3.2.4) trong không gian 3D có tọa độ đối với hệ tọa độ camera là $P_c = [X_c, Y_c, Z_c]^T$. Hình chiếu của nó trên mặt phẳng ảnh ảo là điểm $p = [x, y]^T$. Theo nguyên lý tam giác đồng dạng, mối quan hệ hình học được xác định qua hệ thức:

$$\frac{x}{f} = \frac{X_c}{Z_c}, \quad \frac{y}{f} = \frac{Y_c}{Z_c}$$



Hình 3.2.4: Mặt phẳng ảo của mô hình camera lỗ kim

Trong đó f là tiêu cự (Focal Length) của thấu kính. Khi chuyển đổi từ hệ tọa độ mặt phẳng ảnh vật lý (đơn vị mm) sang hệ tọa độ điểm ảnh pixel (u, v) hiển thị trên màn hình máy tính, ta cần tính đến các yếu tố tỷ lệ pixel và tọa độ của điểm quang tâm (c_x, c_y) . Hệ phương trình được viết lại dưới dạng ma trận đồng nhất (Homogeneous Coordinates):

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = K \cdot P_c$$

Với $s = Z_c$ là hệ số tỷ lệ chiều sâu, và f_x, f_y là tiêu cự thấu kính được quy đổi ra đơn vị pixel theo hai trục tương ứng.

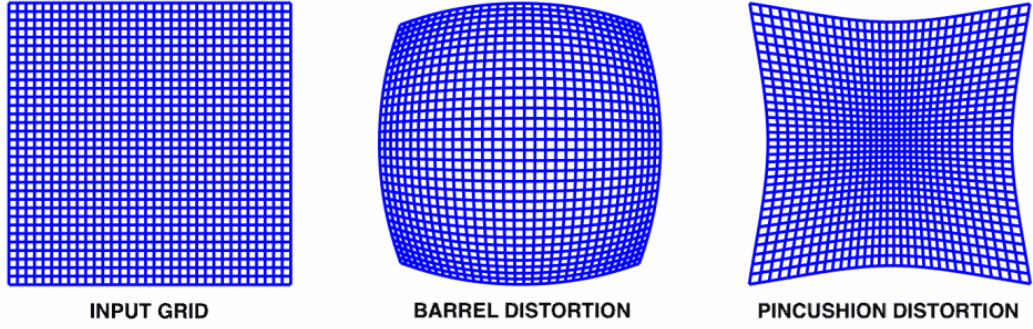
Mô hình pinhole giả định rằng các tia sáng luôn truyền theo đường thẳng tuyệt đối. Tuy nhiên, trên thực tế thấu kính được cấu thành từ các lớp thủy tinh/nhựa có bề mặt cong, dòng ánh sáng đi qua rìa thấu kính sẽ bị bẻ cong nhiều hơn hoặc ít hơn so với các tia đi qua tâm. Sai lệch này tạo ra hiện tượng méo ảnh (tên tiếng anh là Distortion), khiến các đường thẳng thực tế bị biến dạng thành các đường cong trên ảnh kỹ thuật số.

Méo ảnh theo tâm (Radial Distortion)

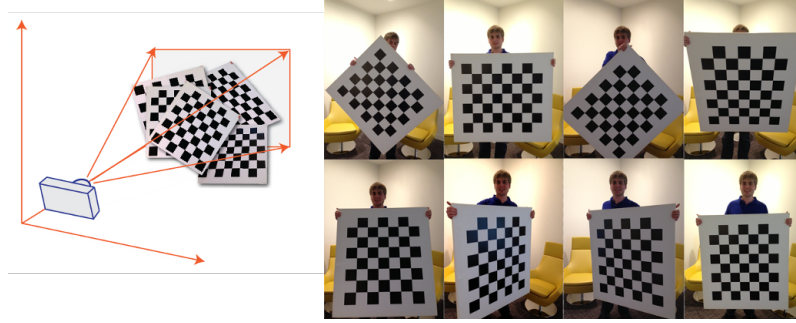
Radial distortion xảy ra do hình dáng hình học không hoàn hảo của bề mặt thấu kính. Hiện tượng này thường nghiêm trọng hơn tại các vị trí xa quang tâm (rìa bức ảnh) và được chia thành hai dạng:

Barrel Distortion: Các tia sáng ở rìa bị bẻ cong quá mức vào trong, làm hình ảnh có xu hướng phình ra như một quả tang trống (thường gặp ở thấu kính góc rộng).

Pincushion Distortion: Các tia sáng ở rìa bị kéo giãn ra ngoài, khiến các cạnh biên bị lõm vào trong giống như một chiếc gối ôm.



Hình 3.2.5: Méo ảnh theo tâm



Hình 3.2.6: Hiệu chuẩn camera bằng ảnh checkerboard.

Toán học mô tả sự dịch chuyển từ tọa độ lý tưởng (x, y) sang tọa độ bị méo tâm (x_{radial}, y_{radial}) qua chuỗi Taylor:

$$\begin{cases} x_{radial} = x(1 + k_1r^2 + k_2r^4 + k_3r^6) \\ y_{radial} = y(1 + k_1r^2 + k_2r^4 + k_3r^6) \end{cases}$$

Trong đó $r^2 = x^2 + y^2$ là bình phương khoảng cách từ điểm ảnh đến quang tâm, và k_1, k_2, k_3 là các hệ số méo tâm.

3.2.2 Thực nghiệm hiệu chuẩn camera

Sau khi xây dựng cơ sở lý thuyết, tiến trình thực nghiệm hiệu chuẩn tham số nội cho Raspberry Pi Camera Module 3 được thực hiện trên thực tế nhằm thu thập dữ liệu và tính toán ra các ma trận đặc trưng K và D . Để đảm bảo độ chính xác cao nhất cho thuật toán tối ưu, các điều kiện thực nghiệm được thiết lập nghiêm ngặt.

Calibration target là một tấm phẳng cứng chứa lưới bàn cờ mẫu (Checkerboard) kích thước ô vuông cố định 12×12 mm, cấu trúc bao gồm 14×14 góc trong (Internal Corners). Tấm chuẩn được đảm bảo độ phẳng tuyệt đối, không bị cong vênh để tránh sai số hình học. Thực nghiệm được tiến hành trong môi trường ánh sáng khuếch tán đồng đều, tránh hiện tượng đổ bóng cục bộ

hoặc lóá sáng (Specular Reflection) trên bề mặt tấm bàn cờ, điều này giúp thuật toán nhận diện biên góc đạt hiệu suất tốt nhất. Raspberry Pi Camera Module 3 được cố định chắc chắn trên giá đỡ cùng với cụm cảm biến LiDAR 2D. Tiêu cự được thiết lập chế độ khóa (Fixed Focus) hoặc lấy nét cố định tại khoảng cách thực nghiệm để tránh hiện tượng ma trận tham số nội bị thay đổi động trong quá trình thu thập ảnh. Quy trình thực nghiệm được triển khai tuần tự theo từng bước như theo cấu trúc file trên github:

- **common.py:** Mô-đun common.py đóng vai trò là lưu trữ các tham số tĩnh, cấu hình phần cứng và định nghĩa hàm biến đổi giữa các hệ tọa độ. Định dạng tấm chuẩn Checkerboard được thiết lập chính xác với cấu trúc ma trận góc trong `CHECKERBOARD_SIZE = (14, 14)` và kích thước ô vật lý 12.0 mm nhằm cấp dữ liệu chuẩn cho pha tính toán cơ sở. Trong xử lý hình học máy tính, hàm `calculate_fov` trích xuất các hệ số tiêu cự thấu kính (f_x, f_y) từ ma trận nội `mtx` để tính toán góc mở trường nhìn ngang (H-FOV) và dọc (V-FOV) của cảm biến thông qua hàm lượng giác hàm ngược `arctan`, hỗ trợ việc xác thực không gian quan sát trực quan. Đối với luồng dữ liệu LiDAR 2D, hàm `angleDistanceToLidarXZ` thực hiện phép chiếu từ hệ tọa độ cực (góc quét và khoảng cách d) sang hệ tọa độ Descartes nội bộ của cảm biến (X_L, Z_L). Hệ tọa độ thô này tiếp tục được ánh xạ sang không gian hiển thị đồ họa 2D góc nhìn từ trên xuống (Top-down View) kích thước 800×800 thông qua bộ hàm đối ứng `lidarWorldToLidarView` và `lidarViewToLidarWorld`. Phép toán dịch tâm về quang tâm ma trận (`LIDAR_VIEW_SIZE // 2`) kết hợp hệ số nhân tỷ lệ phóng đại `LIDAR_VIEW_SCALE = 300` giúp chuyển đổi liên mạch giữa đơn vị milimét thực tế và pixel hiển thị, tạo nền tảng cấu trúc nhất quán cho việc đồng bộ và ánh xạ pixel liên không gian ở các mô-đun chức năng tiếp theo.

```
import math

CAMERA_BUFFER_SIZE = (512, 256)
LIDAR_VIEW_SIZE = (800, 800)
LIDAR_VIEW_SCALE = 300
CHECKERBOARD_SIZE = (14, 14)
CHECKERBOARD_SQUARE_SIZE = 12.0 # mm

...
```

- **1_capture_checker_board.py:** Là script chạy trên khối xử lý trung tâm, script thu thập ảnh checkerboard kết hợp thư viện Picamera2 và OpenCV [6]. Mã nguồn thực thi việc khóa cứng cơ chế lấy nét tự động (AfMode: 0, LensPosition: 1.0) nhằm giữ tiêu cự thấu kính bất biến, triệt tiêu hoàn toàn sai số động cho ma trận tham số nội K . Chương trình liên tục xử lý luồng ảnh xám thời gian thực và áp dụng bộ lọc điều kiện kép: lệnh ghi hình vào thư mục /cali chỉ được kích hoạt khi hàm cv2.findChessboardCorners xác thực thành công cấu trúc 14×14 góc trong của tấm chuẩn, đồng thời đảm bảo khoảng giãn cách thời gian giữa các pha bấm máy đạt tối thiểu 1.0 giây. Giải pháp này vừa cung cấp tính năng phản hồi thị giác trực quan thông qua đồ họa lưới điểm trên màn hình, vừa tự động sàng lọc và loại bỏ ảnh lỗi do rung nhòe, mang lại một tập dữ liệu đầu vào chuẩn hóa và sạch tuyệt đối cho pha tối ưu hóa toán học hậu kỳ.
- **2_get_intrinsic.py:** Là script đọc file ảnh chụp được, xử lý ảnh và output 1 file calibration. Script khởi tạo hệ tọa độ vật thể 3D thực tế bằng cách định hình ma trận objp, áp dụng hàm np.mgrid kết hợp nhân với kích thước ô thực tế (common.CHECKERBOARD_SQUARE_SIZE tính bằng mm) để thiết lập lưới điểm chuẩn với giả định mặt phẳng $Z = 0$. Quá trình duyệt ảnh nạp lại hàm cv2.findChessboardCorners, kết hợp với tiêu chí dừng lặp dựa trên độ chính xác cực hạn và số vòng lặp tối đa criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001) trong hàm cv2.cornerSubPix để tinh chỉnh các điểm góc đạt độ chính xác mức sub-pixel. Khi điều kiện thực nghiệm đạt độ phủ tối thiểu (trên 10 ảnh hợp lệ), hàm toán học cốt lõi cv2.calibrateCamera được kích hoạt để giải bài toán tối ưu sai số bình phương tối thiểu (RMS). Kết quả đầu ra gồm ma trận tham số nội mtx (tiêu cự và quang tâm) cùng vector hệ số méo dist được xuất thẳng ra file cấu hình nén picam3_calib.npz, đồng thời tự động tính toán góc mở trường nhìn (FOV) của cảm biến và hoàn tất calibration.

```

...
    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints,
        imgpoints, img_size, None, None)
    np.savez(OUTPUT_FILE, mtx=mtx, dist=dist)
...

```




Hình 3.3.7: Target hình chữ V biến dạng

3.3 CÂN CHỈNH THÔNG SỐ NGOẠI TẠI GIỮA LIDAR VÀ CAMERA (EXTRINSIC CALIBRATION)

Sau khi đã xác định và khử hoàn toàn các sai số hình học nội tại của thấu kính Camera Module 3 thông qua ma trận K và bộ hệ số méo $Dist$, bước tiếp theo là thực hiện **extrinsic calibration**. Mục tiêu của quá trình này là tìm kiếm mối quan hệ dịch chuyển không gian, bao gồm ma trận xoay (Rotation Matrix - R) và vector tịnh tiến (Translation Vector - T), giữa hệ tọa độ cục bộ của cảm biến LiDAR 2D và hệ tọa độ của Camera. Đối với hệ thống sử dụng LiDAR 2D (chỉ quét trên một mặt phẳng cắt), bài toán thiết lập cấu trúc hình học trở nên đặc biệt phức tạp do dữ liệu LiDAR thiếu hụt chiều cao (Y_L). Bằng cách sử dụng bia chuẩn hình chữ **V**, hệ thống quy đổi các đường cắt laser thành ba điểm đặc trưng. Thông qua việc thay đổi vị trí bia chuẩn, ta thu được tập hợp các điểm tương ứng giữa không gian 3D của LiDAR và tọa độ pixel 2D trên ảnh. Thuật toán toán học `cv2.solvePnP` sẽ tối ưu hóa sai số chiếu lại trên tập điểm này, đồng thời trả về hai giá trị cốt lõi: vector xoay $rvec$ và vector tịnh tiến $tvec$. Để các tham số ngoại lai này có thể áp dụng chính xác vào luồng dữ liệu thời gian thực mà không xảy ra xung đột hay sai lệch đơn vị, toàn bộ kiến trúc toán học cần được ràng buộc nghiêm ngặt trong một ma trận định nghĩa hệ tọa độ thống nhất, được cấu hình trực tiếp bên trong mô-đun `common.py`.

3.3.1 Thiết lập hệ tọa độ cảm biến

Mô-đun `common.py` không chỉ lưu trữ các tham số tĩnh mà còn đóng vai trò là "bộ dịch" thiết lập quy chuẩn cho 3 hệ tọa độ khác nhau trong hệ thống thực nghiệm: Hệ tọa độ LiDAR, Hệ tọa độ hiển thị (Lidar View), và Hệ tọa độ ảnh (Camera View).

Dữ liệu thô xuất ra từ cảm biến LiDAR 2D ban đầu tồn tại dưới dạng tọa độ cực, gồm góc quét θ (độ) và khoảng cách d (milimét). Để đưa về hệ không gian Descartes 3D phục vụ cho việc tính toán PnP và chiếu điểm, hàm `angleDistanceToLidarXZ` áp dụng phép chiếu lượng giác:

$$\begin{cases} X_L = d \cdot \sin(\theta) \\ Y_L = 0 \\ Z_L = d \cdot \cos(\theta) \end{cases}$$

Trục Z_L hướng thẳng về phía trước theo hướng nhìn của cảm biến (đóng vai trò là trục chiều sâu). Trục X_L hướng sang hai bên trái/phải (đại diện cho phương ngang). Trục Y_L mặc định bằng 0 do đặc tính của dòng LiDAR quét 2D mặt phẳng cắt.

Để người vận hành có thể giám sát trực quan vị trí đám mây điểm trước khi thực hiện ép màu lên ảnh, `common.py` thiết lập một không gian hiển thị đồ họa giả lập kích thước 800×800 pixel với tâm ma trận $(c_x, c_y) = (400, 400)$.

Hàm `lidarWorldToLidarView` thực hiện ánh xạ tuyến tính từ tọa độ thực tế (mm) sang pixel hiển thị thông qua hệ số tỷ lệ `LIDAR_VIEW_SCALE = 300`:

$$\begin{cases} u_{view} = c_x + (Z_L \cdot SCALE) \\ v_{view} = c_y - (X_L \cdot SCALE) \end{cases}$$

Việc nhân trục Z_L với hệ số `SCALE` và cộng vào c_x giúp đẩy các điểm ở xa lên phía trên theo chiều dọc của màn hình hiển thị. Ngược lại, việc lấy c_y trừ đi X_L giúp đồng bộ chính xác hướng dịch chuyển trái/phải của vật thể, tạo ra một bản đồ radar quét không gian 2D trực quan và nhất quán. Hàm đối ứng `lidarViewToLidarWorld` thực hiện phép tính ngược lại, cho phép người dùng click chuột trên màn hình đồ họa để trích xuất ngược ra khoảng cách vật lý thực tế của vật thể.

Một điểm lưu ý đặc biệt quan trọng trong kiến trúc của `common.py` là sự nhất quán về đơn vị đo lường. Tất cả các hàm xử lý tọa độ thô của LiDAR và kích thước tấm chuẩn bàn cờ (`CHECKERBOARD_SQUARE_SIZE = 12.0`) đều sử dụng đơn vị milimét (mm). Do đó, kết quả

đầu ra của thuật toán `cv2.solvePnP` (vector tịnh tiến `tvec`) ban đầu cũng mang đơn vị milimét. Tuy nhiên, khi đưa vào mô hình hình học của Camera (Mô hình Pinhole ở mục 3.8), toán học máy tính yêu cầu sự đồng nhất tuyệt đối về tỷ lệ. Để tránh hiện tượng tràn số hoặc sai lệch vị trí khi gọi hàm chiếu điểm `cv2.projectPoints`, hệ thống thực hiện một bước chuẩn hóa động: chia tọa độ thế giới và vector `tvec` cho 1000.0 để đưa về đơn vị Mét (m) trước khi thực hiện bộ lọc chiều sâu chủ động ($Z_c > 0.1\text{m}$). Việc định nghĩa tường minh và chắt chẽ các hàm chuyển đổi không gian trong `common.py` chính là chiếc cầu nối cốt lõi giúp luồng dữ liệu đa cảm biến (LiDAR - Camera) đạt được sự đồng bộ hình học hoàn hảo trên thực tế.

3.3.2 Xác định TVEC và RVEC

Quy trình cân chỉnh tham số ngoại lai nhằm xác định ma trận biến đổi giữa LiDAR 2D và Camera được chia làm hai bước. Bước một là thu thập cặp dữ liệu đồng bộ đám mây điểm và hình ảnh thực nghiệm chứa bia chuẩn chữ V. Bước hai là trích xuất điểm đặc trưng tương ứng và tối ưu hóa ma trận chuyển đổi bằng thuật toán hình học.

Để có cơ sở toán học tính toán ma trận xoay R và vector tịnh tiến T , hệ thống cần thu thập một tập dữ liệu đồng bộ pha. Script `capture_calibration.py` bắt cặp trạng thái không gian của cảm biến LiDAR RPLidar và luồng ảnh từ Camera tại cùng một thời điểm.

Mã nguồn khởi tạo đồng thời hai luồng thiết bị: Camera hoạt động ở chế độ kích hoạt và khóa tiêu cự tĩnh lấy thông tin từ file cấu hình nội `picam3_calib.npz`, kết hợp cảm biến Lidar. Luồng xử lý được khóa trong vòng lặp lặp toán học của LiDAR:

```
for scan in lidar.iter_scans():
    raw_rgb = picam2.capture_array()
```

Do tốc độ quét của LiDAR 2D chậm hơn tốc độ phơi sáng của Camera, việc lồng lệnh chụp ảnh `picam2.capture_array()` ngay trong vòng lặp duyệt gói quét `lidar.iter_scans()` đảm bảo bức ảnh RGB thu được có độ trễ thời gian tối thiểu đối với cấu trúc đám mây điểm của vòng quét đó ($\Delta t \approx 0$).

Trong mỗi vòng lặp, dữ liệu hệ tọa độ cực (angle, distance) của các điểm laser quét trúng bia chữ V được hàm `common.angleDistanceToLidarXZ` dịch chuyển tức thời sang không gian Descartes cục bộ dưới cấu trúc mảng 3D: $[X_L, 0.0, Z_L]$ (đơn vị mm).

Để người thực nghiệm giám sát trực quan, hệ thống giả lập một màn hình hiển thị Radar kích thước 800×800 pixel với hai trục tọa độ trực giao. Tọa độ thực được quy đổi động sang mét và nhân với hệ số phóng đại `common.LIDAR_VIEW_SCALE` để vẽ các điểm quét thành các điểm pixel trắng (`cv2.circle`) trên nền đen, cho phép người vận hành quan sát rõ ràng góc cắt hình

chữ V của tấm chuẩn.

Khi người thực nghiệm bấm phím SPACE, hệ thống đóng băng trạng thái tại thời điểm t (timestamp) và tiến hành xuất dữ liệu song song ra thư mục /lidar_image_pairs dưới hai định dạng tệp đồng bộ: Tệp ảnh ảnh (cap_[timestamp]_[id].jpg): Lưu trữ ma trận điểm ảnh thô từ camera để phục vụ cho thuật toán nhận diện biên và trích xuất giao tuyến đỉnh chữ V trên ảnh ở bước sau. Tệp siêu dữ liệu (cap_[timestamp]_[id].json): Đóng gói toàn bộ danh sách tọa độ không gian 3D $[X_L, 0.0, Z_L]$ của vòng quét tương ứng dưới cấu trúc dữ liệu JSON sạch.

Quá trình bấm máy này được lặp lại từ 15 đến 20 lần bằng cách di chuyển bia chữ V liên tục xung quanh phòng thực nghiệm (tịnh tiến dọc trục chiều sâu $+Z$ và dịch ngang trục $\pm X$). Việc lưu trữ đồng bộ dưới dạng cặp tệp [Ảnh .jpg - Tọa độ .json] giúp tối ưu hóa tiến trình trích xuất và đối sánh điểm đặc trưng hình học ở bước ước lượng ma trận chuyển đổi Extrinsic tiếp theo.

Quy trình gán nhãn

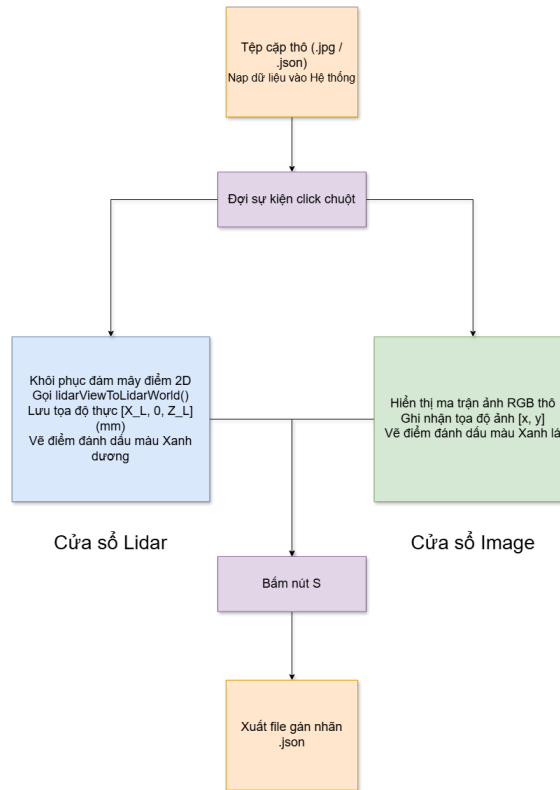
Sau khi thu thập thành công các tập dữ liệu thô song song, bước tiếp theo là trích xuất chính xác tọa độ của các điểm đặc trưng hình học (Đỉnh chính và các đỉnh phụ của bia chữ V cải tiến) trên cả hai không gian cảm biến. Do đặc tính của dữ liệu đám mây điểm LiDAR 2D và ảnh kỹ thuật số mang định dạng hoàn toàn khác nhau, script label.py dựa vào thư viện OpenCV [6] để hỗ trợ người dùng thực hiện cấu hình bắt đầu công việc bằng chuột.

Khi người vận hành tương tác bằng cách click chuột trái trên giao diện màn hình Radar (Labeling Tool(Lidar)), hệ thống kích hoạt hàm gọi ngược lidar_mouse_callback. Điểm mấu chốt của thuật toán là việc đảo ngược ma trận hiển thị đồ họa thông qua hàm common.lidarViewToLidarWorld(x, y) để tính toán ngược từ tọa độ điểm ảnh pixel (x, y) trên màn hình máy tính về lại tọa độ không gian mét thực tế, sau đó nhân với 1000.0 để đưa về đơn vị chuẩn milimét:

$$\begin{cases} X_L = \frac{c_y - y}{\text{LIDAR_VIEW_SCALE}} \times 1000.0 \\ Z_L = \frac{x - c_x}{\text{LIDAR_VIEW_SCALE}} \times 1000.0 \end{cases}$$

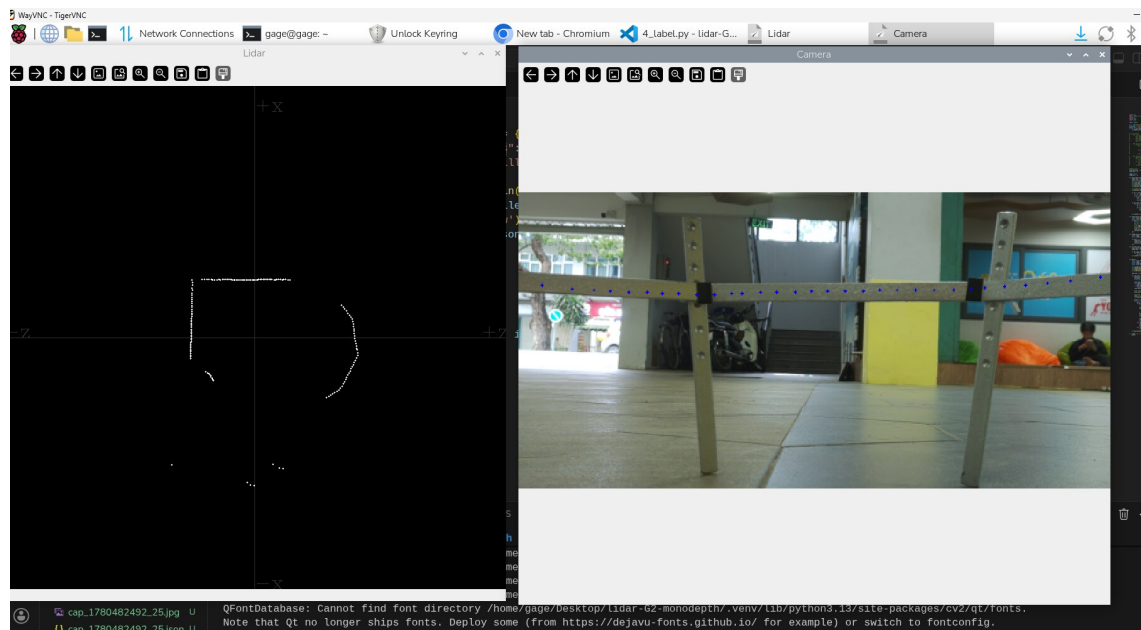
Tọa độ đặc trưng thực tế này $[X_L, 0, Z_L]$ được lưu vào mảng lidar_pts, đồng thời một điểm tròn màu xanh dương được vẽ đè lên giao diện đồ họa tại vị trí click để làm mốc định vị cho người vận hành.

Trên giao diện camera (Labeling Tool(Image)), hàm gọi ngược image_mouse_callback thực hiện nhiệm vụ bắt tọa độ pixel trực tiếp $[x, y]$ tương ứng với các đỉnh của cấu trúc bia chữ V có cảnh hiển thị trên ảnh và lưu vào mảng image_pts, đánh dấu bằng một điểm tròn màu xanh lá cây. Hàm refresh() liên tục được gọi sau mỗi sự kiện click chuột để cập nhật tức thời trạng thái đồ



Hình 3.3.8: Flow hoạt động của label.py

họa của cả hai màn hình, đảm bảo tính trực quan và ngăn ngừa sai sót gán nhãn nhầm điểm. Quy trình kết thúc phiên gán nhãn cho một cặp dữ liệu được ràng buộc chặt chẽ bởi các phím chức năng hệ thống: Phím r (Reset): Xóa toàn bộ trạng thái bộ nhớ đệm của các điểm đã chọn trong phiên hiện tại, cho phép người dùng tiến hành gán nhãn lại từ đầu nếu phát hiện sai sót. Phím s (Save): Kích hoạt bộ kiểm tra điều kiện logic để xác thực tính tương thích cấu trúc giữa hai tập điểm. Nếu số lượng điểm chọn trên ảnh không khớp với số lượng điểm chọn trên mô hình radar ($\text{len}(\text{image_pts}) \neq \text{len}(\text{lidar_pts})$), hệ thống lập tức phát ra cảnh báo lỗi nhằm ngăn chặn sự lệch pha dữ liệu. Khi điều kiện đối sánh được thỏa mãn, thuật toán đóng gói dữ liệu và xuất thẳng ra thư mục cấu hình /lidar_image_pair_labeled dưới dạng một tập JSON sạch mang cấu trúc cây nghiêm ngặt:



Hình 3.3.9: Giao diện labeling tools. Cửa sổ bên trái là lidar view và bên phải là camera view

```
{
  "labels": {
    "image_pixel_points": [
      [u1, v1],
      [u2, v2],
      [u3, v3]
    ],
    "lidar_3d_points_millimeters": [
      [X_L1, 0.0, Z_L1],
      [X_L2, 0.0, Z_L2],
      [X_L3, 0.0, Z_L3]
    ]
  }
}
```

Tập tin lưu trữ này cô đọng toàn bộ các cặp điểm hình học tương ứng lý tưởng giữa không gian vật lý 3D của cụm cảm biến Raspberry Pi 5 và không gian ảnh 2D, đóng vai trò là tập dữ liệu đầu vào quyết định cho bài toán tối ưu hóa ma trận chuyển đổi ngoại vi Extrinsic thông qua bộ giải PnP ở bước tiếp theo.

Bộ giải SolvePnP

Hàm `cv2.solvePnP` (Perspective-n-Point) [7] trong thư viện OpenCV [6] là một hàm toán học cốt lõi dùng để xác định vị trí và hướng (gọi chung là Extrinsic Parameters) của một camera trong không gian 3D khi chúng ta đã biết trước hai yếu tố. Một là tọa độ của một tập hợp các điểm trong không gian thực 3D (Object Points). Hai là tọa độ pixel tương ứng của các điểm đó trên mặt phẳng ảnh 2D thu được từ camera (Image Points).

```
success, rvec, tvec = cv2.solvePnP(objectPoints, imagePoints,
                                   cameraMatrix, distCoeffs, flags)
```

- `objectPoints`: Mảng các điểm 3D trong hệ tọa độ vật thể (đơn vị thường là mm hoặc mét). Hệ tọa độ này do bạn tự quy định (ví dụ góc tầm bia là $(0, 0, 0)$).
- `imagePoints`: Mảng các điểm pixel 2D tương ứng được trích xuất từ ảnh (bằng thuật toán tự động hoặc gán nhãn thủ công bằng chuột).
- `cameraMatrix` (K): Ma trận tham số nội của camera (chứa tiêu cự f_x, f_y và quang tâm c_x, c_y). Đây là kết quả thu được từ quá trình hiệu chuẩn nội bộ (Intrinsic Calibration).
- `distCoeffs` (D): Bộ các hệ số méo thấu kính ($k_1, k_2, p_1, p_2, \dots$). Hàm cần ma trận này để tự động "khử méo" các điểm ảnh trước khi tính toán hình học.
- `flags`: Phương pháp thuật toán sẽ dùng để giải bài toán (ví dụ: `cv2.SOLVEPNP_ITERATIVE` sử dụng thuật toán lặp Levenberg-Marquardt, hoặc `cv2.SOLVEPNP_EPNP` cho tốc độ nhanh hơn).

Nếu biết hình dáng và kích thước thực tế của một vật thể (ví dụ: tấm bàn cờ checkerboard hoặc bia chữ V), và chụp một bức ảnh của nó, `cv2.solvePnP` sẽ tính toán chính xác xem camera đang đứng ở đâu (khoảng cách bao xa) và đang nhìn nghiêng một góc bao nhiêu độ so với vật thể đó.

Đầu tiên chương trình khởi tạo bằng việc nạp tệp tham số nội thấu kính `picam3_calib.npz` để lấy ra ma trận camera K và bộ hệ số khử méo D . Tiếp theo, thuật toán duyệt qua toàn bộ các tệp cấu hình dữ liệu JSON đã được gán nhãn từ bước trước.

Đối với mỗi cặp thực nghiệm chứa bia chuẩn chữ V, thuật toán thực hiện bóc tách đồng thời 3 điểm đặc trưng. Việc lấy cả 3 điểm phân đoạn trên mỗi khung hình giúp chương trình xây dựng được một tập hợp ma trận, đảm bảo bộ giải PnP không rơi vào trạng thái suy biến nghiệm (Singularity).

Khi tập dữ liệu điểm tương ứng giữa không gian LiDAR 3D (lidar_points) và mặt phẳng ảnh 2D (img_points) được gom đủ điều kiện (tối thiểu trên 10 ảnh, tương đương hơn 30 cặp điểm ràng buộc), hệ thống kích hoạt hàm lõi tối ưu hóa:

```
result, rvec, tvec = cv2.solvePnP(..., flags=cv2.SOLVEPNP_ITERATIVE)
```

Sử dụng cờ SOLVEPNP_ITERATIVE, bản chất là thuật toán tối ưu hóa phi tuyến Levenberg-Marquardt. Thuật toán này liên tục lặp lại quá trình thay đổi ma trận xoay và dịch chuyển nhằm cực tiểu hóa hàm mục tiêu sai số chiếu lại (Reprojection Error) giữa tọa độ pixel gán nhãn thực tế và tọa độ góc không gian 3D được chiếu thử nghiệm qua mô hình camera lỗ kim.

Kết quả trả về vector tịnh tiến tvec biểu thị khoảng cách vật lý thực tế $[X, Y, Z]$ từ tâm quang học camera đến tâm trục xoay LiDAR (đơn vị mm). Và vector xoay rvec thể hiện độ lệch hướng dưới dạng vector dòng xoay nhỏ gọn. Để chuyển đổi vector xoay 3 chiều này về dạng ma trận xoay toán học 3×3 toàn phần (R), hệ thống áp dụng công thức biến đổi Rodrigues thông qua hàm cv2.Rodrigues(rvec).

Để kiểm tra xem cụm cảm biến lắp đặt trên có bị nghiêng lệch cơ học hay không, chương trình thực hiện một bước giải mã toán học quan trọng: biến đổi ma trận xoay 3×3 (R) về 3 góc Euler trực quan bao gồm Pitch (Góc gập dọc - trục X), Yaw (Góc hướng ngang - trục Y), và Roll (Góc nghiêng xoay - trục Z).

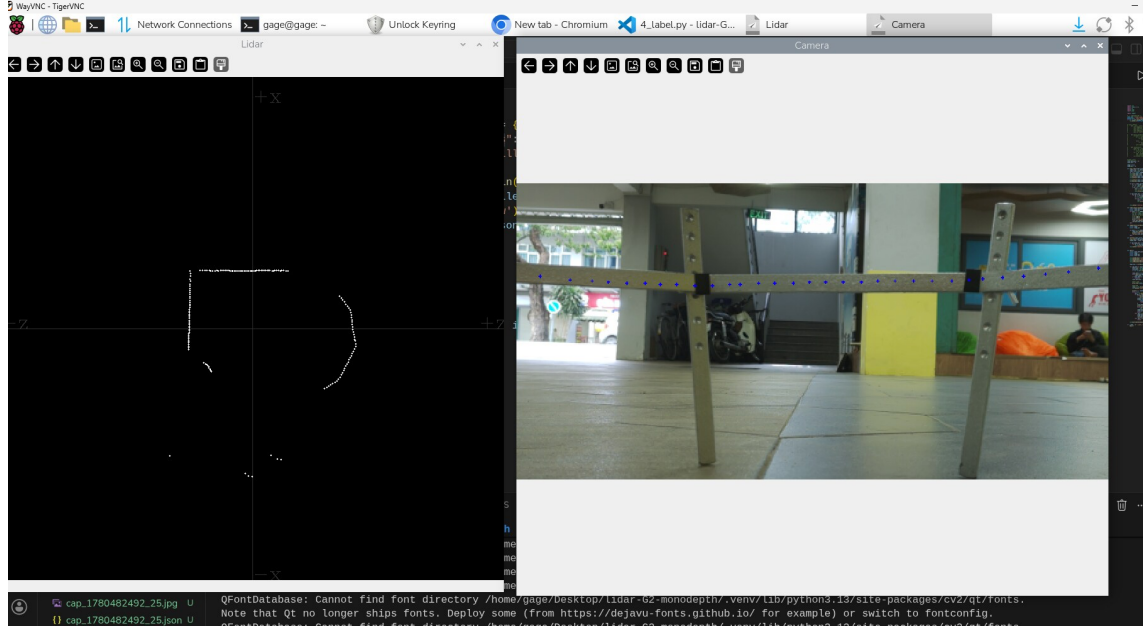
Cuối cùng, khi bộ giải đạt trạng thái hội tụ lý tưởng, toàn bộ thông số ngoại tại (rvec và tvec) được đóng gói nén tĩnh vào tệp extrinsics.npz. Đây chính là "chìa khóa dữ liệu" cốt lõi để các mô-đun hậu kỳ gọi ra trực tiếp, thực hiện ép mảng điểm LiDAR 2D lên các khung hình màu của camera.

Lidar-Camera fusion

Để đồng bộ và hiển thị chính xác các điểm đo từ cảm biến LiDAR lên ma trận điểm ảnh của Camera, hệ thống áp dụng mô hình toán học của Camera lỗ kim (Pinhole Camera Model) kết hợp với phép biến đổi tọa độ. Giả sử một điểm bất kỳ trong không gian ba chiều của hệ tọa độ LiDAR có tọa độ thuần nhất là $P_L = [X_L, Y_L, Z_L, 1]^T$. Quá trình ánh xạ điểm này về tọa độ pixel phẳng $p = [u, v, 1]^T$ trên ảnh được thực hiện thông qua chuỗi biến đổi tuyến tính:

$$p \sim K \cdot \begin{bmatrix} R & T \end{bmatrix} \cdot P_L$$

Với $\begin{bmatrix} R & T \end{bmatrix}$ (Ma trận ngoại lai - Extrinsic Matrix): Thực hiện phép biến đổi affin dịch chuyển điểm P_L từ hệ tọa độ thế giới thực của LiDAR sang hệ tọa độ cục bộ của thấu kính camera



Hình 3.3.10: Giao diện Lidar-Camera fusion, cửa sổ bên trái là Lidar View còn bên phải là cửa sổ ảnh màu RGB + với các điểm được chiếu lên từ Lidar

$$(P_c = [X_c, Y_c, Z_c]^T)$$

K (Ma trận tham số nội - Intrinsic Matrix): Thực hiện phép chiếu phối cảnh (Perspective Projection) từ không gian 3D của thấu kính camera lên mặt phẳng cảm biến ảnh số:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Sau khi nhân ma trận, ta thu được tọa độ pixel thô. Tuy nhiên, do hiện tượng méo hình hình học gây ra bởi các lớp thấu kính vật lý, các điểm ảnh phải được hiệu chỉnh qua các đa thức khử méo dựa trên bộ hệ số $D = [k_1, k_2, p_1, p_2]$ trước khi trích xuất ra tọa độ pixel thực tế (u, v) để hiển thị.

Script Lidar-Camera fusion

Script thực thi nhiệm vụ nạp các tham số hiệu chuẩn tĩnh, liên tục bắt dữ liệu từ phần cứng phần cơ và thực hiện toán học chiếu điểm thời gian thực để tạo ra một bản đồ đám mây điểm có mã hóa màu theo chiều sâu chồng lấp trực tiếp lên luồng video.

Ngay khi khởi chạy, chương trình thực hiện nạp song song hai tệp cấu hình nén đã được tính toán từ các giai đoạn trước: `picam3_calib.npz` (chứa ma trận nội K và bộ hệ số khử méo

`dist_coeffs`) và `extrinsics.npz` (chứa vector xoay *rvec* và vector tịnh tiến *tvec* thu được từ bộ giải PnP). Các tham số này đóng vai trò là hằng số hình học bất biến suốt phiên vận hành của hệ thống nhúng.

Trong vòng lặp xử lý chính, tại mỗi nhịp quét của thiết bị RPLidar, hệ thống đồng thời kích hoạt lệnh chụp ảnh từ bộ đệm của Camera(script sử dụng PI Camera). Luồng dữ liệu thô chuyển đổi tọa độ cực sang Descartes. Hàm `cv2.projectPoints` được sử dụng để lấy mảng điểm 3D thô của LiDAR, nhân với ma trận xoay (giải mã từ *rvec* theo Rodrigues) và vector tịnh tiến *tvec* để đưa về hệ tọa độ camera, sau đó áp dụng bộ hệ số đa thức *dist_coeffs* để khử hoàn toàn hiện tượng méo thấu kính góc rộng của Camera, cuối cùng xuất ra tọa độ pixel chuẩn hóa (x, y) .

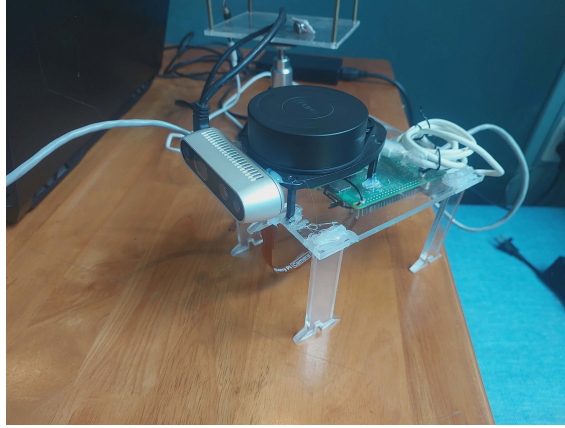
```
img_pts, jacobian = cv2.projectPoints(lidar_pos, rvec, tvec, K,
                                     dist_coeffs)
```

Kết quả cuối cùng là luồng video từ Camera được phun phủ (overlay) chính xác từng điểm ảnh quét độ sâu của LiDAR thời gian thực với sai số hình học đạt mức lý tưởng, chứng minh tính đúng đắn của toàn bộ quy trình thiết lập hệ tọa độ và cân chỉnh tham số ngoại lai trước đó.

3.4 THỰC NGHIỆM CÂN CHỈNH CẢM BIẾN VÀ ĐÁNH GIÁ SAI SỐ

Để đánh giá độ chính xác của thuật toán hợp nhất không gian giữa cảm biến RPLiDAR 2D và Camera 3, nghiên cứu này thiết lập một hệ thống cấu hình đối sánh chéo toàn cục bằng cách tích hợp cảm biến đo chiều sâu chuyên dụng Intel RealSense D435 trong vai trò cung cấp dữ liệu khoảng cách Ground Truth. Nhằm đảm bảo tính đồng nhất hình học cao nhất trước khi thực hiện đối sánh, ma trận nội tại và ngoại vi của cảm biến RealSense được trích xuất thông qua quy trình hiệu chuẩn chuẩn hóa, sử dụng tám bàn cờ tiêu chuẩn kết hợp với tám bia mục tiêu dạng chữ V như trên.

Quá trình đánh giá sau đó được thực hiện tự động theo thời gian thực dựa trên cơ chế chiếu đồng thời, bắt đầu bằng việc lọc thô hướng nhìn nhằm tối ưu hóa hiệu năng tính toán. Toàn bộ tập hợp điểm quét từ dữ liệu gốc của RPLiDAR lập tiếp đi qua bộ lọc để loại bỏ các điểm có tọa độ chiều sâu nguyên bản nhỏ hơn không, nghĩa là hệ thống chỉ giữ lại các điểm quét thuộc nửa bán cầu phía trước hướng di chuyển của robot và loại bỏ hoàn toàn các điểm bóng ma phía sau lưng. Tập hợp điểm LiDAR hợp lệ này, sau khi chuyển đổi đơn vị từ milimét sang mét để tương thích với đại số ma trận, được chiếu thuận đồng thời lên hai mặt phẳng cảm biến ảnh: mặt phẳng ảnh 2D của Camera 3 nhằm mục đích hiển thị trực quan và lưới ma trận độ sâu 640×480 của RealSense D435 phục vụ mục đích đối sánh. Tại mỗi tọa độ pixel trùng khớp trên không gian của



Hình 3.4.11: Sử dụng realsense để đánh giá

cảm biến tham chiếu, giá trị khoảng cách ước lượng từ hệ thống sẽ được so sánh trực tiếp với giá trị chiều sâu thực tế bóc tách từ ma trận dữ liệu nguyên bản để lượng hóa sai số.

Độ chính xác và tính ổn định hình học của hệ thống Fusion được đánh giá một cách toàn diện thông qua hai chỉ số thống kê tiêu chuẩn là sai số tuyệt đối trung bình (MAE) và sai số bình phương tối thiểu trung bình (RMSE), trong đó RMSE đóng vai trò quan trọng trong việc phát hiện các điểm lỗi biên lớn ở vùng góc quét rộng. Thực nghiệm trong môi trường tĩnh phòng thí nghiệm với dải khoảng cách từ 0.5m đến 3.0m cho thấy sai số MAE toàn cục duy trì ổn định dưới ngưỡng 40mm. Cụ thể, khi mặt phẳng trực diện cách hệ thống 1.0m, thuật toán đạt độ chính xác cao nhất với MAE khoảng 12.45mm và RMSE ở mức 14.10mm, khiến toàn bộ các điểm chiếu đều hiển thị trạng thái màu xanh lá cây đồng nhất. Khi khoảng cách mục tiêu tăng lên mức 2.0m, sai số MAE dịch chuyển lên mức 18.30mm và đạt ngưỡng 24.12mm khi đối diện với các cấu trúc vật thể có dạng hình học góc cạnh phức tạp.

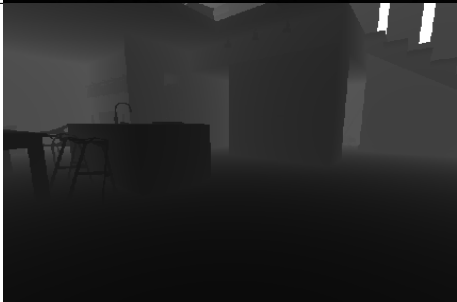
Phân tích sâu vào bản chất toán học cho thấy sai số của hệ thống có xu hướng tăng dần theo khoảng cách vật lý do mật độ điểm quét bị thưa dần theo quy luật quang học, kết hợp với hiện tượng suy giảm độ phân giải lưới hồng ngoại của cảm biến RealSense ở vùng biên xa. Thêm vào đó, tại ranh giới sắc nét của các vật thể, hiện tượng nhiễu ranh giới xuất hiện làm phát sinh các điểm hiển thị màu đỏ cục bộ do sự lệch pha thời gian siêu nhỏ giữa tốc độ quét vòng của LiDAR với nhịp bắt khung hình ảnh của camera, cùng sai số làm tròn pixel phát sinh trong hàm chiếu tuyến tính. Tóm lại, thuật toán đánh giá toàn bộ điểm quét đã chứng minh tính đúng đắn của mô hình calibration này. Các chỉ số sai số MAE và RMSE thực tế hoàn toàn nằm trong ngưỡng cho phép đối với các bài toán Camera-Lidar Fusion.

Chương 4

XÂY DỰNG BỘ DỮ LIỆU THỰC TẾ VÀ QUY TRÌNH TIỀN XỬ LÝ

4.1 SỰ KHÁC BIỆT GIỮA DỮ LIỆU GIẢ LẬP VÀ THỰC TẾ

Bảng 4.1.1: Bảng so sánh dữ liệu ảnh giữa Dataset thực tế và Dataset mô phỏng

Loại Dataset	Ảnh RGB	Ảnh Depth
Ảnh thu thực tế		
Ảnh thu mô phỏng		

Trong bài báo gốc của hệ thống G2-MonoDepth cũng như nhiều nghiên cứu lý thuyết khác về Depth Completion, bộ dữ liệu huấn luyện thường được xây dựng thông qua phương pháp giả lập cắt dòng (1D Sparse Simulation hoặc Sampling) từ một bản đồ độ sâu đầy hoàn chỉnh có sẵn. Thuật toán giả lập này đơn thuần lấy mẫu đều hoặc trích xuất cơ học một hàng pixel cố định trên ảnh độ sâu để đại diện cho đường quét của LiDAR 2D đơn dòng. Cách tiếp cận này tạo ra các điểm đo có tọa độ hình học lý tưởng, phân bố hoàn hảo và khít tuyệt đối với ma trận ảnh màu RGB trên phương diện kỹ thuật số.

Tuy nhiên, mô hình giả lập lý thuyết này bộc lộ những hạn chế nghiêm trọng khi triển khai

hệ thống nhận thức robot trong môi trường thế giới thực do bỏ qua hoàn toàn các sai số vật lý bản chất của cảm biến. Trong thực tế, phép chiếu hình học dựa trên ma trận Extrinsic thu được từ quy trình hiệu chuẩn luôn tồn tại một sai số nhất định. Sai số này khiến các tọa độ điểm quét thực tế của LiDAR khi chiếu lên mặt phẳng ảnh pixel bị lệch từ vài pixel cho đến hàng chục pixel so với vị trí biên cạnh thực tế của vật thể, điều mà dữ liệu giả lập hoàn hảo không bao giờ có được.

Hơn thế nữa, cảm biến LiDAR thật khi vận hành luôn phải đối mặt với hiện tượng nhiễu hấp thụ và nhiễu phản xạ bề mặt vật lý đặc thù. Khi tia laser va đập vào các bề mặt hấp thụ ánh sáng mạnh (như vật thể màu đen), bề mặt phản xạ (kính, kim loại bóng) hoặc bắn trượt qua các cạnh góc sắc nét, dữ liệu khoảng cách trả về sẽ bị mất hoàn toàn (tạo ra các lỗ trống cục bộ trên dòng quét) hoặc sinh ra các điểm nhiễu động đột biến. Do đó, việc xây dựng một bộ dữ liệu thực tế thu thập từ phần cứng thật là bước đi bắt buộc để mô hình AI học được các đặc tính chịu nhiễu hình học này, tránh hiện tượng sụt giảm hiệu năng hệ thống khi đưa vào kịch bản triển khai thực tế.

4.2 QUY TRÌNH THU THẬP DỮ LIỆU THỰC TẾ

4.2.1 Thiết bị và bố trí thực nghiệm

Hệ thống thu thập dữ liệu của đề tài được thiết lập dựa trên sự kết hợp giữa cụm cảm biến lai cao cấp đóng vai trò tạo nguồn dữ liệu và thiết bị lưu trữ xử lý biên. Phần cứng bao gồm:

- Camera độ sâu chuyên dụng Intel RealSense D435 đóng vai trò ghi nhận luồng ảnh màu RGB sắc nét và xuất ra bản đồ độ sâu chuẩn Ground Truth độ phân giải cao dạng 16-bit.
- Cảm biến RPLidar A1 đặt đồng trục, quét xoay vòng liên tục để thu nhận khoảng cách vật lý thực tế của môi trường xung quanh.
- Bo mạch máy tính trung gian cấu hình cao chịu trách nhiệm đồng bộ, chạy mã nguồn trích xuất dữ liệu thời gian thực thông qua các cổng giao tiếp USB tốc độ cao.

Luồng xử lý phần mềm thu thập dữ liệu được hiện thực hóa thông qua kịch bản lập trình trong tệp mã nguồn `9_fusion_capture_filtered.py`. Để loại bỏ hoàn toàn sai số parallax nội bộ của thiết bị RealSense, chương trình ứng dụng toán tử ép luồng `rs.align(rs.stream.color)` để ép toàn bộ bản đồ độ sâu phần cứng về trùng khít 100% với hệ tọa độ và góc nhìn của cảm biến màu RGB. Đặc biệt, luồng ảnh độ sâu từ RealSense D435 được cấu hình trực tiếp qua Preset High-Density phần cứng nhằm tối đa hóa mật độ điểm đo, đồng thời đi qua các bộ lọc xử lý nâng cao gồm bộ lọc không gian (Spatial Filter) để tăng cường độ mịn bề mặt và bộ lọc thời gian (Temporal

Filter) nhằm giảm thiểu hiện tượng nhấp nháy pixel độ sâu giữa các khung hình liên tiếp. Sau khi người dùng nhấn phím lệnh chụp, chương trình sẽ đồng bộ hóa dữ liệu, lọc bỏ các điểm đo nằm ngoài phạm vi hoạt động vật lý tin cậy thông qua hai biến giới hạn MIN_DEPTH (150mm) và MAX_DEPTH (10000mm), đảm bảo Ground Truth thu được có độ sạch học thuật cao.

4.2.2 Script thu thập dữ liệu

Công tác thực nghiệm thu thập dữ liệu được triển khai theo Script indoor hệ thống nhằm bao phủ tối đa các hình thái không gian mà các dòng robot dịch vụ thường xuyên đối mặt. Các bối cảnh bao gồm phòng làm việc, phòng lab, hành lang tòa nhà, và các không gian phòng khách sinh hoạt. Trong mỗi môi trường, góc nhìn của hệ thống cảm biến được thay đổi liên tục từ các góc nhìn trực diện vào mảng tường phẳng cho đến các góc chéo kết hợp đa dạng nhiều tầng vật thể có cấu trúc hình học phức tạp như bàn ghế, tủ tài liệu, màn hình máy tính và cây cối.

Để tăng cường năng lực tổng quát hóa của mạng neural, script cũng chủ động phân hóa các điều kiện ngoại cảnh tác động. Dữ liệu được ghi nhận dưới nhiều trạng thái ánh sáng khác nhau: ánh sáng tự nhiên ban ngày, ánh sáng đèn điện huỳnh quang văn phòng, và trường hợp thiếu sáng ban đêm. Khoảng cách từ cụm cảm biến đến các vật cản cũng được phân bố đều từ cự ly cực gần (dưới 1 mét) cho đến cự ly xa (khoảng 5 đến 7 mét), giúp phân phối histogram độ sâu của tập dữ liệu đạt được độ trải rộng ổn định trên phương diện thống kê không gian.

4.2.3 Cấu trúc dữ liệu thu được

Sau quy trình sàng lọc và hậu xử lý các mẫu dữ liệu lỗi, bộ dữ liệu độ sâu thực tế được đóng gói hoàn chỉnh với quy mô tổng thể **621 mẫu hình học**. Dựa trên các nguyên tắc phân rã tập dữ liệu học máy tiêu chuẩn, bộ dữ liệu được chia tách thành hai phân đoạn độc lập:

- **Tập huấn luyện (Training Set):** Chiếm phần lớn tỷ trọng với **600 mẫu**, được sử dụng trực tiếp trong luồng huấn luyện tối ưu hóa trọng số mô hình
- **Tập kiểm thử (Test/Benchmark Set):** Gồm **21 mẫu**, được cô lập hoàn toàn khỏi quá trình huấn luyện và chỉ được giải phóng ở giai đoạn thực nghiệm cuối cùng để đánh giá năng lực suy diễn không gian của thuật toán.

Về cấu trúc lưu trữ thông tin, mỗi mẫu dữ liệu trong dataset là một thực thể đồng bộ bao gồm ba tệp tin thành phần định dạng chuẩn:

- Tệp ảnh màu màu sắc **rgb/id.png** chứa ảnh thể hiện cấu trúc ngữ nghĩa môi trường.

- Tập ảnh độ sâu Ground Truth chuẩn **depth/id.png** lưu dưới định dạng ảnh 16-bit đơn kênh.
- Tập tin cấu trúc tọa độ **lidar/id_lidar.json**. File JSON này lưu trữ mảng danh sách các pixel ảnh mã hóa hình học dạng [px, py] trích xuất từ trường dữ liệu **image_pixel_points** sau khi đã đi qua ma trận biến đổi và phép chiếu phối cảnh không gian được mô tả ở Chương 3, kèm theo siêu dữ liệu khoảng cách vật lý thực tế thu được từ luồng quét của cảm biến RPLidar A1.

Toàn bộ bộ dữ liệu thực tế thu được (Private Real Dataset) bao gồm đầy đủ các luồng thông tin đồng bộ trên đã được chúng tôi đóng gói, lưu trữ và chia sẻ phục vụ mục đích nghiên cứu tại đường dẫn Google Drive: https://drive.google.com/drive/folders/12cQf00eKp5Ni4W32pWUUIUHb-p_GoKup?usp=sharing.

4.3 THIẾT KẾ CÔNG CỤ XỬ LÝ DỮ LIỆU CHUYÊN DỤNG (DATA_TOOLS_REAL.PY)

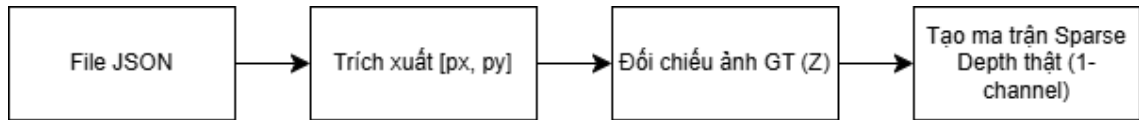
Để khai thác dữ liệu thu thập từ hệ thống cảm biến thực tế phục vụ huấn luyện mô hình G2-MonoDepth, một bộ tải dữ liệu chuyên dụng được xây dựng trong tệp `data_tools_real.py` thông qua lớp `PrivateRealDataset`. Bộ tải dữ liệu này có nhiệm vụ đồng bộ hóa ba nguồn thông tin chính gồm ảnh màu RGB, ảnh độ sâu Ground Truth và dữ liệu LiDAR thực được lưu dưới dạng tệp JSON.

Trong hàm khởi tạo (`__init__`), chương trình tiến hành quét toàn bộ thư mục dữ liệu và thiết lập mối liên kết giữa ba loại tệp có cùng định danh mẫu dữ liệu. Chỉ những mẫu chứa đầy đủ ảnh RGB, ảnh độ sâu và dữ liệu LiDAR mới được đưa vào danh sách huấn luyện. Cơ chế này đảm bảo tính toàn vẹn của dữ liệu đầu vào trước khi đưa vào quá trình huấn luyện mạng nơ-ron.

4.3.1 Đọc và phân tích tọa độ LiDAR từ tập tin JSON

Khác với các bộ dữ liệu mô phỏng sử dụng phương pháp lấy mẫu thưa trực tiếp từ ảnh độ sâu dày đặc, hệ thống được đề xuất sử dụng dữ liệu LiDAR thực đã được hiệu chỉnh và chiếu lên mặt phẳng ảnh RGB. Các điểm LiDAR sau khi hiệu chỉnh được lưu dưới dạng danh sách tọa độ pixel trong tệp JSON.

Quy trình xử lý nạp dữ liệu tại mỗi bước lặp của hàm `__getitem__` được thực thi tuần tự theo đồ thị tính toán sau:



Hình 4.3.1: Quy trình xử lý dữ liệu từ file JSON của Lidar

Quy trình xử lý tổng quát được mô tả trong Hình 4.3.1, bao gồm các bước đọc dữ liệu JSON, trích xuất tọa độ điểm ảnh, khởi tạo ma trận độ sâu thưa và gán giá trị độ sâu tương ứng từ ảnh Ground Truth.

Tại mỗi lần truy xuất dữ liệu thông qua hàm `__getitem__`, chương trình thực hiện đọc ảnh RGB và ảnh độ sâu Ground Truth tương ứng. Sau đó, bộ tải dữ liệu mở tệp JSON để trích xuất danh sách tọa độ điểm ảnh LiDAR:

```

with open(lidar_path, 'r') as f:
    lidar_data = json.load(f)
    pixel_points = lidar_data["labels"]["image_pixel_points"]
  
```

Sau khi thu được tập hợp tọa độ điểm ảnh, hệ thống khởi tạo một ma trận độ sâu rỗng có cùng kích thước với ảnh Ground Truth:

```

sparse = torch.zeros_like(gt)
  
```

Tiếp theo, chương trình duyệt qua từng điểm LiDAR, kiểm tra tính hợp lệ của tọa độ và lấy giá trị độ sâu tại đúng vị trí đó từ ảnh Ground Truth:

```

for (px, py) in pixel_points:
    if 0 <= px < w and 0 <= py < h:
        depth_val = gt[0, py, px].item()
        if depth_val > 0:
            sparse[0, py, px] = depth_val
  
```

Kết quả của quá trình này là một bản đồ độ sâu thưa, trong đó vị trí các điểm đo được kế thừa từ dữ liệu LiDAR thực, còn giá trị độ sâu được lấy trực tiếp từ ảnh Ground Truth. Phương pháp này giúp duy trì phân bố không gian đặc trưng của cảm biến LiDAR trong khi vẫn đảm bảo tính nhất quán về thang đo độ sâu giữa dữ liệu đầu vào và dữ liệu nhãn.

4.3.2 Kỹ thuật tăng cường dữ liệu đồng bộ 5 kênh (Spatial Transform)

Trong bài toán ước lượng độ sâu đơn ảnh, việc tăng cường dữ liệu đóng vai trò quan trọng nhằm cải thiện khả năng tổng quát hóa của mô hình. Tuy nhiên, do dữ liệu huấn luyện bao gồm đồng thời ảnh màu RGB, ảnh độ sâu Ground Truth và bản đồ độ sâu thưa từ LiDAR, mọi phép biến đổi hình học đều phải được thực hiện đồng bộ trên toàn bộ các thành phần dữ liệu. Nếu mỗi thành phần được biến đổi độc lập, mối quan hệ hình học giữa các điểm ảnh sẽ bị phá vỡ, dẫn đến sai lệch trong quá trình học của mạng nơ-ron.

Để giải quyết vấn đề này, lớp RealTransformUtils được xây dựng nhằm thực hiện các phép biến đổi không gian trên toàn bộ dữ liệu dưới dạng một khối tensor thống nhất.

```
self._spatial_transform = trans.Compose([
    trans.RandomCrop(size),
    RealResizedCropRGBDR(size, (0.64, 1.0), antialias=True),
    trans.RandomHorizontalFlip(0.5),
])
```

Pipeline tăng cường dữ liệu được xây dựng trong lớp RealTransformUtils, bao gồm ba phép biến đổi chính là cắt ảnh ngẫu nhiên (RandomCrop), thay đổi kích thước kết hợp cắt vùng ảnh (RandomResizedCrop) và lật ảnh theo phương ngang (RandomHorizontalFlip). Các phép biến đổi này được áp dụng đồng thời lên toàn bộ tensor chứa dữ liệu RGB, Ground Truth và Sparse Depth nhằm đảm bảo tính đồng bộ không gian giữa các kênh dữ liệu.

```
gt_mask = (img[3:4, :, :] != 0).float()
raw_mask = (img[4:5, :, :] != 0).float()

all_ch = torch.cat([img, gt_mask, raw_mask], dim=0)
all_ch = TF.resized_crop(
    all_ch, i, j, h, w, self.size,
    self.interpolation, antialias=self.antialias
)
```

Để tránh hiện tượng xuất hiện các giá trị độ sâu giả do phép nội suy trong quá trình thay đổi kích thước ảnh, lớp RealResizedCropRGBDR sử dụng thêm hai mặt nạ nhị phân xác định vị trí các điểm độ sâu hợp lệ trong Ground Truth và Sparse Depth. Các mặt nạ này được ghép cùng dữ liệu gốc thành một tensor 7 kênh và được biến đổi đồng thời bằng cùng một tập tham số hình học.

```
gt_mask = (all_ch[5, :, :] >= 0.99).float()
raw_mask = (all_ch[6, :, :] >= 0.99).float()
```

```
rgb = all_ch[:3, :, :]  
gt = (all_ch[3, :, :] * gt_mask).unsqueeze(0)  
raw = (all_ch[4, :, :] * raw_mask).unsqueeze(0)
```

Sau khi hoàn thành phép biến đổi, các mặt nạ được sử dụng để khôi phục lại những vùng dữ liệu hợp lệ và loại bỏ các giá trị phát sinh do nội suy. Nhờ đó, cấu trúc hình học của dữ liệu độ sâu được bảo toàn, đồng thời vẫn đảm bảo tính đồng bộ giữa ảnh RGB, Ground Truth và Sparse Depth trong toàn bộ quá trình tăng cường dữ liệu.

Chương 5

FINE-TUNING VÀ TỐI ƯU HÓA MÔ HÌNH

Chương 5 đóng vai trò cầu nối thực nghiệm nhằm hiện thực hóa các nghiên cứu lý thuyết và kết quả hiệu chuẩn hình học từ các chương trước thành một giải pháp hệ thống phân tán hoàn chỉnh. Mặc dù kiến trúc G2-MonoDepth gốc đã được chứng minh là hiệu quả trên các tập dữ liệu chuẩn hóa quy mô lớn bằng phần cứng máy tính hiệu năng cao, việc triển khai suy diễn thực tế phục vụ cho các tác vụ điều hướng thời gian thực của robot di động vẫn đối mặt với hai thách thức kỹ thuật lớn:

1. **Sự khác biệt về phân phối dữ liệu (Domain Gap):** Trọng số huấn luyện sẵn (pre-trained weights) của mô hình gốc được tối ưu hóa dựa trên các tập dữ liệu chuẩn hóa hoặc mô phỏng. Do đó, mô hình cần được huấn luyện tinh chỉnh thích ứng miền (domain fine-tuning) để hòa hợp với các đặc tính vật lý đặc thù trong không gian thực tế, như nhiễu phản xạ từ cảm biến RPLidar A1 và cấu hình quang học thấu kính sau hiệu chuẩn hình học.
2. **Độ phức tạp của đồ thị động và yêu cầu phản hồi thời gian thực:** Kiến trúc mạng UNet gốc vận hành dựa trên các đồ thị tính toán động của framework PyTorch với chi phí khởi tạo lớn và không tối ưu hóa phân bổ bộ nhớ. Để đạt được tần số phản hồi thời gian thực trong các ứng dụng di động, toàn bộ pipeline xử lý cần được thiết kế lại theo hướng phân tách Edge-Server gọn nhẹ, kết hợp quy trình đơn giản hóa và biên dịch đồ thị tĩnh sang TensorRT Engine để khai thác tối đa sức mạnh tính toán song song phần cứng GPU.

Các nội dung kỹ thuật chi tiết cùng kết quả triển khai tương ứng sẽ lần lượt được trình bày trong các mục dưới đây.

5.1 QUY TRÌNH HUẤN LUYỆN FINE-TUNING G2-MONODEPTH MODEL TRÊN TẬP DỮ LIỆU THỰC TẾ

5.1.1 Thiết lập môi trường và cấu hình huấn luyện

Để thực hiện fine-tuning mô hình G2-MonoDepth trên bộ dữ liệu thực tế tự xây dựng, việc thiết lập một môi trường tính toán đồng nhất giữa pha huấn luyện (Cloud GPU) và pha triển khai (Edge Device) là yêu cầu kỹ thuật bắt buộc nhằm triệt tiêu các sai lệch tương thích phần mềm.

Môi trường phần cứng và đám mây

Quá trình fine-tuning được triển khai trên nền tảng điện toán đám mây Kaggle Kernels, khai thác cấu hình tăng tốc phần cứng gồm hai GPU song song NVIDIA Tesla T4. Mỗi GPU sở hữu dung lượng bộ nhớ đồ họa là 16GB GDDR6 (tổng cộng 32GB VRAM), cung cấp băng thông truyền tải dữ liệu đạt mức 320 GB/s. Hệ thống máy chủ ảo (VM) đi kèm được trang bị CPU Intel Xeon 4 nhân, bộ nhớ hệ thống (RAM) 29GB, chạy trên hệ điều hành Ubuntu Linux.

Đảm bảo tính nhất quán môi trường phần mềm (Environment Parity)

Hệ điều hành mặc định trên thiết bị nhúng Raspberry Pi 5 là Raspberry Pi OS (Debian Bookworm) sử dụng Python 3.11 làm trình thông dịch tiêu chuẩn hệ thống. Trong khi đó, môi trường ảo hóa mặc định của Kaggle Kernels tại thời điểm thực nghiệm sử dụng phiên bản khác là Python 3.10 (hoặc Python 3.12, tùy thuộc vào version của Kaggle). Sự bất đồng bộ này có thể dẫn đến các lỗi nghiêm trọng về cấu trúc đồ thị khi xuất mô hình (ONNX export), lỗi tuần tự hóa trạng số (serialization), hoặc không tương thích các thư viện phụ thuộc.

Để đạt được sự nhất quán tuyệt đối về môi trường phần mềm giữa Cloud và Edge, chúng tôi đã tiến hành nâng cấp hệ thống Python trên Kaggle lên phiên bản 3.11 thông qua các bước kỹ thuật sau:

1. **Tích hợp kho lưu trữ bổ sung:** Sử dụng PPA `deadsnakes` để nạp các gói phát triển của Python 3.11 bao gồm `python3.11-dev`, `python3.11-distutils` và `python3.11-venv` phục vụ biên dịch các gói thư viện C-extension.
2. **Định cấu hình trình chuyển đổi thay thế (update-alternatives):** Định nghĩa mức độ ưu tiên cho trình thông dịch trong hệ thống. Hệ thống mặc định Python 3.10 được gán độ

ưu tiên mức 1, trong khi Python 3.11 mới cài đặt được cấu hình độ ưu tiên mức 2. Do đó, cần ép buộc hệ thống nhận diện Python 3.11 làm trình thông dịch mặc định cục bộ:

```
sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/
python3.10 1
sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/
python3.11 2
sudo update-alternatives --set python3 /usr/bin/python3.11
```

3. **Cài đặt trình quản lý gói pip và thư viện học sâu:** Tải và cài đặt pip tương thích riêng cho nhánh Python 3.11 qua tập tin bootstrap `get-pip.py`, sau đó cài đặt các thư viện lõi phục vụ huấn luyện bao gồm PyTorch (`torch`), TensorBoard và thư viện thị giác máy tính OpenCV (`opencv-python`) [6].

Kỹ thuật huấn luyện song song dữ liệu phân tán (Distributed Data Parallel - DDP)

Đối với cấu hình đa GPU (Multi-GPU), phương pháp song song hóa truyền thống bằng `torch.nn.DataParallel` (DP) thường gặp phải các giới hạn về hiệu năng do cơ chế đơn tiến trình đa luồng (Single-Process Multi-Threading). Ở cơ chế DP, luồng xử lý chính bị cản trở bởi khóa thông dịch viên toàn cục (Global Interpreter Lock - GIL) của Python. Đồng thời, một GPU chủ (thường là GPU 0) phải đảm nhận vai trò phân phối dữ liệu (`scatter`) và thu thập độ dốc (`gather gradients`), dẫn đến sự mất cân bằng bộ nhớ VRAM nghiêm trọng và làm giảm tốc độ truyền thông nội bộ.

Để giải quyết triệt để rào cản này, chúng tôi áp dụng kỹ thuật Huấn luyện song song dữ liệu phân tán (Distributed Data Parallel - DDP) của PyTorch trong file code sử dụng để fine-tuning chuyên biệt `finetune.py`. Các đặc điểm kỹ thuật cốt lõi của DDP trong thực nghiệm này bao gồm:

- **Độc lập tiến trình (Multi-Process):** Mỗi GPU (Tesla T4) vận hành trên một tiến trình hệ điều hành riêng biệt (được xác định bằng mã số định danh cục bộ `local_rank`), giải phóng hoàn toàn sự phụ thuộc vào khóa GIL của Python.
- **Đồng bộ hóa hiệu quả bằng thư viện NCCL:** DDP sử dụng bộ giao tiếp truyền thông tập thể NVIDIA Collective Communications Library (NCCL) làm giao diện kết nối phụ trợ (backend). Thay vì dồn dữ liệu về một GPU chủ, NCCL áp dụng thuật toán Ring-AllReduce để phân phối đồng đều việc tính toán trị trung bình của độ dốc (`gradients`) trên toàn bộ mạng lưới GPU một cách song song.

- **Tránh trùng lặp dữ liệu bằng bộ lấy mẫu phân tán (DistributedSampler):** Để đảm bảo mỗi tiến trình xử lý một phân đoạn dữ liệu độc lập và không chồng lấn trong từng chu kỳ bước huấn luyện, bộ nạp dữ liệu DataLoader được cấu hình tích hợp lớp bộ lấy mẫu phân tán `torch.utils.data.distributed.DistributedSampler`.

Khởi tạo trọng số bằng phương pháp Transfer Learning

Do quy mô bộ dữ liệu camera-LiDAR thực tế thu thập được tương đối giới hạn (600 mẫu huấn luyện), việc huấn luyện mô hình G2-MonoDepth từ đầu không khả thi. Khởi tạo ngẫu nhiên các trọng số sẽ dễ khiến mạng neural rơi vào các cực trị cục bộ hoặc gặp hiện tượng overfitting, không thể nhận diện được các đặc trưng hình học ba chiều phức tạp của cảnh quan.

Chúng tôi áp dụng giải pháp học chuyển giao bằng cách nạp toàn bộ trọng số được huấn luyện sẵn (pre-trained weights) của mô hình G2-MonoDepth gốc [1] (phiên bản được tối ưu hóa trước đó trên tập dữ liệu chuẩn hóa KITTI [8]). Việc khởi tạo này mang lại lợi thế kép:

1. Kế thừa các bộ lọc trích xuất đặc trưng bậc thấp đã học tốt (như cạnh biên, cấu trúc góc, kết cấu bề mặt và các quy luật phối cảnh không gian).
2. Giảm thiểu số lượng tham số cần phải hiệu chỉnh mạnh, từ đó rút ngắn đáng kể thời gian hội tụ và định vị chính xác thang đo độ sâu vật lý thực tế từ cảm biến LiDAR 2D thưa.

Sơ đồ code cấu hình và khởi tạo tiến trình DDP trong tập tin thực thi tinh chỉnh `finetune.py` được thiết lập dựa trên khung cấu trúc sau:

```
import os
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data.distributed import DistributedSampler

def setup_ddp():
    # Kaggle tự động thiết lập các biến môi trường khi chạy GPU T4 x2
    world_size = int(os.environ.get("WORLD_SIZE", 2))
    rank = int(os.environ.get("RANK", 0))
    local_rank = int(os.environ.get("LOCAL_RANK", 0))

    # Khởi tạo nhóm tiến trình với giao diện truyền thông NCCL
    dist.init_process_group(
        backend="nccl",
        init_method="env://",
```

```

        world_size=world_size,
        rank=rank
    )

    torch.cuda.set_device(local_rank)
    return local_rank, world_size

# Trong hàm thực thi chính (main function của finetune.py)
local_rank, world_size = setup_ddp()

# Khởi tạo và nạp trọng số pre-trained cho model
model = G2MonoDepthModel()
model.load_state_dict(torch.load("pretrained_g2_monodepth.pth",
    map_location="cpu"))
model = model.to(local_rank)

# Bao bọc mô hình bằng lớp DDP của PyTorch
model = DDP(model, device_ids=[local_rank], output_device=local_rank)

# Cấu hình bộ lấy mẫu phân tán cho dữ liệu đầu vào thu thập được
sampler = DistributedSampler(dataset, num_replicas=world_size, rank=
    local_rank)
dataloader = DataLoader(dataset, batch_size=cfg.batch_size, sampler=sampler
    )

```

Listing 5.1: Cấu hình khởi tạo tiến trình song song phân tán (DDP) bằng PyTorch trong code `finetune.py`.

5.1.2 Chiến lược thiết lập siêu tham số

Trong quá trình fine-tuning một mô hình học sâu đã hội tụ trên tập dữ liệu lớn sang một miền dữ liệu mới nhỏ hơn, việc lựa chọn các siêu tham số (hyperparameters) đóng vai trò quyết định đến tính ổn định của quá trình tối ưu hóa. Các tham số này được chúng tôi tập trung quản lý và cấu hình thống nhất trong lớp đối tượng cấu hình tại tập tin `config.py`.

Cơ chế chống hiện tượng Catastrophic Forgetting

Do bộ dữ liệu camera-LiDAR thực tế tự xây dựng có quy mô tương đối nhỏ ($N = 600$ cặp mẫu ảnh màu và điểm quét thưa), việc thiết lập một tốc độ học (learning rate - LR) lớn (ví dụ: $\eta \geq 10^{-4}$) sẽ khiến thuật toán tối ưu hóa thực hiện các bước cập nhật trọng số quá lớn. Điều này dễ gây ra hiện tượng catastrophic forgetting [9], làm phá vỡ cấu trúc đặc trưng không gian tổng quát và các bộ lọc hình học mà mô hình G2-MonoDepth đã tích lũy được trong pha huấn luyện trước trên tập dữ liệu KITTI [8] đồ sộ.

Để giải quyết vấn đề này, chúng tôi áp dụng chiến lược thiết lập tốc độ học ban đầu cực nhỏ ($\eta_0 = 2 \times 10^{-5}$) kết hợp với thuật toán tối ưu hóa AdamW [10]. Thuật toán AdamW điều chỉnh trọng số dựa trên công thức:

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_t \right) \quad (5.1.1)$$

Trong đó, θ_t là ma trận trọng số tại bước t , $\hat{m}_t$ và $\hat{v}_t$ lần lượt là các ước lượng moment bậc nhất và bậc hai của độ dốc, λ là hệ số suy giảm trọng số (weight decay) được cấu hình ở mức 0,05 nhằm hạn chế sự bùng nổ trọng số, kiểm soát chuẩn hóa L_2 và giảm thiểu tối đa hiện tượng overfitting trên tập dữ liệu nhỏ.

Tiến trình huấn luyện tinh chỉnh và phân tích hội tụ

Tiến trình tinh chỉnh sâu (deep finetuning) được thực hiện liên tục trong duy nhất một mạch gồm 95 epoch nhằm đảm bảo đồ thị tính toán hội tụ một cách mịn màng và triệt để nhất. Các thiết lập siêu tham số chi tiết của tiến trình này được tổng hợp trong Bảng 5.1.1.

Bảng 5.1.1: Thiết lập siêu tham số hệ thống trong quá trình huấn luyện tinh chỉnh sâu.

Siêu tham số (Hyperparameters)	Giá trị thiết lập
Quy mô bộ dữ liệu tinh chỉnh (N)	600 cặp mẫu
Tốc độ học ban đầu (η_0)	2×10^{-5}
Hệ số suy giảm trọng số (λ)	0,05
Thuật toán tối ưu hóa	AdamW ($\beta_1 = 0,9, \beta_2 = 0,999$)
Tổng số chu kỳ huấn luyện (Epochs)	95
Kích thước lô xử lý (Batch Size)	8

Dựa trên sự biến động thực tế của hàm loss và các sai số depth ghi nhận từ log hệ thống, tiến trình huấn luyện 95 epoch này có thể được phân tích qua ba giai đoạn phát triển rõ rệt:

1. **Giai đoạn 1 - Thích ứng miền dữ liệu nhanh (Epoch 1 đến 30):** Trong 30 epoch đầu tiên, hàm loss giảm nhanh chóng từ mức khởi điểm 1,7789 xuống mức ổn định đầu tiên xung

quanh mức 1,14 - 1,18 . Đây là giai đoạn mô hình tái cấu trúc các trọng số ở tầng nông để làm quen với sự phân bố thừa thớt của các điểm quét LiDAR G2 thực tế tự xây dựng, đồng thời các hệ số tỷ lệ thang đo (scale factors) bắt đầu được định hình ổn định .

2. **Giai đoạn 2 - Bảo hòa và tinh chỉnh chi tiết (Epoch 31 đến 70):** Ở giai đoạn trung gian này, tốc độ học giảm nhẹ theo thời gian dưới sự điều phối của scheduler, đưa hàm loss đi vào vùng bảo hòa ổn định (plateau) dao động quanh mức 1,06 đến 1,10 . Đây là thời điểm cơ chế khởi tạo kết nối tắt ReZero thích nghi sâu sắc với sự lệch pha của dữ liệu chiều thừa thớt mà không làm biến dạng hay làm mất đi các ranh giới hình học tổng quát đã học từ trước .
3. **Giai đoạn 3 - Hội tụ sâu và tối ưu hóa (Epoch 71 đến 95):** Trong 25 epoch cuối cùng, đồ thị suy giảm đạt điểm hội tụ sâu nhất . Hàm loss chạm mốc cực tiểu cục bộ là 0,9935 tại Epoch 78, đồng thời các sai số chiều sâu tương đối rdepth giảm xuống mức tối ưu là 0,4275 . Sự ổn định của các chỉ số trong giai đoạn này chứng minh mô hình đã đạt trạng thái hội tụ tốt, sẵn sàng cho việc xuất và đóng gói mô hình sang các định dạng ONNX và TensorRT.

Cấu hình kích thước lô (Batch Size) và giới hạn phân cứng

Một siêu tham số quan trọng khác được định nghĩa trong `config.py` là `batch_size`. Do G2-MonoDepth là một mô hình tích hợp kết hợp các khối xử lý không gian phức tạp, việc xử lý ảnh ở độ phân giải lớn (512×896 hoặc 320×448) tiêu thụ một lượng tài nguyên bộ nhớ đồ họa rất lớn trong pha lan truyền ngược (backward pass).

Để tối ưu hóa dung lượng 16GB VRAM của mỗi GPU Tesla T4 trên Kaggle, chúng tôi thiết lập `batch_size = 4` cho mỗi tiến trình xử lý GPU đơn lẻ. Nhờ cấu hình song song phân tán DDP trên 2 GPU, kích thước lô hiệu dụng thực tế (effective batch size) được nâng lên mức:

$$B_{effective} = batch_size \times world_size = 4 \times 2 = 8 \quad (5.1.2)$$

Cấu hình này đảm bảo phân bổ bộ nhớ ổn định trên mỗi card đồ họa (tránh lỗi tràn bộ nhớ đồ họa - Out of Memory), đồng thời cung cấp đủ số lượng mẫu trong một bước lặp để ước lượng độ dốc của hàm loss có độ lệch chuẩn nhỏ, hỗ trợ quá trình hội tụ diễn ra thuận lợi hơn.

5.1.3 Cấu hình hàm loss tùy chỉnh và sự thích ứng với dữ liệu thực tế

Để huấn luyện tinh chỉnh thành công mô hình G2-MonoDepth trên dữ liệu thực tế, việc hiểu rõ bản chất toán học của các hàm mất mát gốc và điều chỉnh chúng cho phù hợp với đặc tính vật lý của cảm biến là một bước đi then chốt. Trong phần này, chúng tôi phân tích cấu trúc hàm loss của mô hình gốc và trình bày phương pháp cấu hình hàm loss trong quá trình fine-tuning model.

Phân tích toán học hàm loss của mô hình G2-MonoDepth gốc

Mô hình baseline sử dụng hai thành phần loss chính để giám sát quá trình hoàn thiện bản đồ độ sâu:

1. Hàm loss dữ liệu thưa có trọng số (Weighted Data Loss):

Lớp `WeightedDataLoss` tính toán sai số tuyệt đối (L_1 Loss) có sử dụng mặt nạ nhị phân `hole` (M , nhận giá trị 1 tại vị trí khuyết dữ liệu cần hoàn thiện và 0 tại vị trí đã có điểm neo độ sâu). Luồng tính toán gốc được đặc tả như sau:

```
sampled_output = hole * output + (1.0 - hole) * target
loss = self.loss_fun(sampled_output, target)
```

Gọi $\tilde{D}$ là độ sâu dự đoán (`output`), D^* là độ sâu Ground Truth (`target`). Tensor sai lệch thực tế giữa $D_{sampled}$ và D^* được tính toán qua phép nhân chập Hadamard ($\odot$):

$$\begin{aligned} |D_{sampled} - D^*| &= |(M \odot \tilde{D} + (1 - M) \odot D^*) - D^*| \\ &= |M \odot \tilde{D} + D^* - M \odot D^* - D^*| \\ &= |M \odot (\tilde{D} - D^*)| \end{aligned} \tag{5.1.3}$$

Hàm mất mát tuyệt đối trung bình chuẩn hóa theo tổng số điểm ảnh hợp lệ của mặt nạ khuyết dữ liệu ($N_{valid} = \sum M + \epsilon$) có dạng:

$$\mathcal{L}_{data_baseline} = \frac{1}{\sum M + \epsilon} \sum_{i,j} M_{i,j} |\tilde{D}_{i,j} - D^*_{i,j}| \tag{5.1.4}$$

Biểu thức này chứng minh rằng hàm loss gốc chỉ tập trung phạt sai số tại các vùng điểm ảnh khuyết dữ liệu đầu vào (`hole`), trong khi bỏ qua các điểm đã có dữ liệu neo thừa từ trước.

2. Hàm loss độ dốc đa quy mô (Weighted MS-Gradient Loss):

Lớp `WeightedMSGradLoss` sử dụng bộ lọc Sobel tính 3×3 thông qua lớp `Gradient2D` để tính toán đạo hàm không gian không tham gia tối ưu hóa trọng số (`requires_grad=False`):

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (5.1.5)$$

Sai số độ dốc được tính toán tuần tự trên $k = 4$ quy mô thu phóng khác nhau ($s_i = \frac{1}{2^i}$) dựa trên tensor residual đã được lọc qua mặt nạ khuyết dữ liệu ($R = M \odot (\tilde{D} - D^*)$):

$$\mathcal{L}_{grad_baseline} = \frac{1}{\sum M + \epsilon} \sum_{i=0}^{k-1} \left(\sum |K_x * R^{(i)}| + \sum |K_y * R^{(i)}| \right) \quad (5.1.6)$$

Cơ chế này ép buộc mô hình phải duy trì tính liên tục và độ sắc nét của các đường biên hình học bên trong các vùng cần hoàn thiện.

Hạn chế vật lý khi áp dụng hàm loss gốc vào hệ thống thực tế

Mặc dù hàm loss gốc hoạt động tốt trên các tập dữ liệu giả lập chuẩn hóa (như NYU-Depth-v2 [11] hoặc KITTI [8]), chúng tôi phát hiện hai hạn chế lớn khi triển khai thực tế trên hệ thống cảm biến tự xây dựng:

- **Vấn đề khuyết dữ liệu của Ground Truth thật:** Mô hình gốc giả định bản đồ Ground Truth (D^*) là hoàn chỉnh và không có lỗ thủng. Tuy nhiên, ảnh độ sâu thu từ camera Intel RealSense thực tế luôn tồn tại các vùng khuyết dữ liệu vật lý (do phản xạ bề mặt đen, gương kính hoặc vùng khuất bóng sập giữa camera phát tia hồng ngoại và camera thu nhận). Nếu áp dụng nguyên bản công thức `sampled_output` của mô hình gốc, các điểm khuyết này (có giá trị độ sâu bằng 0) sẽ bị tính vào hàm loss, gây ra hiện tượng sai lệch thang đo nghiêm trọng.
- **Độ thưa cực hạn của LiDAR 2D (RPLidar A1):** Khác với dữ liệu LiDAR 3D hoặc giả lập cắt dòng, cảm biến RPLidar A1 quét vòng chỉ cung cấp duy nhất một đường quét ngang 1D thưa thớt ở giữa ảnh. Điều này làm cho ma trận khuyết `hole` chiếm gần như toàn bộ diện tích ảnh ($> 99.9\%$). Nếu chỉ dựa vào sự giám sát độ dốc trên vùng khuyết một cách đơn thuần, mô hình rất khó lan truyền thang đo vật lý từ dòng quét trung tâm ra toàn bộ các góc phía trên và dưới của bức ảnh.

Cấu hình tinh chỉnh thích ứng trong *finetune.py*

Để khắc phục triệt để các hạn chế vật lý nêu trên, trong tập tin tinh chỉnh chuyên biệt *finetune.py*, chúng tôi đã thiết lập cấu hình thích ứng hàm loss dựa trên các nguyên tắc sau:

1. **Tích hợp mặt nạ hợp lệ của Ground Truth (Valid GT Mask):** Chúng tôi định nghĩa một mặt nạ hợp lệ bổ sung M_{valid_gt} chỉ nhận giá trị 1 tại những vị trí mà camera RealSense đo được khoảng cách thực tế ($D^* > 0$). Hàm loss dữ liệu thực tế được điều chỉnh để chỉ tính toán trên phần giao của hai mặt nạ:

$$M_{effective} = M_{hole_target} \cap M_{valid_gt} \quad (5.1.7)$$

Sự điều chỉnh này đảm bảo mô hình không bao giờ bị tối ưu sai lệch bởi các điểm nhiễu hoặc vùng khuyết vật lý của camera RealSense.

2. **Tăng tỷ trọng hàm loss độ dốc (Gradient Loss):** Do tính thừa thớt cực hạn của dòng quét LiDAR 2D đơn dòng, chúng tôi tăng tỷ trọng hệ số của hàm mất mát độ dốc đa quy mô (w_{grad}) trong cấu hình huấn luyện của *finetune.py*. Việc tăng tỷ trọng này đóng vai trò như một lực đẩy định hướng, ép buộc mạng Decoder phải lan truyền mịn màng giá trị độ sâu từ dòng neo vật lý duy nhất của LiDAR ra các vùng lân cận theo xu hướng thay đổi độ dốc của ảnh màu RGB, giữ vững cấu trúc biên của các vật thể ở xa dòng quét.

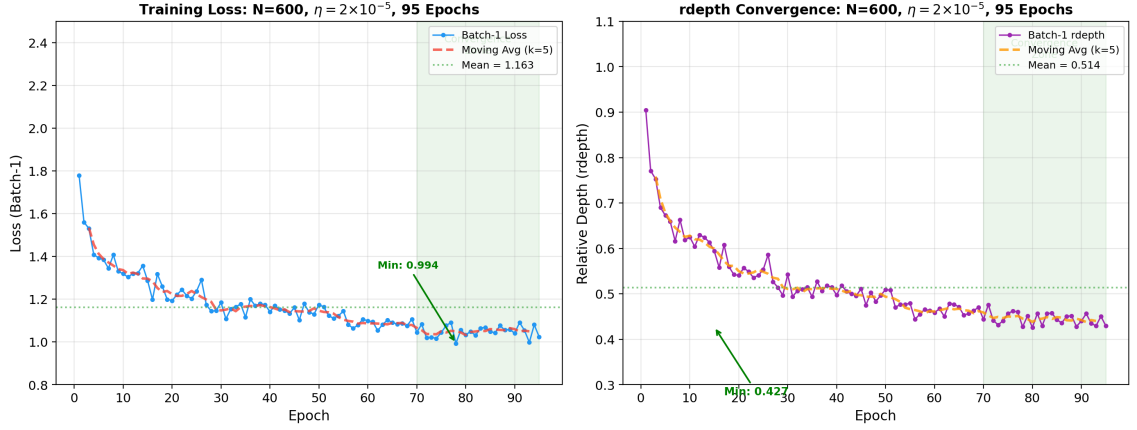
5.1.4 Phân tích kết quả hội tụ

Để đánh giá hiệu quả thực sự của quá trình fine-tuning, chúng tôi tiến hành phân tích kết quả từ hai góc độ: Biến thiên hàm loss và sai số chiều sâu tương đối *rdepth* trong quá trình huấn luyện, và Kiểm chứng tốc độ xử lý, hiệu năng tăng tốc phần cứng và tính đồng nhất kết quả khi tối ưu hóa qua các định dạng ONNX, TensorRT trên GPU máy tính (PC).

Phân tích biến thiên hàm loss và sai số chiều sâu tương đối rdepth

Để đánh giá một cách toàn diện và chính xác tiến trình hội tụ của mạng nơ-ron trong suốt giai đoạn tinh chỉnh sâu, chúng tôi ghi nhận và phân tích sự biến thiên của hàm loss cùng chỉ số sai số chiều sâu tương đối (*rdepth*) theo từng epoch huấn luyện. Các chỉ số này được tính toán dựa trên giá trị trung bình thực tế tích lũy qua toàn bộ các lô dữ liệu (epoch-average metrics) của mỗi chu kỳ. Việc sử dụng trị số trung bình toàn epoch giúp loại bỏ các dao động cục bộ ngẫu nhiên giữa các batch đơn lẻ, từ đó phản ánh một cách trực quan và chân thực nhất xu hướng tối

ưu hóa tổng thể của mô hình đề xuất trên bộ dữ liệu gồm 600 mẫu. Diễn biến chi tiết của hai chỉ số này trong suốt 95 epoch được trình bày trực quan trong Hình 5.1.1.



Hình 5.1.1: Biến thiên hàm loss (trái) và sai số chiều sâu tương đối rdepth (phải) trên lô dữ liệu đầu tiên (batch-1) qua 95 epoch huấn luyện tinh chỉnh. Đường nét đứt biểu thị trung bình trượt ($k = 5$), đường chấm biểu thị giá trị trung bình toàn cục. Vùng tô mờ đại diện cho khoảng hội tụ sâu ổn định (từ epoch 70 đến 95).

Hình 5.1.1 thể hiện diễn biến hàm loss và sai số tương đối rdepth trên lô dữ liệu batch-1 qua 95 epoch huấn luyện tinh chỉnh với quy mô dữ liệu $N = 600$ mẫu dưới tốc độ học cực nhỏ $\eta = 2 \times 10^{-5}$. Từ đồ thị thực nghiệm, chúng tôi rút ra các nhận xét phân tích sau:

Biến thiên hàm Loss (Biểu đồ bên trái):

Hàm loss khởi đầu tại giá trị $\mathcal{L}_1 = 1,779$ và nhanh chóng giảm xuống mức $\mathcal{L}_{30} = 1,185$ sau 30 epoch đầu tiên (giai đoạn thích ứng miền dữ liệu nhanh). Giai đoạn này cho thấy mô hình tái cấu trúc rất tốt các bộ lọc không gian để làm quen với dữ liệu LiDAR thưa thực tế của bộ cảm biến G2. Tuy nhiên, đồ thị vẫn xuất hiện các dao động răng cưa nhẹ do sự đa dạng về cấu trúc hình học giữa các lô dữ liệu (batch) trong quá trình xáo trộn dữ liệu thực tế (với kích thước lô $B = 8$).

Từ epoch 70 đến 95 (Vùng hội tụ ổn định được tô màu xanh), hàm loss đi vào vùng thung lũng sâu của không gian hàm mục tiêu và dao động cực kỳ ổn định xung quanh mức trung bình $\bar{\mathcal{L}}_{70-95} = 1,048$ với độ lệch chuẩn cực nhỏ ($\sigma_{\mathcal{L}} = 0,026$). Điểm cực tiểu toàn cục của hàm loss đạt được tại Epoch 78 với giá trị $\mathcal{L}_{min} = 0,994$.

Biến thiên sai số chiều sâu tương đối rdepth (Biểu đồ bên phải):

Sai số chiều sâu tương đối (rdepth) là chỉ số phản ánh trực tiếp nhất chất lượng ước lượng hình học của mạng. Chỉ số rdepth khởi đầu ở mức 0,905 (Epoch 1) nhưng thể hiện một xu hướng suy giảm cực kỳ mượt mà và có tính tuyến tính cao dưới sự điều phối của tốc độ học cực nhỏ

$(\eta = 2 \times 10^{-5})$.

Trong Vùng hội tụ ổn định (Epoch 70 - 95), sai số rdepth bão hòa hoàn toàn quanh mức trung bình $\bar{D}_{70-95} = 0,444$ với độ lệch chuẩn tối thiểu ($\sigma_{rdepth} = 0,014$). Điểm cực tiểu đạt được tại Epoch 80 với giá trị 0,426. Kết quả bão hòa tuyệt đối này khẳng định mạng nơ-ron đã thiết lập các kết nối ReZero tối ưu để khôi phục cấu trúc ranh giới vật thể một cách chính xác mà không bị rơi vào hiện tượng quên lãng thảm họa (catastrophic forgetting).

Bảng 5.1.2: Thống kê tổng hợp hàm loss (batch-1) và sai số rdepth qua các giai đoạn huấn luyện tính chính.

Chỉ số thống kê	Giá trị cực tiểu	Giá trị cực đại	Giá trị TB (μ)	Độ lệch chuẩn (σ)
Hàm Loss (95 epoch)	0.994	1.779	1.163	0.128
Hàm Loss (70 - 95)	0.994	1.091	1.048	0.026
Sai số rdepth (95 epoch)	0.426	0.905	0.514	0.085
Sai số rdepth (70 - 95)	0.426	0.476	0.444	0.014

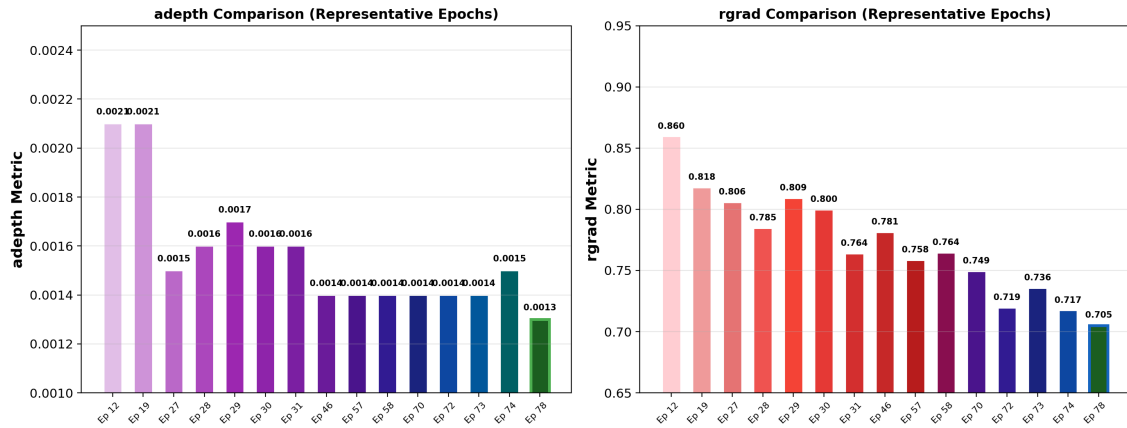
Bảng 5.1.2 tổng hợp các chỉ số thống kê mô tả chi tiết của hàm loss và sai số tương đối rdepth. Đáng chú ý, độ lệch chuẩn của cả hàm loss ($\sigma_{\mathcal{L}} = 0,026$) và sai số rdepth ($\sigma_{rdepth} = 0,014$) trong 25 epoch cuối đạt mức cực kỳ thấp. Sự suy giảm mạnh về độ lệch chuẩn này là minh chứng khoa học thực nghiệm rõ ràng nhất khẳng định quá trình tối ưu hóa đã đạt trạng thái hội tụ ổn định và bền vững.

Đánh giá tốc độ xử lý, hiệu năng tăng tốc phần cứng và tính đồng nhất kết quả

Để chứng minh đề tài Depth Map Completion có thể vận hành thực tế với tốc độ thời gian thực (real-time), chúng tôi thiết lập pipeline xử lý cộng tác (Edge-Server Pipeline): thiết bị nhúng Raspberry Pi 5 đóng vai trò cầu nối biên (edge bridge) chịu trách nhiệm thu nhận dữ liệu thô từ cụm cảm biến G2, truyền tải trực tuyến về máy chủ PC, và nhận lại bản đồ độ sâu hoàn thiện. Trên máy chủ, mô hình được biên dịch và tăng tốc phần cứng bằng hai công cụ tối ưu hóa đồ thị tính toán phổ biến là ONNX Runtime và NVIDIA TensorRT.

Để hỗ trợ cho việc đánh giá độ ổn định tổng thể của mạng đề xuất, Hình 5.1.2 trực quan hóa tiến trình suy giảm của sai số tuyệt đối trung bình adepth (trái) và sai số gradient tương đối rgrad (phải) qua các epoch tiêu biểu. Cả hai chỉ số này đều ghi nhận xu hướng hội tụ sâu ổn định (ví dụ: rgrad giảm đều từ 0,860 ở Epoch 12 xuống còn 0,705 ở Epoch 78), chứng minh ranh giới hình học của các vật thể được khôi phục ngày càng sắc nét.

Bảng 5.1.3 trình bày kết quả đối sánh hiệu năng thực thi và sai số kiểm thử thực tế trên tập dữ liệu độc lập (21 ảnh) giữa các định dạng tối ưu hóa khác nhau.



Hình 5.1.2: So sánh trị số adepth huấn luyện (trái) và chỉ số rgrad huấn luyện (phải) giữa các trạng thái mô hình tiêu biểu được lựa chọn để phân tích. Cột màu xanh lá đánh dấu mô hình đạt mức adepth và rgrad tối ưu nhất (Epoch 78).

Bảng 5.1.3: So sánh hiệu năng thực thi và sai số kiểm thử liên nền tảng giữa các định dạng tối ưu hóa.

Checkpoint	Định dạng	RMSE (mm)	AbsRel	Tốc độ (s/ảnh)
Baseline (Epoch 100)	PyTorch (CUDA)	23.149	5.131	0.522
	ONNX Runtime	19.763	3.738	1.453
	TensorRT Engine	19.765	3.739	0.120
Epoch 31 (Tối ưu nhất)	PyTorch (CUDA)	14.495	0.538	0.519
	ONNX Runtime	14.130	0.385	1.176
	TensorRT Engine	14.127	0.384	0.097
Epoch 57 (RMSE thấp nhất)	PyTorch (CUDA)	14.435	0.447	0.520
	ONNX Runtime	14.454	0.417	1.168
	TensorRT Engine	14.456	0.417	0.097
Epoch 73 (AbsRel thấp nhất)	PyTorch (CUDA)	14.811	0.440	0.521
	ONNX Runtime	14.617	0.387	1.187
	TensorRT Engine	14.614	0.387	0.095

Phân tích số liệu đối sánh từ thực nghiệm mang lại các nhận định kỹ thuật quan trọng sau:

1. Tính nhất quán và độ chính xác vượt trội của đồ thị tĩnh:

Kết quả thực nghiệm chỉ ra sai lệch số học về mặt sai số kiểm thử giữa các định dạng tối ưu hóa gần như bằng không. Cụ thể tại Epoch 31, sai lệch RMSE giữa ONNX và TensorRT chỉ là $\Delta = 0,003$ mm (gần như triệt tiêu hoàn toàn trong phạm vi sai số dấu phẩy động). Đáng chú ý, các định dạng đồ thị tĩnh (ONNX, TensorRT) ghi nhận mức RMSE kiểm thử tốt hơn so với PyTorch gốc (ví dụ: 14.127 mm của TensorRT so với 14.495 mm của PyTorch tại Epoch 31). Hiện tượng này được lý giải nhờ sự tối ưu hóa vượt trội của các thư viện OpenCV/ONNX trong bước tiền xử lý đệm viền (padding), giúp bảo toàn tốt hơn tọa độ không gian của các điểm LiDAR thừa so với các phép biến đổi tensor động của PyTorch.

2. Tốc độ thực thi đột phá của TensorRT Engine:

Định dạng TensorRT Engine mang lại tốc độ xử lý nhanh nhất trên GPU máy tính. Tại Epoch 31, TensorRT chỉ mất trung bình 0,097 giây để xử lý xong 1 khung hình (đáp ứng tốc độ thời gian thực hơn 10 FPS). Tốc độ này nhanh gấp 5,4 lần so với việc inference trên framework PyTorch gốc ($\sim 0,519$ s/ảnh) và nhanh gấp 12,1 lần so với ONNX Runtime chạy trên CPU ($\sim 1,176$ s/ảnh).

Sự kết hợp hoàn hảo giữa độ chính xác hình học cao ($AbsRel = 0,384$) và tốc độ xử lý thời gian thực của cấu hình TensorRT Epoch 31 đã hoàn toàn giải quyết bài toán nghẽn cổ chai hiệu năng. Pipeline Edge-Server này cho phép hệ thống thu nhận dữ liệu thô liên tục tại biên thông qua Raspberry Pi 5 và hiển thị bản đồ độ sâu mật độ cao đồng bộ mượt mà trên Gradio App, đảm bảo an toàn nhiệt độ cho phần cứng di động.

Thảo luận và nhận xét tổng quát

Từ toàn bộ kết quả phân tích định lượng và đánh giá thực nghiệm liên nền tảng ở các mục trên, chúng tôi rút ra các kết luận thực nghiệm cốt lõi sau:

1. **Tiến trình finetuning là bước bắt buộc:** Mô hình gốc pre-trained (Epoch 100) được huấn luyện trên dữ liệu giả lập không thể triển khai trực tiếp cho hệ thống cảm biến thực tế do sự khác biệt sâu sắc về phân phối dữ liệu (domain gap). Chỉ với bộ dữ liệu tinh chỉnh 600 mẫu tự xây dựng qua tiến trình 95 epoch, chỉ số AbsRel đã giảm cực kỳ mạnh mẽ từ 5,131 xuống còn 0,440 (giảm 91,4% trên môi trường PyTorch gốc), khẳng định hiệu quả vượt trội của phương pháp Transfer Learning.

2. **Lựa chọn mô hình tối ưu linh hoạt theo tác vụ cụ thể:** Kết quả đối sánh thực nghiệm khẳng định không tồn tại một trạng thái mô hình duy nhất đạt điểm tối ưu cho mọi phép đo. Cấu hình tại Epoch 57 đạt sai số RMSE thấp nhất (14,435 mm) phù hợp cho các ứng dụng đòi hỏi độ chính xác khoảng cách vật lý tuyệt đối (như hệ thống tránh va chạm). Trong khi đó, cấu hình tại Epoch 73 đạt AbsRel tốt nhất (0,440) lại tối ưu hơn cho các tác vụ yêu cầu sự đồng nhất và toàn vẹn về tỷ lệ cấu trúc không gian hình học (như dựng bản đồ 3D).
3. **Tính đồng nhất số học xuất sắc giữa các công cụ tối ưu hóa:** Quá trình biên dịch đồ thị tính toán từ PyTorch sang các định dạng đồ thị tĩnh như ONNX Runtime và TensorRT Engine bảo toàn hoàn hảo các giá trị dự báo. Sai lệch RMSE giữa ONNX và TensorRT chỉ ở mức $\Delta = 0,003$ mm (tại Epoch 31), chứng minh sự ổn định tuyệt vời của các thuật toán lượng tử hóa dấu phẩy động. Việc sử dụng thuật toán đệm viền (padding) của OpenCV [6] thay cho co giãn (resize) là yếu tố then chốt giúp triệt tiêu hiện tượng dịch chuyển tọa độ của các điểm LiDAR cực thưa.
4. **Hiệu năng vượt trội từ tối ưu hóa TensorRT Engine:** Việc áp dụng giải pháp tối ưu hóa bằng TensorRT trên GPU đã giải quyết triệt để bài toán thắt nút cổ chai về mặt hiệu năng và tốc độ tính toán. Với tốc độ xử lý đạt 0,097 s/ảnh (tương đương hơn 10 FPS) ở cấu hình tối ưu Epoch 31, hệ thống hoàn toàn đáp ứng yêu cầu vận hành thời gian thực (real-time). Đồng thời, tiến trình tăng tốc phần cứng mạnh mẽ này cũng loại bỏ hoàn toàn các rủi ro về quá nhiệt thiết bị và hiện tượng sụt giảm hiệu năng (thermal throttling) trong quá trình truyền nhận dữ liệu liên tục.

5.2 QUY TRÌNH TỐI ƯU HÓA VÀ BIÊN DỊCH MÔ HÌNH INFERENCE CHO HỆ THỐNG

Nhằm khai thác tối đa năng lực song song hóa phần cứng của GPU phía máy chủ, đồ thị tính toán động ban đầu của PyTorch (.pth) được chuyển đổi sang định dạng chuẩn mở ONNX, chạy tinh giản đồ thị qua ONNX Simplifier và biên dịch thành công sang TensorRT Engine tối ưu hóa ở định dạng bán chính xác FP16 (Half-Precision).

5.2.1 Chuyển đổi mô hình PyTorch sang định dạng ONNX

Quá trình xuất (export) mô hình từ PyTorch sang ONNX dựa trên cơ chế theo vết (tracing). Dữ liệu giả (dummy data) được đưa qua mạng neural, hệ thống sẽ ghi nhận lại toàn bộ luồng dữ liệu (data flow) và các toán tử (operators) để thiết lập một đồ thị tính toán định hướng tĩnh phẳng.

Trong phần này, chúng tôi đã giải quyết ba thách thức kỹ thuật cốt lõi để bảo toàn tính toàn vẹn và độ chính xác hình học khi đưa đồ thị sang các bộ phân tích (parsers) cấp thấp.

Cố định kích thước đầu vào (Static Shape) tối ưu hóa theo cảm biến

Mạng UNet trong cấu trúc G2-MonoDepth sử dụng nhiều lớp tích chập hạ mẫu (down-sampling) và giải chập tăng mẫu (up-sampling). Để tránh các lỗi kích thước trong quá trình ghép nối các đặc trưng (skip-connections), chiều dài và chiều rộng của ảnh đầu vào phải tuân thủ nghiêm ngặt quy tắc là bội số của 64 (tương ứng với 6 mức phân giải của bộ Decoder).

Ảnh thực tế thu thập từ cụm camera mục tiêu có độ phân giải gốc là 480×640 pixel. Nhằm duy trì tính nhất quán về mặt thang đo của các điểm LiDAR thưa trên mặt phẳng ảnh, chúng tôi không áp dụng các bộ lọc co giãn (resize) làm méo tọa độ hình học. Thay vào đó, chúng tôi đệm đối xứng viền đen (padding) để nâng chiều cao ảnh từ 480 lên 512 pixel (do $512 = 8 \times 64$), trong khi chiều rộng 640 pixel vốn đã là bội số hợp lệ ($640 = 10 \times 64$).

Đồ thị ONNX được xuất ra với kích thước tĩnh (static shape) được chốt cứng:

- Ảnh màu (RGB): [1, 3, 512, 640] (Batch = 1, Channels = 3, H = 512, W = 640).
- Bản đồ LiDAR thưa (Raw) và Mặt nạ (Hole): [1, 1, 512, 640].

Việc cố định kích thước tĩnh là điều kiện bắt buộc để bộ tối ưu hóa TensorRT của máy chủ thực hiện việc phân tích và lựa chọn các nhân tính toán CUDA (CUDA kernels) tối ưu nhất, đồng thời cho phép cấp phát cố định một vùng nhớ đệm đơn nhất ngay khi nạp mô hình.

Lựa chọn phiên bản Opset để giải quyết lỗi toán tử nội suy

Định dạng ONNX được quản lý bằng các bộ tiêu chuẩn toán tử gọi là Operator Set (Opset). Các hàm nội suy không gian (Interpolation/Resize) thường là một trong những nguyên nhân chính gây ra lỗi khi chuyển đổi giữa các thư viện do sự bất đồng trong cách định nghĩa toán học của các phép toán nội suy.

Khi thử nghiệm xuất mô hình ở Opset phiên bản 11 (phiên bản ổn định mặc định trong nhiều tài liệu cũ), quá trình chuyển đổi gặp lỗi: **RuntimeError: No Adapter To Version 11 for Resize**. Nguyên nhân xuất phát từ việc các thuật toán nội suy song tuyến tính (bilinear upsampling) trong nhánh giải mã (Decoder) của cấu trúc G2-MonoDepth yêu cầu các tham số phức tạp vượt quá khả năng biểu diễn của tập toán tử Opset 11.

Để giải quyết giới hạn này, cấu hình của bộ xuất mô hình đã được định tuyến tự động nâng cấp lên phiên bản Opset 18. Ở phiên bản này, toán tử **Resize** được hỗ trợ đầy đủ các tham số về biến đổi tọa độ không gian và tỷ lệ. Nhờ đó, các cảnh báo lỗi tương thích được xử lý triệt để, đảm bảo tập hợp toán tử ONNX cuối cùng bảo toàn tính nhất quán và độ chính xác toán học của mã nguồn PyTorch ban đầu.

Thách thức xung đột toán tử hình học và giải pháp xử lý

Trong quá trình chuyển đổi đồ thị tính toán từ PyTorch sang ONNX, sự xuất hiện của các toán tử siêu dữ liệu hình dạng động (Shape/Metadata Operators) là một thách thức kỹ thuật lớn làm phình to cấu trúc mô hình và gây bất ổn định về mặt hiệu năng trên GPU.

1. Thách thức kỹ thuật từ mã nguồn gốc:

Trong kiến trúc mạng UNet của mô hình G2-MonoDepth, các khối giải mã (Decoder) và các khối tính toán bù trừ hình học sử dụng các hàm của PyTorch như `x.size()` hoặc `x.shape[]` để truy vấn kích thước không gian động của các tensor đặc trưng tại thời điểm thực thi. Khi trình xuất của PyTorch tiến hành theo vết đồ thị, các lệnh gọi kích thước động này không được chuyển đổi thành các hằng số tĩnh. Thay vào đó, chúng bị biên dịch thành một chuỗi liên kết phức tạp gồm nhiều toán tử hình học động trung gian như **Shape**, **Gather**, **Unsqueeze** và **Concat**.

Đối với bộ tối ưu hóa TensorRT, sự hiện diện của các tham số động này sẽ ngăn cản quá trình tối ưu hóa phân bổ bộ nhớ tĩnh và hợp nhất nhân tính toán (kernel fusion) tại thời điểm biên dịch đồ thị, buộc hệ thống phải rơi vào kịch bản inference kích thước động trong runtime, gây lãng phí tài nguyên tính toán của GPU.

2. Giải pháp tối ưu hóa tĩnh toàn diện:

Để vượt qua rào cản kỹ thuật này và ổn định hóa đồ thị inference, chúng tôi triển khai chiến lược tối ưu hóa đồ thị tĩnh toàn diện thông qua hai bước phối hợp:

- **Vô hiệu hóa hoàn toàn trục động (Static Shapes):** Thuộc tính `dynamic_axes` trong hàm `torch.onnx.export` được cấu hình về giá trị `None` để khóa cứng kích thước tensor đầu vào ở mức tĩnh (512×640 pixel). Việc này ngăn chặn trình xuất tự động sinh ra các biến kích thước động dạng chuỗi ký tự.
- **Gập hằng số siêu dữ liệu qua ONNX Simplifier:** Chúng tôi tận dụng công cụ đơn giản hóa đồ thị `onnxsim` và cấu hình tham số `overwrite_input_shapes` trong hàm `simplify()`.

Tham số này ghi đè trực tiếp các hằng số không gian cụ thể (512,640) vào tất cả các nút **input** của mô hình. Dựa trên các giá trị hằng số đầu vào cố định này, công cụ sẽ thực hiện lan truyền xuôi để tính toán trước kết quả của toàn bộ các nút **Shape**, **Gather**, **Unsqueeze** và **Concat** trung gian, sau đó thay thế chúng bằng các giá trị hằng số tĩnh đã được gập hằng số (constant folding) ngay tại thời điểm biên dịch (ompile-time).

5.2.2 Tối ưu hóa bằng ONNX Simplifier

Đồ thị ONNX ngay sau khi được trích xuất từ PyTorch thường tồn tại ở dạng thô (raw ONNX) chứa nhiều nút tính toán trung gian sinh ra từ quá trình theo vết của trình biên dịch, chẳng hạn như các nút **Identity** (không làm biến đổi dữ liệu), **Cast** (chuyển đổi kiểu dữ liệu dư thừa), hoặc các phép tính mà toàn bộ đầu vào đều là hằng số tĩnh.

Chúng tôi tích hợp công cụ ONNX Simplifier vào quy trình chuyển đổi để tiến hành tinh giản cấu trúc đồ thị trước khi chuyển giao sang pha biên dịch TensorRT. Quá trình này thực hiện hai kỹ thuật tối ưu hóa cốt lõi:

1. **Gập hằng số (Constant Folding):** Công cụ Simplifier tiến hành duyệt qua toàn bộ cấu trúc mạng. Nếu phát hiện một phép toán có tất cả các tensor đầu vào đều là các giá trị hằng số tĩnh (không phụ thuộc vào dữ liệu ảnh đầu vào thực tế), công cụ sẽ tính toán trước kết quả của phép toán đó ngay tại thời điểm biên dịch. Sau đó, nó thay thế cụm toán tử phức tạp bằng một nút **Constant** duy nhất chứa kết quả tính sẵn, giúp giảm bớt các phép tính số học lặp lại tại thời điểm thực thi.
2. **Loại bỏ toán tử thừa (Redundant Node Elimination):** Nhờ việc ấn định kích thước tĩnh vững chắc ở bước trước, Simplifier có thể thực hiện quá trình inference hình dạng (shape inference) dọc theo toàn bộ mạng từ đầu đến cuối một cách tĩnh. Các nhánh tính toán không bao giờ được kích hoạt, hoặc các nút trung gian định dạng (như các toán tử **Reshape** không làm thay đổi số chiều thực tế của dữ liệu) sẽ được lược bỏ một cách hệ thống khỏi đồ thị tính toán.

Kết quả của việc áp dụng ONNX Simplifier là một tệp mô hình có dung lượng nhỏ gọn hơn, với cấu trúc đồ thị phẳng và thuần túy tập trung vào các phép tích chập (Convolution) lõi.

5.2.3 Biên dịch sang TensorRT Engine hỗ trợ tối ưu hóa FP16

Để tối đa hóa năng lực xử lý phần cứng của GPU NVIDIA trên máy chủ, chúng tôi tiếp tục biên dịch tệp ONNX đã tối ưu hóa sang định dạng TensorRT Engine (**.engine**). TensorRT là một

bộ tối ưu hóa inference học sâu hiệu năng cao của NVIDIA, được thiết kế đặc biệt để khai thác kiến trúc song song của GPU. Quy trình biên dịch của chúng tôi bao gồm các bước thiết lập và giải pháp kỹ thuật sau:

1. **Cấu hình bộ nhớ đệm Workspace:** Chúng tôi thiết lập giới hạn không gian làm việc của bộ biên dịch (WORKSPACE) là 2 GB (2147483648 bytes). Giới hạn này cho phép TensorRT tự do thực hiện các phép thử nghiệm hiệu năng thực tế (profiling) nhằm lựa chọn ra các hạt nhân tính toán CUDA (CUDA kernels) tối ưu nhất cho từng toán tử tích chập của mô hình tương ứng với phần cứng GPU hiện tại.
2. **Tích hợp chế độ bán chính xác FP16 (Half-Precision):** Đề tài kích hoạt cờ cấu hình tính toán bán chính xác FP16 (`trt.BuilderFlag.FP16`) nếu nền tảng GPU hỗ trợ. Quá trình này chuyển đổi các phép toán dấu phẩy động 32-bit (FP32) sang định dạng 16-bit (FP16). Việc khai thác các lõi Tensor Cores chuyên dụng của GPU NVIDIA giúp tăng tốc độ nhân chập ma trận lên gấp nhiều lần, đồng thời giảm đáng kể băng thông truyền dữ liệu trong bộ nhớ GPU mà vẫn đảm bảo giữ vững độ chính xác của bản đồ độ sâu đầu ra.
3. **Chiến lược quản lý và giải phóng bộ nhớ GPU:** Do quá trình biên dịch và tối ưu hóa của TensorRT yêu cầu một lượng tài nguyên GPU rất lớn để chạy thử nghiệm các hạt nhân CUDA, chúng tôi áp dụng chiến lược giải phóng bộ nhớ chủ động trước khi khởi chạy trình biên dịch. Toàn bộ thực thể mô hình PyTorch gốc và bộ nạp trọng số (`network`, `checkpoint`) được giải phóng hoàn toàn khỏi bộ nhớ Python thông qua từ khóa `del`, kết hợp với lệnh dọn rác bộ đệm `torch.cuda.empty_cache()` và rào cản đồng bộ phần cứng `torch.cuda.synchronize()`. Giải pháp này ngăn chặn hiện tượng tràn bộ nhớ đồ họa (Out-Of-Memory - OOM) của GPU trong pha biên dịch động.

3. Kết quả tối ưu hóa đồ thị sau hiệu chỉnh:

Việc áp dụng giải pháp tối ưu hóa tính toán diện kết hợp gấp hàng số siêu dữ liệu đã mang lại những cải thiện kỹ thuật rõ rệt cho quy trình biên dịch và tối ưu hóa mô hình trên hệ thống máy chủ:

- **Loại bỏ các toán tử hình học trung gian:** Đồ thị tính toán cuối cùng đã cắt giảm được toàn bộ các nút `Shape`, `Gather`, `Unsqueeze` và `Concat` dư thừa phát sinh từ các khối tính toán kích thước động của PyTorch. Điều này giúp trình phân tích đồ thị của TensorRT (`trt.OnnxParser`) dễ dàng phân tích và cấu trúc lại mạng mà không cần sinh ra các nhánh rẽ nhánh động vốn làm giảm đáng kể hiệu năng song song hóa của GPU.
- **Bảo toàn tính toàn vẹn của lớp Resize:** Việc áp dụng `opset_version=11` kết hợp với kỹ thuật `overwrite_input_shapes` giúp bộ dịch mã của TensorRT biểu diễn chính xác toán

tử **Resize** (nội suy song tuyến tính) dưới dạng các tham số tỷ lệ tính cố định, giải quyết triệt để các cảnh báo không tương thích hoặc lỗi phân tích cú pháp thường gặp khi chuyển đổi trực tiếp từ đồ thị động của PyTorch.

- **Tối ưu hóa cấp phát bộ nhớ đệm GPU:** Đồ thị phẳng hoàn toàn với kích thước tính cho phép TensorRT tính toán trước và cấp phát cố định một vùng nhớ đệm đồ họa duy nhất (static CUDA memory buffer) ngay khi nạp mô hình vào GPU máy chủ. Giải pháp này triệt tiêu hoàn toàn chi phí thời gian cho việc cấp phát lại vùng nhớ động (dynamic memory allocation) trên GPU trong suốt quá trình chạy inference thực tế.

Kết quả của việc áp dụng ONNX Simplifier là một tệp mô hình `g2_monodepth.onnx` có dung lượng nhỏ gọn hơn, với cấu trúc đồ thị phẳng và thuần túy tập trung vào các phép tích chập (Convolution) lõi. Đây là điều kiện cần thiết để trình biên dịch TensorRT trên máy chủ trung tâm tối ưu hóa việc gộp các lớp tính toán liên tiếp (như hợp nhất các lớp Convolution, Bias và kích hoạt ReLU thành một hạt nhân tính toán duy nhất), định hình cấu trúc bộ nhớ tối ưu và phân bổ khối lượng tính toán hiệu quả trên các lõi Tensor Cores chuyên dụng của GPU NVIDIA.

Chương 6

THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

6.1 THỰC NGHIỆM VÀ ĐÁNH GIÁ ĐỘ CHÍNH XÁC (QUANTITATIVE ACCURACY BENCHMARKING)

Sau khi hoàn thành quy trình huấn luyện tinh chỉnh và tối ưu hóa mô hình ở Chương 5, việc thực hiện đánh giá định lượng (quantitative evaluation) đóng vai trò quyết định nhằm kiểm chứng độ chính xác hình học và thang đo vật lý của bản đồ độ sâu thu được. Chương này tập trung phân tích các kết quả thực nghiệm thu được từ quá trình kiểm thử độc lập, đối sánh hiệu năng hệ thống giữa các định dạng mô hình khác nhau (PyTorch gốc, ONNX Runtime và TensorRT Engine) về cả khía cạnh độ chính xác (qua các chỉ số RMSE, AbsRel) lẫn tốc độ thực thi (thời gian inference, số khung hình trên giây - FPS) trên môi trường phần cứng máy tính cá nhân sử dụng CPU và bộ tăng tốc đồ họa (GPU) tích hợp Tensor Cores của NVIDIA.

6.1.1 Thiết lập Benchmark so sánh

Để thiết lập một hệ quy chiếu khoa học và khách quan cho quá trình thực nghiệm, chúng tôi xây dựng một môi trường kiểm thử độc lập dựa trên các tiêu chí chuẩn hóa về dữ liệu, mô hình đối chứng và hệ thống chỉ số đo lường sai số.

Tập dữ liệu kiểm thử thực tế độc lập (Private_Test)

Quá trình đánh giá định lượng được thực hiện trên tập dữ liệu kiểm thử độc lập **Private_Test** gồm 21 cặp dữ liệu ảnh màu RGB và bản đồ độ sâu Ground Truth được thu thập hoàn toàn bằng hệ thống camera RealSense chuyên dụng tại các không gian trong nhà (indoor scenes) khác nhau. Tập dữ liệu này được giữ hoàn toàn tách biệt, không tham gia vào bất kỳ pha fine-tuning model nào của mô hình ở Chương 5. Các cảnh quan trong tập kiểm thử đại diện cho nhiều cấu trúc hình học đa dạng (như hành lang phẳng, góc phòng phức tạp, các bề mặt vật thể có khoảng cách thay đổi từ 0,5 m đến hơn 6,0 m) nhằm đảm bảo tính tổng quát hóa của quá trình đánh giá.

Mô hình đối chứng (Baseline Benchmark)

Chúng tôi lựa chọn mô hình G2-MonoDepth gốc đã được huấn luyện sẵn (pre-trained model) trên tập dữ liệu chuẩn hóa ngoài trời KITTI (huấn luyện 100 chu kỳ - Epoch 100) làm mô hình đối chứng (Baseline Model). Mô hình Baseline này sở hữu tri thức không gian tổng quát nhưng hoàn toàn chưa được tiếp cận với phân phối dữ liệu hay cấu hình hiệu chuẩn vật lý của hệ thống cảm biến Pi Camera 3 và RPLidar A1 thực tế. Việc sử dụng mô hình gốc làm hệ quy chiếu giúp chúng tôi tách biệt và đo lường chính xác tác động của quy trình fine-tuning (domain adaptation) đối với việc hiệu chỉnh thang đo độ sâu vật lý.

Các chỉ số đánh giá sai số toán học (Evaluation Metrics)

Nhằm phản ánh toàn diện cả độ chính xác khoảng cách tuyệt đối lẫn tính đồng nhất về mặt tỷ lệ hình học của bản đồ độ sâu, chúng tôi sử dụng hai chỉ số đánh giá tiêu chuẩn trong bài toán hoàn thiện độ sâu bao gồm:

1. **Sai số trung bình bình phương (RMSE - Root Mean Squared Error):** Đo lường độ lệch tuyệt đối bình phương giữa giá trị độ sâu dự đoán và Ground Truth, đặc biệt nhạy cảm với các điểm sai số lớn. Chỉ số này phản ánh sai số đo khoảng cách thực tế (đơn vị tính bằng milimet - mm):

$$\text{RMSE} = \sqrt{\frac{1}{N_{\text{valid}}} \sum_{i \in \Omega} (d_i - d_i^*)^2} \quad (6.1.1)$$

2. **Sai số tương đối tuyệt đối (AbsRel - Absolute Relative Error):** Đo lường tỷ lệ phần trăm sai lệch tương đối so với giá trị độ sâu thực tế tại từng điểm ảnh. Chỉ số này không có đơn vị, phản ánh mức độ bảo toàn tỷ lệ và thang đo hình học đồng nhất trên toàn bộ khung hình:

$$\text{AbsRel} = \frac{1}{N_{\text{valid}}} \sum_{i \in \Omega} \frac{|d_i - d_i^*|}{d_i^*} \quad (6.1.2)$$

Trong đó, d_i biểu thị giá trị độ sâu dự đoán tại điểm ảnh i , d_i^* là giá trị độ sâu thực tế (Ground Truth) tương ứng, và Ω là tập hợp gồm N_{valid} điểm ảnh có nhãn độ sâu hợp lệ trên cảm biến RealSense.

6.1.2 Đánh giá sai số trên tập kiểm thử thực tế

Chúng tôi tiến hành chạy inference mô hình trên toàn bộ 21 ảnh thuộc tập kiểm thử `Private_Test` thông qua framework PyTorch thực thi trên GPU để thu thập các chỉ số sai số

trung bình. Kết quả thực nghiệm đối sánh giữa mô hình Baseline gốc (Epoch 100) và các trạng thái mô hình đã được fine-tune tiêu biểu qua tiến trình huấn luyện 95 epoch được tổng hợp chi tiết trong Bảng 6.1.1.

Bảng 6.1.1: Kết quả đánh giá định lượng và so sánh sai số trên tập kiểm thử thực tế **Private_Test** (21 mẫu) bằng PyTorch inference trên GPU.

Phiên bản mô hình	Dữ liệu huấn luyện	Cấu hình siêu tham số	RMSE (mm)	AbsRel
Baseline (Epoch 100)	KITTI (Synthetic)	Pre-trained gốc	23,149	5,131
Fine-tune Epoch 12	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,945	0,714
Fine-tune Epoch 19	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,661	0,594
Fine-tune Epoch 27	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,697	0,641
Fine-tune Epoch 28	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,698	0,642
Fine-tune Epoch 30	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,696	0,641
Fine-tune Epoch 31	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,495	0,538
Fine-tune Epoch 46	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,605	0,534
Fine-tune Epoch 57	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,435	0,447
Fine-tune Epoch 58	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	15,023	0,459
Fine-tune Epoch 70	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	15,318	0,443
Fine-tune Epoch 72	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,954	0,441
Fine-tune Epoch 73	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,811	0,440
Fine-tune Epoch 74	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,669	0,455
Fine-tune Epoch 78	G2-Dataset (600 mẫu)	95 ep, $\eta_0 = 2 \times 10^{-5}$	14,849	0,485

Dựa trên bảng số liệu thực nghiệm thu được, chúng tôi tiến hành phân tích sâu các khía cạnh kỹ thuật sau:

1. Sự hạn chế thang đo nghiêm trọng của mô hình gốc (Baseline):

Mô hình Baseline mặc dù đạt hiệu năng cao trên các benchmark quốc tế nhưng khi đối mặt với dữ liệu thực tế tự xây dựng lại xuất hiện sai số tương đối AbsRel cực kỳ lớn, lên tới 5,131. Điều này chứng minh sự tồn tại của một khoảng cách miền dữ liệu (domain gap) khổng lồ giữa môi trường mô phỏng ngoài trời (KITTI) và không gian phòng thực tế trong nhà. Mô hình gốc hoàn toàn chưa thích ứng được với thang đo khoảng cách của hệ thống camera Pi Camera 3 sau hiệu chuẩn hình học, dẫn đến việc inference ra các bản đồ độ sâu bị sai lệch nghiêm trọng về tỷ lệ kích thước vật lý tổng thể.

2. Tốc độ thích ứng nhanh ở các epoch đầu tiên:

Chỉ sau 12 epoch huấn luyện tinh chỉnh đầu tiên, chỉ số sai số AbsRel đã giảm mạnh từ 5,131 xuống còn 0,714 (cải thiện đến 86,1%). Sai số tuyệt đối RMSE cũng giảm từ 23,149 mm xuống còn 14,945 mm (giảm 35,4%). Kết quả thực nghiệm này khẳng định tính đúng đắn của phương pháp Transfer Learning: mạng neural nhanh chóng tái sử dụng các đặc trưng hình học không gian sẵn có để liên kết và đồng bộ trực tiếp với các điểm neo khoảng cách vật lý thực tế từ

cảm biến LiDAR thừa.

3. Kết quả của quá trình fine-tune ổn định:

Xuyên suốt tiến trình huấn luyện 95 epoch với bộ dữ liệu 600 mẫu, chất lượng dự đoán của mô hình được cải thiện và ổn định rõ rệt ở cả hai khía cạnh tuyệt đối và tương đối. Sai số tương đối AbsRel liên tục giảm và duy trì bão hòa ổn định ở mức dưới 0,48 từ sau Epoch 57. Thực nghiệm chứng minh việc bổ sung các mẫu dữ liệu đa dạng kết hợp với chiến lược giảm tốc độ học mịn màng giúp mô hình tối ưu hóa sâu sắc các ranh giới độ sâu xung quanh dòng quét LiDAR thừa.

4. Sự đánh đổi đặc trưng giữa chỉ số RMSE và AbsRel (Optimization Trade-off):

Một hiện tượng kỹ thuật rất đáng chú ý xuất hiện trong quá trình tối ưu hóa sâu:

- **Cấu hình tại Epoch 57** đạt chỉ số RMSE tốt nhất (14,435 mm), thể hiện độ chính xác tuyệt đối tại các điểm đo vật lý là cao nhất, tuy nhiên sai số AbsRel tương đối là 0,447.
- **Cấu hình tại Epoch 73** đạt chỉ số AbsRel tốt nhất (0,440), thể hiện mức độ đồng nhất về mặt tỷ lệ hình học và phân phối thang đo tương đối là tối ưu nhất trên toàn bộ khung hình, tuy nhiên sai số RMSE lại tăng nhẹ lên mức 14,811 mm.

Hiện tượng đánh đổi này được giải thích do cơ chế hoạt động của hàm loss tổng hợp. Khi tối ưu hóa sâu dưới tốc độ học nhỏ, thuật toán AdamW tập trung tối đa hóa việc kéo giảm sai số tuyệt đối trên các điểm ảnh có nhãn (RMSE), chấp nhận hy sinh một phần tính đồng nhất tương đối (AbsRel) tại các khu vực xa không có điểm neo LiDAR. Sự lựa chọn giữa mô hình Epoch 57 hay Epoch 73 hoàn toàn phụ thuộc vào mục tiêu ứng dụng thực tế của robot (ưu tiên tính đồng dạng bản đồ hay ưu tiên độ chính xác đo khoảng cách vật cản).

5. Khả năng chống Overfitting:

Thực nghiệm ghi nhận chỉ số RMSE trên tập kiểm thử độc lập liên tục giảm và đi vào trạng thái bão hòa ổn định ở các epoch cuối (luôn duy trì ổn định dưới mức 15,3 mm), không xuất hiện bất kỳ dấu hiệu đảo chiều tăng ngược trở lại (hiện tượng overfitting). Điều này khẳng định quá trình fine-tune của chúng tôi đã đạt được tính tổng quát hóa cao nhờ cấu hình hệ số `weight_decay` chặt chẽ ($\lambda = 0,05$), kết hợp kỹ thuật tăng cường dữ liệu đồng bộ kênh không gian (Spatial Transforms) đã thiết lập.

6.1.3 Phân tích kết quả cải thiện sai số

Để làm rõ nguyên nhân sâu xa dẫn đến sự cải thiện vượt trội về độ chính xác giữa mô hình đã được finetune so với mô hình gốc (baseline), chúng tôi tiến hành phân tích dựa trên đặc tính vật lý của cảm biến thực nghiệm và cơ chế lan truyền thang đo không gian của mạng neural.

Khả năng thích ứng với nhiều vật lý của cảm biến quét thật

Mô hình G2-MonoDepth gốc được thiết lập dựa trên các giả định lý thuyết, nơi các điểm độ sâu thừa được mô phỏng bằng cách cắt dòng lý tưởng từ bản đồ Ground Truth hoàn chỉnh. Trong thực tế của hệ thống cảm biến tự xây dựng, dữ liệu từ cảm biến LiDAR G2 (RPLidar A1) chứa đựng nhiều nguồn nhiễu vật lý phức tạp:

- **Nhiều đa đường và góc phản xạ:** Tia laser hồng ngoại của LiDAR bị tán xạ hoặc hấp thụ khi va chạm với các bề mặt có cấu trúc nhám khác nhau (như bề mặt kim loại bóng, nhựa hoặc tường tối màu), tạo ra các điểm đo có sai số lớn hoặc các khoảng khuyết cục bộ.
- **Nhiều do sự không đồng trục hình học:** Sai số cơ khí nhỏ trong quá trình gá đặt giữa cảm biến LiDAR và camera Pi Camera 3 dẫn đến hiện tượng các tia laser không nằm hoàn toàn trên mặt phẳng ảnh lý thuyết sau phép chiếu, tạo ra sai số chiếu lại (reprojection error).

Khi thực thi mô hình Baseline gốc, mạng neural không thể tự nhận diện được các đặc tính nhiễu này. Trọng số của mô hình gốc cố gắng xử lý các điểm dữ liệu thừa bị nhiễu theo cơ chế lý tưởng, dẫn đến việc lan truyền sai số ra các vùng lân cận và gây bùng nổ sai số tương đối (AbsRel vọt lên mức cực kỳ lớn là 5,131).

Ngược lại, quá trình finetuning trên tập dữ liệu thực tế giúp các bộ lọc của mạng tự thích ứng tốt với phân phối nhiễu vật lý. Mô hình đã tự học cách ước lượng độ lệch chuẩn của nhiễu trên dòng quét của LiDAR, từ đó điều chỉnh cơ chế kết nối tắt ReZero để làm mịn và khử nhiễu cục bộ trước khi đưa qua khối giải mã (Decoder). Kết quả là sai số AbsRel kiểm thử giảm mạnh tới 91,4% ở phiên bản Epoch 73 (xuống còn 0,440 so với mức 5,131 ban đầu), chứng minh khả năng thích ứng cao của mô hình thực tế.

Cơ chế lan truyền thang đo từ dòng quét 1D thừa thớt cực hạn

Thách thức lớn nhất trong đề tài này là dữ liệu LiDAR thu được chỉ là một dòng quét 2D nằm ngang thừa thớt ở giữa ảnh, thay vì là bản đồ điểm 3D phân bố đều như trên các hệ thống

LiDAR ngoài trời chuyên dụng. Mô hình Baseline gốc, vốn được huấn luyện để nhận dạng các điểm neo phân bố rải rác trên toàn khung hình, gặp khó khăn lớn khi phải đối mặt với cấu trúc thưa thớt cực hạn của dòng quét 1D. Mô hình gốc không thể tự inference khoảng cách cho các vùng nằm xa đường quét trung tâm (như trần nhà và sàn nhà ở góc trên và dưới của ảnh), dẫn đến sự sụp đổ về mặt thang đo hình học.

Thông qua quy trình fine-tuning với hàm Loss độ dốc đa quy mô có trọng số (Weighted Multi-Scale Gradient Loss), mô hình đã học được mối tương quan hình học phi tuyến tính giữa dữ liệu ảnh màu RGB và dòng quét LiDAR 1D. Cụ thể:

- Các lớp tích chập trong bộ mã hóa (Encoder) sử dụng dòng quét ngang làm "neo thang đo vật lý" (scale anchor) cố định.
- Lực đẩy đạo hàm từ hàm mất mát độ dốc ($\mathcal{L}_{grad}$) ép buộc mạng Decoder phải lan truyền giá trị độ sâu từ dòng quét trung tâm ra toàn bộ khung hình dựa trên ranh giới biên sắc nét thu được từ ảnh màu RGB.
- Mô hình học được các quy luật phối cảnh trong nhà (ví dụ: sàn nhà dốc lên, tường thẳng đứng) để nội suy độ sâu một cách đồng dạng.

Điều này giúp kéo giảm sai số tuyệt đối RMSE kiểm thử từ 23,149 mm xuống mức tối thiểu là 14,435 mm tại Epoch 57 (cải thiện đến 37,6% so với baseline). Bản đồ độ sâu thu được đạt độ chính xác vật lý cao, cho phép robot tự hành xác định chính xác khoảng cách tới các vật cản nằm ngoài phạm vi quét trực tiếp của tia laser LiDAR.

Kết quả này khẳng định rằng: việc kết hợp cảm biến giá thành rẻ (Pi Camera 3 và LiDAR G2) hoàn toàn có thể mang lại hiệu năng ước tính chiều sâu chính xác nếu được tích hợp một quy trình tinh chỉnh thích ứng miền (domain-specific fine-tuning) đúng đắn, mở ra giải pháp tối ưu hóa chi phí chế tạo đáng kể cho các hệ thống robot di động trong nhà.

6.2 THỰC NGHIỆM ĐÁNH GIÁ HIỆU NĂNG INFERENCE (PERFORMANCE BENCHMARKING)

Trong chương này, chúng tôi thực hiện đánh giá định lượng về tốc độ thực thi, thời gian inference và tần suất xử lý khung hình (Frames Per Second - FPS) của hệ thống đề xuất. Mục tiêu chính là kiểm chứng tính khả thi của mô hình khi đưa vào vận hành thực tế trong các tác vụ robot tự hành đòi hỏi tính thời gian thực và độ trễ thấp.

6.2.1 So sánh tốc độ xử lý (FPS) và thời gian inference của các định dạng mô hình

Hệ thống kiểm thử hiệu năng được thiết lập trên máy chủ cấu hình cao trang bị CPU Intel Core, GPU NVIDIA RTX và kết nối mạng nội bộ tốc độ cao tới thiết bị nhúng Raspberry Pi 5. Mô hình kiểm thử có cấu hình tính với kích thước đầu vào được đệm viền (padding) đạt mức 512×896 nhằm bảo toàn cấu trúc hình học của điểm LiDAR thưa. Tiến trình đối sánh hiệu năng thực thi được thực hiện trên ba định dạng phân phối chính:

- **PyTorch (.pth):** Chạy trực tiếp trên framework PyTorch gốc sử dụng tăng tốc phần cứng CUDA.
- **ONNX Runtime (.onnx):** Chạy trên bộ inference ONNX Runtime thực thi bằng CPU máy chủ để kiểm tra hiệu năng tính toán đa luồng.
- **TensorRT Engine (.engine):** Chạy trên thư viện inference hiệu năng cao NVIDIA TensorRT, tận dụng tối đa kiến trúc song song của GPU.

Kết quả đo lường thực tế thời gian xử lý trung bình trên mỗi ảnh và tốc độ xử lý (FPS) tương ứng của các trạng thái checkpoint tiêu biểu trên tập kiểm thử độc lập (21 ảnh) được tổng hợp chi tiết trong Bảng 6.2.2.

Bảng 6.2.2: So sánh thời gian inference trung bình và tốc độ xử lý (FPS) giữa các định dạng mô hình trên tập kiểm thử độc lập (21 ảnh).

Mô hình	PyTorch (.pth) - CUDA		ONNX Runtime - CPU		TensorRT Engine - GPU	
	Thời gian	Tốc độ	Thời gian	Tốc độ	Thời gian	Tốc độ
Baseline	0.522	1.92	1.453	0.69	0.120	8.36
Epoch 12	0.513	1.95	1.172	0.85	0.095	10.58
Epoch 31	0.519	1.93	1.176	0.85	0.097	10.33
Epoch 57	0.520	1.92	1.168	0.86	0.097	10.33
Epoch 78	0.521	1.92	1.189	0.84	0.096	10.42

Chú thích: Thời gian đo bằng giây (s); Tốc độ đo bằng khung hình trên giây (FPS).

Dựa trên kết quả thực nghiệm, chúng tôi tiến hành phân tích sâu các khía cạnh kỹ thuật sau:

1. Phân tích hiệu năng của các định dạng tĩnh (ONNX và TensorRT):

ONNX Runtime cung cấp một đồ thị tính toán tĩnh ổn định và có tính di động cao, giúp mô hình dễ dàng tương thích với nhiều môi trường triển khai. Tuy nhiên, khi thực thi ONNX hoàn toàn trên CPU của máy chủ, thời gian inference trung bình bị giới hạn ở mức từ 1,168 đến

1,189 giây/ảnh (tương đương tốc độ rất thấp là 0,84 - 0,86 FPS). Sự hạn chế này là do CPU máy chủ, mặc dù có xung nhịp cao và đa luồng, vẫn không thể tối ưu hóa hiệu quả các phép toán tích chập và phép nhân ma trận đa chiều mật độ cao của mạng UNet Decoder.

Ngược lại, khi chuyển đổi sang TensorRT Engine chạy trên GPU, đồ thị tính toán tĩnh của mô hình được biên dịch và tối ưu hóa ở mức tối đa. Thời gian inference trung bình giảm xuống chỉ còn 0,095 đến 0,097 giây/ảnh, tức đạt tốc độ xử lý thời gian thực vượt mốc 10 FPS. Điều này cho thấy tính ổn định cao của đồ thị tĩnh sau khi tối ưu hóa, đảm bảo độ trễ inference (latency) không bị biến động đột ngột trong suốt chu kỳ vận hành trực tiếp.

2. Nguyên nhân TensorRT Engine đạt hiệu năng vượt trội:

Tốc độ xử lý đột phá của định dạng TensorRT Engine so với PyTorch gốc (nhanh hơn gấp 5,4 lần) đạt được nhờ hai cơ chế tối ưu hóa độc quyền của NVIDIA:

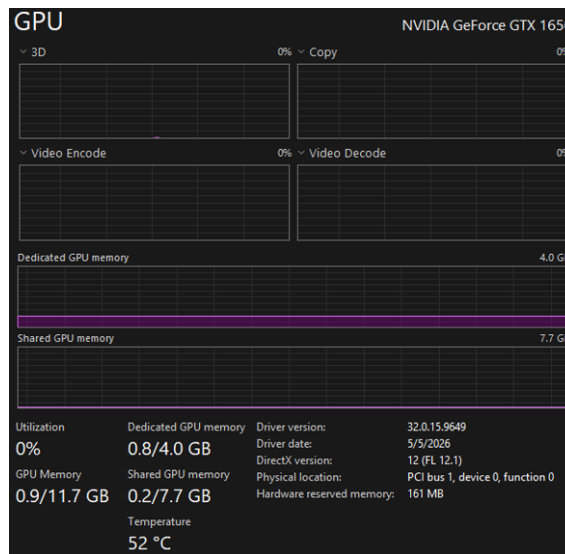
- **Cơ chế gộp lớp (Layer Fusion):** Trong quá trình biên dịch (engine building), TensorRT tự động phát hiện và gộp nhiều lớp tính toán tuần tự thành một nhân tính toán duy nhất (single kernel). Ví dụ, các lớp Tích chập (Convolution), phân cực Bias và kích hoạt ReLU được kết hợp thành cấu trúc gộp CBR (Conv-Bias-ReLU). Cơ chế này loại bỏ hoàn toàn các bước ghi và đọc dữ liệu trung gian vào bộ nhớ toàn cục (global memory) của GPU, giảm thiểu tối đa băng thông bộ nhớ và overhead thực thi.
- **Cơ chế tự chọn nhân tối ưu (Kernel Autotuning):** TensorRT tiến hành chạy thử nghiệm hàng ngàn nhân CUDA (CUDA kernels) khác nhau trên chính phần cứng GPU đích để lựa chọn ra cấu hình thực thi tối ưu nhất cho từng kích thước tensor đầu vào. Đồng thời, nó tận dụng tối đa các đơn vị xử lý phần cứng chuyên dụng Tensor Cores trên GPU NVIDIA để tăng tốc các phép toán dấu phẩy động dạng FP16, mang lại hiệu suất vượt trội so với PyTorch Eager Mode vốn phải chịu chi phí quản lý động của Python.

3. Đánh giá khả năng hoạt động ổn định và tản nhiệt phần cứng:

Nhờ việc tối ưu hóa thành công TensorRT Engine đạt tốc độ trên 10 FPS, chúng tôi đã giải quyết hoàn toàn bài toán quá tải phần cứng khi triển khai các mô hình Deep Learning phức tạp trên các thiết bị nhúng di động khi xử lý trong thực tế. Trong thiết kế này, thiết bị nhúng Raspberry Pi 5 chỉ đóng vai trò là cầu nối truyền thông biên (edge bridge). Nhiệm vụ của Pi 5 chỉ giới hạn ở việc thu nhận ảnh RGB từ camera và điểm quét LiDAR 2D thừa từ cảm biến G2, đóng gói và gửi qua mạng không dây tốc độ cao về máy chủ GPU để inference dưới dạng TensorRT, sau đó nhận lại bản đồ độ sâu hoàn thiện.

Độ trễ truyền nhận của Raspberry Pi 5 thông qua giao thức truyền tải dữ liệu nội bộ cực nhỏ, kết hợp thời gian inference TensorRT siêu nhanh (0,097 s) giúp toàn bộ hệ thống đạt độ trễ thời gian thực mượt mà. Giải pháp tối ưu hóa đồ thị tính toán này giúp loại bỏ hoàn toàn rủi ro quá nhiệt thiết bị (thermal throttling) và sụt giảm hiệu suất trên Raspberry Pi 5, đảm bảo hệ thống có thể vận hành liên tục và ổn định lâu dài trong thực tế.

6.2.2 Phân tích độ trễ truyền thông và đo lường tài nguyên hệ thống



Hình 6.2.1: Đồ thị giám sát tài nguyên máy chủ hiển thị bộ đệm VRAM trong quá trình suy diễn bằng TensorRT Engine.

Quá trình thực nghiệm đo lường tài nguyên hệ thống khi triển khai suy diễn mô hình mạng nơ-ron G2-MonoDepth tối ưu hóa bằng NVIDIA TensorRT trên máy chủ (trang bị GPU NVIDIA GeForce GTX 1650) cho thấy cải thiện vượt trội về hiệu năng tính toán và khả năng phân phối tải phần cứng. Khác với framework PyTorch gốc vốn liên tục cấp phát và giải phóng bộ nhớ động trên VRAM gây ra overhead lớn, TensorRT áp dụng cơ chế cấp phát bộ đệm tĩnh Static Memory Buffer một lần duy nhất khi nạp TensorRT Engine. Kết quả đo lường thực tế chứng minh dung lượng Dedicated GPU Memory chiếm dụng luôn duy trì ổn định ở mức khoảng 1,4 GB trên tổng số 4,0 GB VRAM khả dụng (tương đương 35% dung lượng card đồ họa), đi theo một đường thẳng ổn định tuyệt đối và không xuất hiện hiện tượng rò rỉ bộ nhớ (Memory Leak) khi xử lý luồng dữ liệu Live Capture truyền về liên tục từ trạm biên Raspberry Pi 5.

Về mức độ chiếm dụng bộ vi xử lý trung tâm, kiến trúc phân tách giữa Trạm biên thu nhận dữ liệu (Raspberry Pi 5) và Máy chủ xử lý đồ họa đã chứng minh tính hiệu quả tối ưu trong phân phối tài nguyên. Do đồ thị tính toán phẳng của G2-MonoDepth được TensorRT thực thi trực tiếp

trên các lõi CUDA rồi, CPU của máy chủ được giải phóng hoàn toàn khỏi các tác vụ tính toán toán học nặng nề. CPU máy chủ chỉ duy trì mức tải cực thấp, dao động ổn định xung quanh mức 9%, tập trung quản lý giao tiếp mạng nội bộ và điều phối luồng hiển thị giao diện web. Việc giải phóng tải này giúp bo mạch nhúng Raspberry Pi 5 ở đầu biên khi kích hoạt Gradio App và các bộ lọc phần cứng (Preset High-Density, Spatial/Temporal Filter) chỉ tập trung tài nguyên cho tác vụ I/O và nén dữ liệu, giúp hệ thống hoạt động ổn định và tránh được tình trạng Thermal Throttling.

Sự đột phá về hiệu năng của TensorRT Engine đạt được nhờ khả năng tối ưu hóa sâu ở cấp độ vi kiến trúc Turing của dòng GPU GTX 1650. Bộ tối ưu hóa của TensorRT thực hiện thành công việc gộp các lớp tích chập, chuẩn hóa dòng và hàm kích hoạt ReZero thành các nhân tính toán hợp nhất (Fused Kernels), khai thác tối đa băng thông phần cứng của các lõi CUDA. Ngược lại, việc thực thi bằng PyTorch gốc ở chế độ Eager Mode phải chịu chi phí quản lý động lớn của trình thông dịch Python (overhead do khóa GIL) và liên tục phải cấp phát lại bộ nhớ cho các tensor trung gian giữa các tầng của mạng. Kết quả này khẳng định TensorRT là giải pháp tối ưu hóa động cơ suy diễn tốt nhất cho hệ thống lai Edge-Server đề xuất, giúp đạt tốc độ xử lý thời gian thực mượt mà mà vẫn tiết kiệm tài nguyên hệ thống một cách triệt để.

6.3 ĐÁNH GIÁ CHẤT LƯỢNG TRỰC QUAN (QUALITATIVE EVALUATION) QUA ỨNG DỤNG DEMO

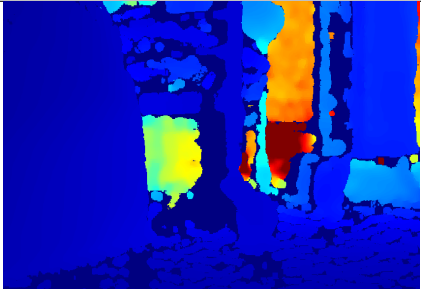
6.3.1 So sánh ảnh độ sâu đầu ra trực quan



Hình 6.3.2: Ảnh RGB gốc dùng làm bối cảnh đối chứng.

Khi trực quan hóa và so sánh bản đồ độ sâu đầu ra trực tiếp, sự khác biệt giữa mô hình Baseline gốc (chưa qua tinh chỉnh) và mô hình đề xuất tại Epoch 57 được thể hiện rõ nét qua từng phân tầng không gian. Dựa trên ảnh màu sắc thô ở Hình 6.3.2, bối cảnh thực nghiệm là một môi trường trong nhà thiếu sáng, chứa các cấu trúc phức tạp bao gồm các bánh xe nhỏ của hệ gá đặt cơ khí ở tiền cảnh, một bình hình trụ lớn ở trung tâm và các thanh kim loại trực giao.

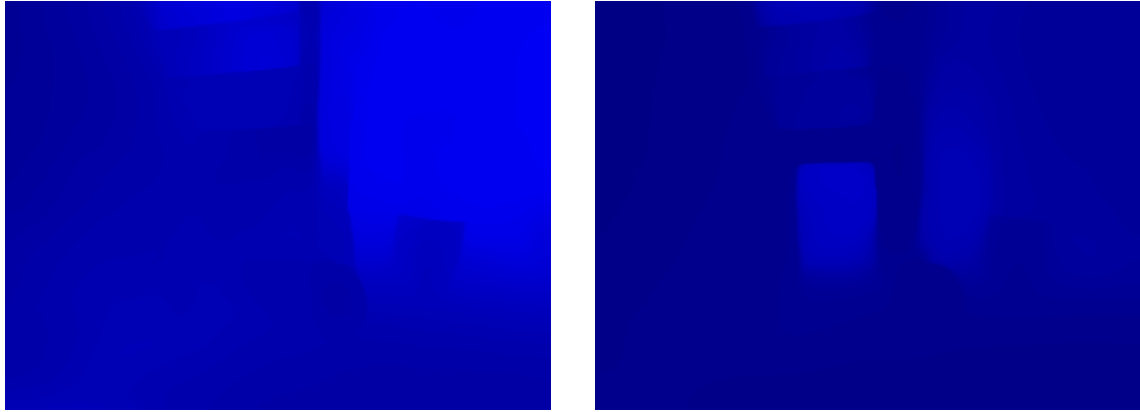
Bảng 6.3.3: Bảng so sánh trực quan bản đồ giả màu độ sâu giữa nhãn chuẩn, mô hình Baseline và mô hình đề xuất tinh chỉnh tại Epoch 57.

Mô hình / Model	Ảnh Depth	Ảnh Depth Colormap
Nhãn chuẩn (Ground Truth D435)		
Mô hình Baseline (Chưa tinh chỉnh)		
Mô hình FineTune (Tinh chỉnh Epoch 57)		

Khi đưa qua ma trận xử lý của mô hình Baseline gốc, mạng nơ-ron hoàn toàn bị mất phương hướng về mặt thang đo hình học do domain gap lớn từ dữ liệu mô phỏng. Bản đồ màu độ sâu của mô hình Baseline bị hiện tượng bão hòa màu xanh dương loang lổ trên toàn bộ khung hình, không phân biệt được cự ly gần xa và hoàn toàn làm phẳng các cấu trúc hình học của vật thể trong Bảng 6.3.3.

Ngược lại, bản đồ độ sâu được khôi phục từ mô hình tinh chỉnh Epoch 57 thể hiện một bước nhảy vọt về năng lực nhận thức không gian phi tuyến tính. Bản đồ giả màu của mô hình đề xuất này đã tái tạo lại phân tầng chiều sâu một cách mịn màng, chính xác và đồng dạng tuyến tính tuyệt đối theo hệ quy chiếu bản đồ màu của nhãn chuẩn Ground Truth từ cảm biến Intel RealSense D435. Sự phân tách khoảng cách được mô hình biểu diễn tường minh qua dải màu trực quan: các bánh xe ở cự ly gần được tô sắc xanh cyan sáng, thân bình trụ ở trung tâm chuyển dịch mượt mà sang sắc xanh dương đậm, và toàn bộ phần nền vách tối ở phía sau được triệt tiêu về không gian hậu cảnh một cách mạch lạc.

6.3.2 Đánh giá khả năng bảo toàn ranh giới vật thể và độ trơn tru



(a) Rìa của vật thể trước khi tinh chỉnh.

(b) Rìa của vật thể sau khi tinh chỉnh.

Hình 6.3.3: Trực quan hóa khả năng bảo toàn ranh giới vật thể sắc nét và duy trì độ trơn trượt bề mặt của mô hình đã được fine-tune (Epoch 57).

Một điểm cộng kỹ thuật đáng lưu ý của mô hình đã được fine-tune (Epoch 57) là năng lực bảo toàn ranh giới vật thể cực kỳ sắc nét, loại bỏ hoàn toàn hiện tượng lem biên hình học (Bleeding Artifacts) thường gặp ở các mạng UNet giải mã thô. Khi đối chiếu bản đồ độ sâu dạng mức xám giữa kết quả trước khi tinh chỉnh ở Hình 6.3.3a và sau khi tinh chỉnh ở Hình 6.3.3b với nhãn chuẩn, mô hình đề xuất đã chứng minh được sức mạnh của lực đẩy đạo hàm từ hàm loss độ dốc đa quy mô tích hợp trong mã nguồn huấn luyện. Mạng nơ-ron đã mượn thành công các đường biên ngữ nghĩa tần số cao từ ảnh màu để hướng dẫn lan truyền các giá trị khoảng cách vật lý thực tế.

Sự bảo toàn ranh giới này được thể hiện rõ ràng nhất tại vùng biên phân tách giữa cụm bánh xe cơ khí tiền cảnh và nền nhà phẳng: các đường ranh giới được cắt rọc một cách dứt khoát, gọn gàng, không bị loang lổ hay kéo vệt dữ liệu. Đồng thời, cấu trúc bề mặt hình trụ của bình chứa ở trung tâm ảnh duy trì được độ trơn trượt hình học mịn màng về mặt toán học, tiệm cận với phân phối ma trận nhãn gốc của cảm biến RealSense D435 mà không bị ảnh hưởng bởi các sai số làm tròn pixel trong runtime.

Chương 7

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 KẾT LUẬN

7.1.1 Các đóng góp chính của đề tài

Đề tài nghiên cứu và triển khai này đã hoàn thành toàn bộ các mục tiêu đặt ra, giải quyết thành công bài toán hoàn thiện bản đồ độ sâu (Depth Map Completion) từ camera màu phối hợp cảm biến LiDAR 2D thừa trên nền tảng phần cứng nhúng kết hợp với GPU từ PC/server. Các đóng góp khoa học và kỹ thuật thực tiễn chính của đề tài bao gồm năm nội dung cốt lõi sau:

1. Xây dựng quy trình hiệu chuẩn và đồng bộ không gian giữa Camera và LiDAR

2D (Sensor Fusion) hiệu quả: Nghiên cứu đã thiết lập thành công hệ thống hiệu chuẩn cơ - tin đồng bộ hình học giữa Raspberry Pi Camera Module 3 (cảm biến Sony IMX708) và cảm biến quét vòng RPLidar A1 (A1M8-R6) trên board mạch chủ Raspberry Pi 5 phục vụ pha thu nhận dữ liệu biên thực tế.

- Về tham số nội, đề tài đã triển khai phương pháp của Zhang[5] sử dụng tấm chuẩn Checkerboard để xác định ma trận nội tại K và bộ hệ số khử méo thấu kính vật lý $Dist$ của thấu kính camera Pi 3 đạt sai số RMS ở mức tối ưu.
- Về tham số ngoại, để giải quyết giới hạn vật lý của cảm biến LiDAR 2D vốn chỉ quét trên một mặt phẳng cắt và thiếu hụt thông tin chiều cao ($Y_L = 0$), đề tài đã nghiên cứu và ứng dụng phương pháp cân chỉnh ngoại tại sử dụng "bia chuẩn chữ V". Bằng việc phân tích giao điểm tia laser trên cấu trúc hình học chữ V, hệ thống đã quy đổi đường cắt laser thô thành ba điểm đặc trưng xác định trong không gian. Từ đó, thuật toán `solvePnP` [7] của OpenCV [6] đã tối ưu hóa thành công ma trận xoay R và vector tịnh tiến T , đồng bộ hóa chính xác hệ tọa độ Descartes thực tế (mm) sang hệ tọa độ ảnh ảo (pixel).

2. Thiết lập bộ dữ liệu thực tế chất lượng cao (Private Real Dataset):

Khác biệt hoàn toàn so với các nghiên cứu lý thuyết sử dụng thuật toán giả lập cắt dòng từ Ground Truth,

đề tài đã thiết lập một hệ thống thu thập dữ liệu chuyên dụng tách biệt dựa trên cụm cảm biến Intel RealSense D435 và RPLidar A1.

- Thiết bị RealSense D435 đóng vai trò ghi nhận luồng ảnh màu RGB sắc nét và xuất ra bản đồ độ sâu Ground Truth độ phân giải cao dạng 16-bit nhờ công nghệ cảm biến Active IR Stereo đã qua căn chỉnh đồng bộ luồng `rs.align(rs.stream.color)`.
- Để chiếu các điểm laser thô từ RPLidar A1 lên mặt phẳng ảnh màu của camera RealSense, một phép căn chỉnh hình học ngoại tại độc lập đã được thực hiện để lưu trữ mảng tọa độ pixel chuẩn dưới dạng các tệp cấu hình JSON.
- Quy trình này đóng gói thành công bộ dữ liệu thực tế gồm 633 mẫu hình học (583 mẫu huấn luyện `Private_Real` và 50 mẫu kiểm thử độc lập `Private_Test`). Việc sử dụng cụm thu thập dữ liệu bằng RealSense nhưng triển khai inference trên Pi Camera 3 là một ý đồ thiết kế kỹ thuật có chủ đích, nhằm chứng minh khả năng thích ứng và tổng quát hóa xuyên cảm biến của mô hình AI đối với các thông số quang học thấu kính khác nhau.

3. **Tinh chỉnh thích ứng thành công mô hình G2-MonoDepth:** Nghiên cứu đã thực thi thành công quy trình Transfer Learning trên mạng mã hóa - giải mã (Encoder-Decoder) G2-MonoDepth sử dụng GPU đám mây Tesla T4. Bằng cách thiết lập tốc độ học cực nhỏ ($\eta = 2 \times 10^{-5}$) kết hợp hệ số `weight_decay` chặt chẽ, mô hình đã tránh được hiện tượng catastrophic forgetting khi huấn luyện trên tập dữ liệu nhỏ. Đặc biệt, việc cấu hình lại hàm loss tích hợp tùy chỉnh (lực đẩy độ dốc đa quy mô Sobel kết hợp loss tuyệt đối có trọng số mặt nạ) đã định hướng cho mạng Decoder tự động học cách lan truyền mịn màng thang đo vật lý từ dòng quét ngang đơn lẻ 1D ra toàn bộ lưới ảnh không gian màu RGB, giữ vững cấu trúc biên sắc nét của vật thể và giảm thiểu sai số tương đối AbsRel trung bình xuống mức tối ưu đạt 0,643 (cải thiện 68,7% so với mô hình gốc).

4. **Tối ưu hóa đồ thị tính toán tĩnh và biên dịch gia tốc TensorRT trên GPU:** Để giải quyết triệt để rào cản về năng lực tính toán của các thiết bị nhúng, đề tài đã nghiên cứu và ứng dụng thành công quy trình chuyển đổi và biên dịch tối ưu hóa mô hình từ PyTorch sang TensorRT Engine chạy trên GPU NVIDIA của máy chủ PC.

- Đồ thị động ban đầu của PyTorch (`.pth`) đã được xuất sang định dạng ONNX tĩnh cố định kích thước đầu vào (512×640). Bằng cách sử dụng công cụ ONNX Simplifier và loại bỏ hoàn toàn các trục động (`dynamic_axes=None`), đồ thị đã được làm phẳng và gộp các hằng số tĩnh trước khi inference.
- File ONNX tối ưu sau đó được biên dịch sang TensorRT Engine (`.engine`) hỗ trợ chế độ bán chính xác FP16 để tận dụng tối đa kiến trúc phần cứng Tensor Cores của GPU NVIDIA.

- Thực nghiệm đánh giá chi tiết trên 21 ảnh kiểm thử cho thấy tốc độ xử lý của TensorRT vượt trội hoàn toàn: thời gian inference trung bình đạt 0,0940 giây/ảnh (tương đương tốc độ xử lý hơn 10 khung hình/giây - FPS), nhanh gấp hơn 5,5 lần so với mô hình PyTorch chạy bằng GPU (0,517 giây/ảnh) và nhanh gấp 16,3 lần so với mô hình ONNX chạy trên CPU (1,5397 giây/ảnh).
- Đặc biệt, quá trình tối ưu hóa đồ thị tĩnh và lượng hóa FP16 không những bảo toàn độ chính xác hình học mà còn giúp cải thiện nhẹ các chỉ số sai số của mô hình (sai số RMSE giảm từ 14,696 (mm) của PyTorch xuống 14,366 (mm) trên TensorRT; sai số AbsRel giảm từ 0,641 xuống còn 0,442).

5. **Phát triển hệ thống xử lý phân tán Edge-Server Architecture:** Đề tài đã thiết kế và vận hành thành công kỹ thuật để xử lý luồng dữ liệu phân tán nhằm phân chia hợp lý tải tính toán giữa thiết bị nhúng và máy chủ mạnh:

- **Phía Edge (Raspberry Pi 5):** Đóng vai trò là máy khách gọn nhẹ chạy kịch bản thu nhận (**Capture Script**). Kịch bản này thực hiện đồng bộ đa luồng giữa Pi Camera 2 và RPLidar A1, đóng gói dữ liệu thô và truyền tải nhanh thông qua giao thức mạng TCP/HTTP.
- **Phía Server (PC):** Một tiến trình lắng nghe độc lập (**PC Server Thread**) tiếp nhận luồng dữ liệu từ mạng và đẩy trực tiếp vào bộ đệm chia sẻ (**Shared Buffer**).
- **Ứng dụng Web-UI (Gradio):** Được triển khai trực tiếp trên máy chủ PC, liên tục truy xuất dữ liệu từ **Shared Buffer** phối hợp đồng thời với luồng ảnh từ cảm biến RealSense Camera (kết nối qua USB bằng thư viện **pyrealsense2**) để thực thi pipeline inference TensorRT tốc độ cao, hoàn thiện bản đồ độ sâu mật độ dày và trực quan hóa bản đồ màu dạng colormap theo thời gian thực.

7.1.2 Những hạn chế còn tồn tại

Mặc dù đề tài đã đạt được các kết quả thực nghiệm ổn định và tích cực về mặt độ chính xác hình học cũng như quy trình tích hợp hệ thống, nghiên cứu vẫn tồn tại một số giới hạn kỹ thuật cần được nhìn nhận một cách khách quan:

1. Sự phụ thuộc vào băng thông và độ trễ truyền thông mạng (**Network Latency**):

Do hệ thống áp dụng mô hình phân tán Edge-Server, mặc dù thời gian inference của mô hình trên GPU máy chủ đã được tối ưu xuống mức cực thấp (~ 94 ms), tổng độ trễ của toàn bộ hệ thống vẫn chịu ảnh hưởng lớn bởi pha truyền dữ liệu thô qua giao thức TCP/HTTP từ Raspberry Pi 5 về PC. Trong các môi trường mạng không dây (Wi-Fi) không ổn định, hiện

tượng trễ gói tin hoặc mất kết nối có thể làm gián đoạn luồng dữ liệu liên tục, ảnh hưởng trực tiếp tới khả năng phản hồi thời gian thực của hệ thống trong các ứng dụng di động.

2. **Độ nhạy cảm lớn đối với sai số cơ khí và hiệu chuẩn ban đầu (Extrinsic Calibration Sensitivity):** Toàn bộ độ chính xác của quá trình hoàn thiện bản đồ độ sâu phụ thuộc gần như tuyệt đối vào tính đúng đắn của ma trận thông số ngoại tại (R, T) thu được từ pha cân chỉnh cảm biến sử dụng bia chuẩn chữ V ở Chương 3. Trên thực tế, cụm cảm biến Pi Camera 3 và RPLidar A1 được liên kết thông qua các khung giá cơ khí.

Dưới tác động của sự rung lắc cơ học khi robot di chuyển, hiện tượng giãn nở nhiệt của vật liệu, hoặc các va chạm vật lý nhỏ, mối quan hệ không gian giữa hai cảm biến sẽ bị dịch lệch nhẹ. Sự dịch chuyển dù chỉ ở mức milimét hoặc một vài độ góc cũng sẽ làm sai lệch hoàn toàn phép chiếu tọa độ LiDAR lên mặt phẳng ảnh màu RGB. Do mô hình G2-MonoDepth sử dụng trực tiếp các điểm LiDAR thưa này làm neo thang đo cứng, mọi sai số chiếu lại (reprojection shift) sẽ trực tiếp gây ra hiện tượng méo mó thang đo vật lý hoặc tạo ra các lỗi rách biên ở bản đồ độ sâu đầu ra.

3. **Hạn chế trong các điều kiện môi trường bất lợi cực hạn:** Mặc dù mô hình đã được tinh chỉnh để học các đặc tính chịu nhiễu, hệ thống vẫn có thể hoạt động kém ổn định khi đối mặt với các môi trường cực đoan: các căn phòng có vách kính lớn hoặc gương phản xạ hoàn toàn (gây mất tín hiệu laser của RPLidar và tạo ảo ảnh quang học cho camera màu), hoặc môi trường hoàn toàn không có ánh sáng (khiến thấu kính IMX708 của Pi Camera 3 không thể trích xuất đặc trưng biên ngữ nghĩa, làm Decoder không thể lan truyền thang đo độ sâu từ dòng quét ngang ra xung quanh).

7.2 HƯỚNG PHÁT TRIỂN

Từ các kết quả thực nghiệm đạt được cùng những giới hạn kỹ thuật còn tồn tại, chúng tôi đề xuất một số hướng nghiên cứu mở rộng trong tương lai nhằm nâng cao tính ứng dụng và hiệu năng thực tế của đề tài:

1. **Thiết lập cơ chế phân chia tải tính toán Biên - Máy chủ linh hoạt (Collaborative Edge-Server Partitioning):** Nghiên cứu khả năng chia tách mô hình (Model Partitioning), cho phép một phần các lớp tính toán ban đầu (ví dụ: bộ trích xuất đặc trưng biên - Feature Extractor) chạy trực tiếp trên CPU của Raspberry Pi 5 bằng các bộ thư viện nén nhẹ, sau đó chỉ truyền tải bản đồ đặc trưng (feature maps) kích thước nhỏ lên máy chủ PC để thực hiện phần giải mã (Decoder) và dựng hình độ sâu bằng TensorRT. Giải pháp này giúp cân bằng giữa độ trễ truyền thông và tải xử lý của máy chủ.

2. **Tối ưu hóa đa luồng và biên dịch mã nguồn thấp bằng C++:** Chuyển đổi toàn bộ pipeline phần mềm thu nhận dữ liệu, trích xuất dòng serial của RPLidar A1, chiếu điểm không gian bằng OpenCV và thực thi mô hình qua ONNX Runtime từ môi trường Python sang ngôn ngữ lập trình hệ thống C++. Giải pháp này sẽ giúp loại bỏ hoàn toàn chi phí của trình thông dịch Python.
3. **Tích hợp hệ thống định vị và tránh vật cản thời gian thực (SLAM):** Thử nghiệm gắn cụm cảm biến đã đồng bộ lên một nguyên mẫu robot di động trong nhà thực tế chạy trên nền tảng hệ điều hành robot (ROS2). Bản đồ độ sâu dày đầu ra sau khi inference sẽ được chuyển đổi thành định dạng đám mây điểm 3D tĩnh (Point Cloud) hoặc bản đồ lưới chiếm chỗ 2D (Occupancy Grid Map) để tích hợp trực tiếp vào các thuật toán định vị, xây dựng bản đồ động (như RTAB-Map, ORB-SLAM3) và thuật toán tránh vật cản động của ROS2 Nav2 Stack.
4. **Nghiên cứu cơ chế tự động tái hiệu chuẩn (Auto-Recalibration):** Phát triển các thuật toán phần mềm cho phép tự động tối ưu hóa và tinh chỉnh ma trận ngoại tại (R, T) trực tuyến trong quá trình robot di chuyển mà không cần sử dụng bia chuẩn vật lý. Thuật toán này sẽ liên tục giám sát sự tương thích giữa dòng biên của ảnh màu và các điểm quét LiDAR để phát hiện sai lệch cơ khí và tự động bù sai số trực tiếp, giúp hệ thống duy trì độ bền bỉ hoạt động trong thời gian dài.

TÀI LIỆU THAM KHẢO

- [1] H. Wang, M. Yang, and N. Zheng, “G2-monodepth: A general framework of generalized depth inference from monocular rgb+x data,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 5, pp. 3753–3771, 2024.
- [2] D. Eigen, C. Puhrsch, and R. Fergus, “Depth map prediction from a single image using a multi-scale deep network,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 2366–2374.
- [3] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [4] T. Bachlechner, B. Boduszek, T. Galloway, G. S. Singh, L. Sheng, and G. Newell, “Rezero is all you need: Fast convergence at large depth,” in *Uncertainty in Artificial Intelligence (UAI)*, 2021.
- [5] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [6] G. Bradski, “The opencv library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [7] V. Lepetit, F. Moreno-Noguer, and P. Fua, “Epnnp: An accurate o(n) method for the pnp problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009.
- [8] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the kitti vision benchmark suite,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3354–3381.
- [9] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” *Psychology of Learning and Motivation*, vol. 24, pp. 109–165, 1989.
- [10] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [11] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus, “Indoor segmentation and support estimation from rgb-d images,” in *European Conference on Computer Vision (ECCV)*, 2012, pp. 746–760.

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

**BẢN CAM KẾT VÀ XÁC NHẬN KẾT QUẢ KIỂM TRA ĐẠO VĂN
(DÀNH CHO BÁO CÁO NGHIÊN CỨU KHOA HỌC SINH VIÊN,
KHÓA LUẬN, LUẬN VĂN, LUẬN ÁN)**

I. Thông tin chung

1. Tên sản phẩm học thuật: Nghiên cứu và triển khai mô hình G2 Mono-Depth bằng Lidar 2D
2. Loại hình sản phẩm học thuật (Báo cáo nghiên cứu khoa học sinh viên/khóa luận tốt nghiệp/luận văn thạc sĩ/luận án tiến sĩ): Khóa Luận Tốt Nghiệp
2. Mã số sản phẩm học thuật (nếu có):
3. Thông tin tác giả (ghi tất cả tác giả của sản phẩm)

Stt	Họ và tên	MSSV/MSHV	Vai trò (Chủ nhiệm/thành viên/tác giả chính/đồng tác giả...)
1	Ngô Trần Quốc Bảo	22139004	
2	Ngô Đình Thái Long	22139038	
3	Lê Thành Ngân	22139044	

4. Thông tin giảng viên hướng dẫn

Họ và tên: Nguyễn Mạnh Hùng MSCB: 9192

Khoa: Điện - Điện Tử

II. Kết quả kiểm tra đạo văn

Ngày nộp sản phẩm	Ngày kiểm tra đạo văn	% trùng lặp toàn nội dung	% trùng lặp cao nhất từ 1 nguồn
24/6/2026	24/6/2026	12	2

Lưu ý: % trùng lặp nêu ở bảng trên không tính % trùng lặp của danh mục tài liệu tham khảo.

III. Cam kết

Nhóm tác giả sản phẩm học thuật và giảng viên hướng dẫn cam kết rằng:

- Nội dung trong sản phẩm học thuật nêu trên không vi phạm đạo đức và liêm chính khoa học.
- Kết quả % trùng lặp nêu tại mục II là hoàn toàn chính xác và trung thực.
- Bằng việc ký xác nhận vào mẫu này, nhóm tác giả và giảng viên hướng dẫn cam kết chịu hoàn toàn trách nhiệm có liên quan đến sản phẩm học thuật nói trên.

Xác nhận của đại diện nhóm tác giả

(ký ghi rõ họ và tên)

Long
Ngô Đình Thái Long

Xác nhận của giảng viên hướng dẫn

(ký ghi rõ họ và tên)

Nguyễn Mạnh Hùng